

Gradient Boosting and the Price of Interpretability: XGBoost to Predict Credit Card Defaults

Phase 2 — 3031ICT Data Mining. Title page not counted toward the 10-page limit.

1. Introduction

Retail credit lending presents an asymmetric cost structure. The two errors a lending model can make are not equivalent: the unrecovered capital from extending credit to a client who defaults substantially exceeds the forgone net interest margin from declining a client who would have repaid (Elkan, 2001). Any evaluation that treats these errors as interchangeable — accuracy being the obvious offender — is therefore measuring the wrong quantity. Compounding this, the domain is regulated: automated lending decisions in Australia must rest on logic that is transparent and contestable by the applicant (*National Consumer Credit Protection Act 2009* (Cth)). A lender consequently needs two properties at once, and they pull in opposite directions — a prediction function accurate enough to price risk, and a decision process explainable enough to defend.

Phase 1 established the baseline for that tension on the UCI *Default of Credit Card Clients* dataset (Yeh & Lien, 2009), using directly interpretable models whose decision logic can be read off the fitted object. Phase 2 deliberately crosses to the other side of the trade-off. **XGBoost** (Chen & Guestrin, 2016) is an ensemble of gradient-boosted decision trees: rather than fitting one readable rule set, it fits hundreds of shallow trees sequentially, each correcting the residual error of those before it. On heterogeneous, dense and noisy tabular data of exactly this kind, gradient-boosted trees remain competitive with or superior to deep learning while requiring a fraction of the tuning effort (Shwartz-Ziv & Armon, 2022), and the credit-scoring benchmark literature reaches the same conclusion for this application domain specifically (Lessmann et al., 2015).

The cost of that capability is transparency. A Phase 1 decision tree can be printed as a rule set and audited line by line. A tuned XGBoost model here comprises 499 trees; its decision function is a sum over hundreds of piecewise-constant surfaces and is not readable in any practical sense. The additive structure that makes boosting accurate is precisely what destroys direct rule-based interpretability. This report therefore pairs the model with **SHAP** (Lundberg & Lee, 2017; Lundberg et al., 2020), a post-hoc attribution method whose tree-specific variant computes exact Shapley values for an ensemble in polynomial time, decomposing any individual score into per-feature contributions that sum back to it.

This report documents the implementation, theoretical basis, predictive performance and post-hoc interpretability of an XGBoost classifier applied to the Yeh and Lien (2009) dataset, in order to answer the two questions posed in Phase 1:

1. Do the marginal performance gains achieved by a non-linear ensemble architecture justify the corresponding loss of direct rule-based interpretability?
2. Can post-hoc explainability techniques bridge that gap, providing decision auditability sufficient for a regulated lending context?

Scope note. Consistent with the single-algorithm requirement, XGBoost is the only algorithm *implemented and tuned* in this report. The majority-class, logistic-regression, decision-tree and random-forest entries in §3.1 are untuned `scikit-learn` reference baselines included solely to locate XGBoost on a capability axis; they are not contributions of this phase.

2. Solution

2.1 Existing methods and prior implementations

Yeh and Lien (2009) introduced this dataset and compared six classifiers — K-nearest neighbour, logistic regression, discriminant analysis, naïve Bayes, a neural network and a classification tree — concluding that the neural network led on their area-ratio criterion. Their study predates the gradient-boosting era entirely, which leaves an obvious gap: the dataset's canonical benchmark has never been evaluated against the model family that currently dominates tabular classification.

That family's direct ancestor is Friedman's (2001) gradient boosting machine, which recast boosting as gradient descent in function space: each new weak learner is fitted to the negative gradient of the loss with respect to the current ensemble prediction. Friedman's formulation uses only first-order information and controls complexity implicitly, through tree depth and shrinkage. Johnson and Zhang (2014) later showed with regularized greedy forest that penalising the ensemble structure *explicitly* improves generalisation, but their coordinate-descent procedure requires fully re-optimising leaf weights and is prohibitively slow. Chen and Guestrin's (2016) contribution was to obtain the benefit of the explicit penalty without the cost, by taking a second-order approximation of the loss, which admits a closed-form optimal leaf weight and hence a closed-form split score.

The broader credit-scoring literature supports the choice empirically rather than theoretically. Lessmann et al. (2015) benchmarked 41 classifiers across eight retail credit datasets and found heterogeneous ensembles consistently at the top of the ranking, with single classifiers — including the neural network favoured by Yeh and Lien — materially behind. They also recommend AUC over accuracy for this task, a recommendation followed here, extended with PR-AUC on the grounds that precision–recall curves are the more informative summary under class imbalance (Davis & Goadrich, 2006; Saito & Rehmsmeier, 2015).

On the interpretability side, Lundberg and Lee (2017) unified six prior attribution methods under Shapley values from cooperative game theory, and Lundberg et al. (2020) gave TreeSHAP, an exact polynomial-time algorithm for tree ensembles. This matters for the present argument: for an arbitrary model, Shapley values must be approximated by sampling, and an approximate explanation is weak grounds for an adverse-action notice. For a tree ensemble the explanation is exact, which is the specific reason the accuracy–interpretability trade-off may be recoverable here and not in general. The cost-sensitive framing is taken from Elkan (2001), whose result that the optimal decision threshold for a calibrated classifier under a cost ratio r is $1/(1 + r)$ is used in §3.2 as an independent check on an empirically selected threshold.

2.2 The tool: the XGBoost package for Python

XGBoost is a distributed, open-source gradient-boosting library originally developed by Tianqi Chen at the University of Washington, with bindings for C++, Python, R, Java, Scala and Julia (XGBoost Developers, 2024). This project uses the Python package (xgboost 3.x, installed from PyPI). Several production-grade gradient-boosting libraries exist; Table 1 records why this one was selected for this problem.

Table 1 — Candidate gradient-boosting libraries and the basis for selection.

Library (PyPI)	Origin	Distinguishing mechanism	Verdict
xgboost	Chen & Guestrin (2016)	Level-wise growth, sparsity-aware split finding, explicit L1 and L2 penalties in the objective	Selected. Direct control of γ , λ and α makes the regularisation study of §2.5 possible
lightgbm	Ke et al. (2017), Microsoft	Leaf-wise growth, GOSS row sampling, exclusive feature bundling	Speed advantages are asymptotic; at $29,944 \times 23$ they do not materialise, and leaf-wise growth overfits more readily at this size
catboost	Prokhorenkova et al. (2018), Yandex	Ordered boosting, ordered target statistics for categoricals	Headline advantage is high-cardinality categorical data; here the widest categorical is EDUCATION at four levels
scikit-learn HistGradientBoostingClassifier	Pedregosa et al. (2011)	Histogram binning, LightGBM-inspired	Exposes <code>l2_regularization</code> only — no γ or L1 control, so most of the search space in §2.5 is unreachable

2.3 How the algorithm works

XGBoost predicts by summing the outputs of K regression trees — an additive model in which each tree is fitted to the residual error of its predecessors:

$$\hat{y}_i = \phi(x_i) = \sum_{k=1}^K f_k(x_i), \quad f_k \in \mathcal{F}$$

where $\mathcal{F} = \{f(x) = w_q(x)\}$ is the space of regression trees. The structure function q maps an input vector x_i to a leaf index within a tree of T leaves, and w is the vector of leaf weights assigned to those partitions.

Regularised objective. To balance empirical loss against model complexity, XGBoost minimises

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

Gamma (γ) is a fixed structural cost per leaf and therefore acts as a pruning threshold: a candidate split is retained only if its empirical gain exceeds γ . Lambda (λ) applies L2 shrinkage to leaf weights, smoothing the fitted values and curbing variance under noisy features. The practical advantage over regularized greedy forest, which used a comparable penalty, is that this particular specification admits a closed-form leaf score, and hence far cheaper optimisation (Johnson & Zhang, 2014; Chen & Guestrin, 2016).

Second-order optimisation. Because the objective's arguments are functions, it cannot be minimised by ordinary gradient descent in Euclidean space. Instead, at round t the model greedily adds the tree that most improves the objective, which is approximated by a second-order Taylor expansion:

$$\mathcal{L}^{(t)} \simeq \sum_i \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

with $g_i = \partial l / \partial \hat{y}^{(t-1)}$ and $h_i = \partial^2 l / \partial \hat{y}^{(t-1)2}$, both evaluated at the previous round's prediction. Retaining the curvature term h_i makes each round a Newton–Raphson step in function space rather than a first-order one, so the ensemble converges in fewer rounds on this highly non-convex landscape.

Optimal leaf weights and the split criterion. For a fixed structure the objective is separable and strictly convex in w , so setting the first partial derivative to zero gives the optimal weight and, substituting back, the structure score:

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \quad \tilde{\mathcal{L}}^{(t)}(q) = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T$$

where $G_j = \sum g_i$ and $H_j = \sum h_i$ summed over the instances in leaf j . Enumerating all tree topologies is infeasible, so XGBoost grows greedily from the root, scoring each candidate split by the loss reduction

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

and discarding any split with non-positive gain, which post-prunes automatically. To preserve generalisation, newly added trees are scaled by a shrinkage factor η and rows/columns are randomly subsampled before split search. These — γ , λ , α , η , subsample and colsample — are exactly the levers tuned in §2.5.

2.4 Core functions of the library

The implementation uses nine methods of the `xgboost` package. Six are reached through the scikit-learn-compatible `XGBClassifier` wrapper; the remaining three require dropping to the native `Booster` object underneath.

Table 2 — XGBoost API surface used, and the purpose of each call.

Method call	Used in	Purpose
<code>XGBClassifier(**params)</code>	<code>src/models.py</code>	Constructor. Carries all hyperparameters — note <code>early_stopping_rounds</code> and <code>eval_metric</code> moved here from <code>.fit()</code> in XGBoost 2.0
<code>.fit(X, y, eval_set=, verbose=)</code>	<code>src/runner.py</code>	Training. <code>eval_set</code> is the slice early stopping monitors
<code>.predict_proba(X)[: , 1]</code>	<code>src/runner.py</code> , <code>src/tuning.py</code>	Default probability — the input to every metric and to threshold selection
<code>.predict(X, output_margin=True)</code>	<code>experiments/exp11_shap.py</code>	Raw log-odds, needed for the SHAP additivity

Method call	Used in	Purpose
		check
.best_iteration	src/runner.py, exp02	The round count early stopping actually selected (449, against a 2,000 ceiling)
.get_booster()	src/models.py, exp12	Escape hatch to the native API
Booster.get_score(importance_type=)	exp12_stability.py	gain / cover / weight / total_gain importances, compared against SHAP in §3.4
Booster.get_dump()	src/models.py	Per-tree text; its length is the tree count used as the interpretability cost
Booster.save_model()	exp02_tuning.py	Persists the tuned model as JSON so downstream experiments reuse one fitted object

2.5 Our implementation

The pipeline is organised so that no experiment constructs an estimator or a CV loop of its own — configurations that drift apart are the usual source of numbers that cannot be reconciled later. All estimators come from one factory, and all evaluation goes through one function.

Data and splits. The 30,000 raw rows contain 108 duplicates across 52 groups, 21 of them with contradictory labels; de-duplication precedes the split, leaving 29,944 rows and 6,622 defaults (22.11% prevalence) over 23 features. An 80/20 stratified hold-out (random_state=42) yields 23,955 training and 5,989 test rows, the latter containing 1,324 positives. The hold-out is read exactly once, in exp14_final.py, after every modelling choice has been frozen.

The model factory. Every XGBoost variant in the report is built by this one function, so the fixed infrastructure settings cannot diverge between arms:

```
# src/models.py
def xgb_classifier(params=None, spw=1.0, early_stopping=True,
                  enable_categorical=False, seed=SEED):
    """XGBoost with the project's fixed infrastructure settings."""
    p = dict(XGB_FIXED) # tree_method="hist", eval_metric="auc", n_jobs=-1
    p["enable_categorical"] = enable_categorical
    p["random_state"] = seed
    p["scale_pos_weight"] = spw
    if early_stopping: # constructor, NOT .fit(), since XGBoost 2.0
        p["n_estimators"] = N_ESTIMATORS_CAP # 2000 is a ceiling; ES picks the real count
        p["early_stopping_rounds"] = ES_ROUNDS
    if params:
        p.update(params)
    return XGBClassifier(**p)
```

Two settings are pinned rather than tuned. tree_method="hist" selects the histogram split-finder described in §2.3. enable_categorical is forced False because xgboost 3.4 defaults it to True, and shap 0.52 reads that flag alone when deciding whether a model contains categorical splits — leaving it on silently blocks interventional TreeSHAP and would have invalidated §3.4.

The single evaluation path. cv_evaluate fits the factory on every fold and reports mean ± SD across fits. Crucially, the early-stopping eval_set is carved out of each fold's *training* half, so the validation fold is never used to choose the round count and the hold-out is never seen at all:

```
# src/runner.py
def fit_one(est, X_tr, y_tr, seed=SEED):
    """Fit one estimator, carving the early-stopping slice out of the training data."""
    if _uses_early_stopping(est):
        i_fit, i_es = train_test_split(
            np.arange(len(y_tr)), test_size=ES_SLICE, stratify=y_tr, random_state=seed)
        est.fit(X_tr.iloc[i_fit], y_tr[i_fit],
               eval_set=[(X_tr.iloc[i_es], y_tr[i_es])], verbose=False)
    else:
        est.fit(X_tr, y_tr)
    return est

def cv_evaluate(X, y, factory, folds, seed=SEED, collect_oof=True):
    y = np.asarray(y).astype(int)
    per_fold, oof_sum, oof_n = [], np.zeros(len(y)), np.zeros(len(y))
```

```

for tr, va in folds:
    est = fit_one(factory(), X.iloc[tr], y[tr], seed=seed)
    proba = est.predict_proba(X.iloc[va])[:, 1] # never .predict()
    per_fold.append(_fold_metrics(y[va], proba))
    oof_sum[va] += proba
    oof_n[va] += 1
summary = {"f{m}_{s}": fn([f[m] for f in per_fold])
           for m in ("roc_auc", "pr_auc", "brier")
           for s, fn in (("mean", np.mean), ("sd", lambda v: np.std(v, ddof=1)))}
return {"cv": summary, "per_fold": per_fold,
        "oof": np.where(oof_n > 0, oof_sum / np.maximum(oof_n, 1), np.nan)}

```

Hyperparameter selection uses `StratifiedKFold(5, shuffle=True, random_state=42)` on the training rows only; the variance reported in §3.1 uses `RepeatedStratifiedKFold(5 × 5, random_state=7)`, a different seed so the 25 reported fits are not correlated with the folds the tuner optimised against.

Hyperparameter search. Optuna's TPE sampler (Akiba et al., 2019) searches the eight dimensions of §2.3 with a median pruner, reporting after each fold so hopeless trials are abandoned early:

```

# src/tuning.py
def make_objective(X, y, folds, spw=1.0, seed=SEED, space=None):
    def objective(trial):
        params, aucs = suggest(trial, space), []
        for k, (tr, va) in enumerate(folds):
            est = fit_one(xgb_classifier(params, spw=spw, seed=seed),
                          X.iloc[tr], y[tr], seed=seed)
            aucs.append(roc_auc_score(y[va], est.predict_proba(X.iloc[va])[:, 1]))
            trial.report(float(np.mean(aucs)), step=k) # lets MedianPruner cut early
            if trial.should_prune():
                raise optuna.TrialPruned()
        return float(np.mean(aucs))
    return objective

```

`scale_pos_weight` is deliberately excluded from the space: ROC-AUC is invariant to monotone score transformations, so the tuner would be blind to it and choose arbitrarily. Fixing it per arm turns the class-imbalance study into a controlled comparison instead of a confound. `n_estimators` is likewise not searched — early stopping determines it. 150 trials (142 complete, 8 pruned) ran in 1,166 s at ≈2.3 s per trial. Ranges and selected values are given in Table 3.

Table 3 — Optuna search space and selected hyperparameters (150 TPE trials).

Parameter	Range	Scale	Selected
max_depth	3 – 8	linear	6
eta	0.01 – 0.30	log	0.0125
min_child_weight	1.0 – 20.0	log	4.19
subsample	0.60 – 1.00	linear	0.604
colsample_bytree	0.50 – 1.00	linear	0.800
gamma	0.0 – 5.0	linear	0.695
reg_lambda	0.01 – 100	log	34.3
reg_alpha	0.001 – 10	log	0.0015
n_estimators	capped 2,000	—	449 (early stopping)

Cost-sensitive decision rule. The cost ratio is anchored in credit-risk parameters rather than chosen for convenience: a false negative costs $\text{LGD} \times \text{EAD}$ (unsecured retail loss-given-default ≈ 0.70) and a false positive costs the forgone net interest margin (≈ 0.12), giving $r = 0.70/0.12 \approx 5.83$, reported as 6:1. The operating threshold is selected on out-of-fold training predictions and then frozen — sweeping a threshold against test labels is the single most common fatal error in this task:

```

# src/costs.py
def elkan_p_star(r):
    """Elkan (2001): acting is optimal when p * C_FN > (1 - p) * C_FP, i.e. p > 1/(1+r)."""
    return 1.0 / (1.0 + r)

def pick_threshold_oof(y_oof, proba_oof, r=R_HEADLINE, ead=None, grid=None):
    """Cost-minimising threshold chosen on OOF training predictions, then frozen."""
    grid = threshold_grid() if grid is None else np.asarray(grid, dtype=float)
    costs = np.array([cost_at_threshold(y_oof, proba_oof, t, r=r, ead=ead) for t in grid])
    return float(grid[int(np.argmin(costs))]), {...}

```

Reproducibility. Seeds are fixed in `src/config.py`; every experiment writes a JSON result file recording its seed, library versions, runtime and configuration, and `experiments/verify.py` re-checks the headline figures against those files. The full listing is in Appendix B.

3. Key Results and Metrics

This section reports the metrics committed to in Phase 1: ROC-AUC and PR-AUC across a complexity ladder, the formalised cost matrix and the threshold it fixes, and the confusion matrix and total expected cost on the untouched hold-out. It then tests whether SHAP recovers auditable decision logic.

3.1 Discrimination across the complexity ladder

Table 4 reports eight model configurations evaluated on the same 25 fits. "Read" is the interpretability cost: the number of parameters, rules or nodes a human must read to reconstruct one decision.

Table 4 — Discrimination and calibration across the complexity ladder (5 × 5 repeated stratified CV, n = 23,955).

Rung	Model	ROC-AUC	PR-AUC	Brier	Acc @ 0.5	Read
0	Majority class	0.5000 ± 0.0000	0.2212	0.2212	0.7788	1
1	PAY ₀ , depth-1 tree	0.6441 ± 0.0075	0.3774	0.1460	0.8197	1
1b	PAY ₀ as a raw score	0.6910 ± 0.0083	0.4394	—	—	1
2	Logistic regression	0.7248 ± 0.0086	0.5034	0.1448	0.8094	24
3	Decision tree (depth-tuned)	0.7542 ± 0.0103	0.5062	0.1383	0.8195	109
4	Random forest	0.7641 ± 0.0096	0.5374	0.1384	0.8183	692,674
5	XGBoost, library defaults	0.7619 ± 0.0080	0.5293	0.1415	0.8185	100
6	XGBoost, tuned	0.7845 ± 0.0071	0.5602	0.1348	0.8195	499

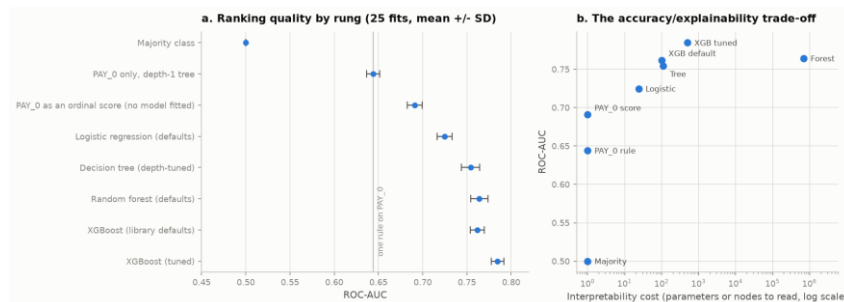


Figure 1. ROC-AUC per rung (left) and ROC-AUC against interpretability cost on a log axis (right).

Accuracy is measurably deceptive here, as Phase 1 anticipated. It spans 0.7788–0.8197 across the whole ladder while ROC-AUC spans 0.5000–0.7845: a metric that cannot separate a one-line rule from a tuned ensemble is not measuring capability. Rung 1b — an unfitted raw feature used directly as a score — beats the depth-1 tree by +0.0469 ROC-AUC. Untuned XGBoost does not beat the random forest, so XGBoost's advantage here is a *tuning* advantage, not an architectural one; what it does win on is cost, since the forest needs 692,674 nodes against 499 trees for comparable discrimination. Tuning is the single largest gain in the table, +0.0226 against a pooled fold-SD of 0.0107 (≈2.1 SD).

PR-AUC widens the spread, as predicted. It runs 0.2212 → 0.5602 against a 0.2212 base rate — a 2.53× lift over chance — where ROC-AUC's 0.5000 → 0.7845 is only 1.57× over its own floor. Rung 1 → rung 6 is +0.1828 PR-AUC against +0.1404 ROC-AUC: the abundance of true negatives really was flattering the ROC reading. Figure 1 also shows the first step is by far the most valuable, PAY₀ alone delivering 0.6910 of the eventual 0.7845.

3.2 The cost matrix and the operating threshold

Table 5 — Cost matrix. Costs are in units of forgone margin; $r = C_{FN} / C_{FP} = 6$.

	Predicted repay	Predicted default
Actual repay	TN — cost 0	FP — cost 1 (forgone margin)
Actual default	FN — cost 6 (unrecovered balance)	TP — cost 0

Table 6 — Threshold sweep on out-of-fold predictions. P^* is the Elkan (2001) analytic optimum $1/(1+r)$.

r	Elkan P^*	Empirical τ	Min cost	Cost at $\tau = 0.5$	Saving	Recall	Precision
1	0.5000	0.5360	4,286	4,306	20	0.335	0.699
2	0.3333	0.3180	7,189	7,711	522	0.529	0.561
5	0.1667	0.1781	12,978	17,926	4,948	0.719	0.408
6	0.1429	0.1341	14,239	21,331	7,092	0.829	0.333
10	0.0909	0.0841	16,575	34,951	18,376	0.947	0.267
20	0.0476	0.0541	18,058	69,001	50,943	0.990	0.236

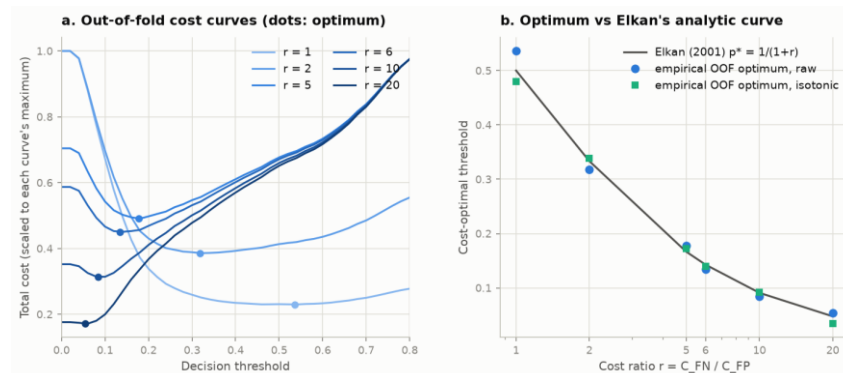


Figure 2. Out-of-fold cost against threshold, one curve per cost ratio with the cost-minimising point marked (left); empirical optima against Elkan's analytic curve (right).

The threshold is where the money is. At $r = 6$ the sweep selects $\tau = 0.1341$, and using it instead of 0.5 cuts out-of-fold cost from 21,331 to 14,239 — a 33.2% reduction obtained by changing one number, against +0.0226 ROC-AUC for the entire 150-trial search.

Two conditions make that trustworthy. First, τ is selected on out-of-fold *training* predictions only and then frozen. Second, Elkan's derivation assumes true posteriors, so it applies only to a calibrated model — and this one is, predicting a mean 0.2189 against a 0.2212 base rate with ECE 0.0212, which is not the default expectation for boosted trees (Niculescu-Mizil & Caruana, 2005). Mean absolute deviation between the empirical τ and $1/(1+r)$ is 0.0141 raw and 0.0079 after isotonic calibration; calibrating *tightens* the agreement, which makes the correspondence confirmatory rather than coincidental.

3.3 Hold-out confusion matrices and total expected cost

The hold-out (5,989 rows, 1,324 positive) was read once, after every choice above was frozen.

Table 7 — Hold-out results at the frozen threshold ($n = 5,989$; 1,324 positives).

Configuration	τ	ROC-AUC	PR-AUC	TN	FP	FN	TP	Cost = 6·FN + FP
PAY_0 single rule	0.0001	0.6430	0.3763	0	4,665	0	1,324	4,665
XGBoost, defaults	0.0881	0.7607	0.5270	1,974	2,691	191	1,133	3,837
XGBoost, tuned	0.1341	0.7829	0.5638	2,355	2,310	206	1,118	3,546
Tuned, class-reweighted	0.3580	0.7829	0.5622	2,469	2,196	219	1,105	3,510
Tuned, isotonic-calibrated	0.1401	0.7810	0.5469	3,127	1,538	363	961	3,716
Tuned, no PAY_0	0.1381	0.7519	0.5057	2,099	2,566	207	1,117	3,808
Tuned, no demographics	0.1341	0.7806	0.5614	2,321	2,344	210	1,114	3,604

The headline reaches 0.7829 against 0.7845 in CV — inside one fold-SD — catching 1,118 of 1,324 defaults. The same model at $\tau = 0.5$ costs 5,336, so the frozen threshold cuts cost by 33.5%, closely matching the 33.2% predicted out-of-fold; that agreement is the strongest single piece of evidence that the leakage protocol held. The trade can be read directly: the tuned model accepts 2,310 false alarms to avoid all but 206 of the 1,324 defaults, because $206 \times 6 = 1,236$ against 2,310 is where the sweep settled.

The isotonic arm is the instructive counter-example. It is the most *accurate* row in the table, cutting false positives by 772, yet it costs 170 more, because it converts those false positives into 157 additional missed defaults. Accuracy and cost move in opposite directions here, which is the whole argument for the cost matrix.

Paired DeLong tests (DeLong et al., 1988) give +0.1399 ROC-AUC [95% CI +0.1268, +0.1531], $p = 1.5 \times 10^{-96}$, for the tuned model against the single rule, and +0.0222 [+0.0145, +0.0300] against library defaults — six times more of the climb comes from being an ensemble at all than from being a well-tuned one.

3.4 Post-hoc interpretability: SHAP

Table 8 — Top ten features by mean |SHAP|, with cross-measure ranks and five-seed stability.

#	Feature	mean SHAP	Share	Rank by gain	Rank by permutation	Rank SD, 5 seeds
1	PAY_0	0.3780	22.3%	1	1	0.00
2	LIMIT_BAL	0.2078	12.3%	8	2	0.00
3	BILL_AMT1	0.1254	7.4%	10	3	0.45
4	PAY_AMT2	0.1087	6.4%	7	22	0.55
5	PAY_AMT1	0.0966	5.7%	9	16	1.41
6	PAY_2	0.0907	5.3%	2	11	1.10
7	PAY_3	0.0837	4.9%	3	9	1.00
8	PAY_AMT3	0.0816	4.8%	11	8	0.55
9	PAY_4	0.0609	3.6%	4	5	0.55
10	PAY_AMT6	0.0524	3.1%	15	23	2.77

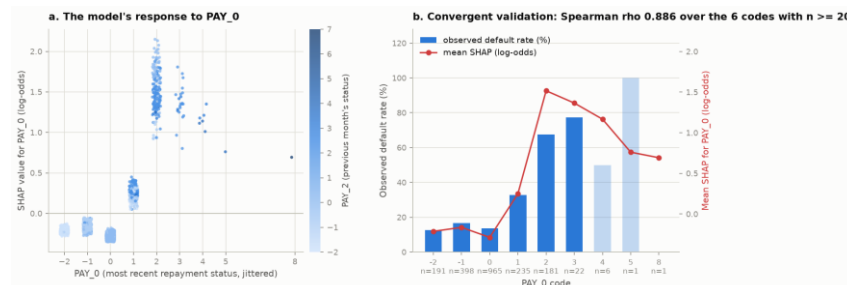


Figure 3. SHAP value for PAY_0 by observed code, coloured by PAY_2 (left); mean SHAP per code against observed default rate (right).

The six repayment-status columns take 40.2% of total attribution against 16.5% for the six BILL_AMT columns. The attribution also validates convergently: mean SHAP per PAY_0 code runs $-0.214, -0.165, -0.288, +0.250, +1.517, +1.365$ for codes -2 to 3 , so code 0 is the most negative of any code — inverting the intuitive ordering — and rank correlation with observed default rate is $\rho = 0.886$. Over five reseeded refits, Spearman ρ between mean |SHAP| vectors averages 0.9375 with top-10 Jaccard 0.8727, and PAY_0 and LIMIT_BAL never move. Instability concentrates exactly where collinearity predicts: mean rank SD is 1.73 across BILL_AMT1–BILL_AMT6 against 1.16 elsewhere.

TreeSHAP's additivity guarantee holds numerically — reconstructed margins match `predict(output_margin=True)` to a maximum error of 3.7×10^{-6} — so an individual decision can be decomposed into contributions that provably sum to the score. That is sufficient to answer "why was this applicant declined". Two limits travel with it: SHAP explains the model, not the borrower, and the collinear BILL_AMT block is defensible only as a group total.

A fairness probe closes the section. Removing all four demographic columns costs only 0.0013 ROC-AUC, yet 58.0% of the male–female selection-rate gap survives, and the model *amplifies* the disparity it does reproduce — a 3.44 pp observed gap becomes an 8.72 pp selection gap. EDUCATION fails the four-fifths rule outright (selection ratio 0.535). Blinding the model to a protected attribute therefore does not blind it to the attribute's correlates (Barocas & Selbst, 2016).

4. Summary

This project implemented a single algorithm — XGBoost, via the Python `xgboost` package — on the UCI *Default of Credit Card Clients* dataset, to test whether the performance gain of a non-linear ensemble justifies its loss of transparency, and whether post-hoc explanation can recover that transparency.

Answering the research questions. On Q1, the gain is real but stops early. Moving from one readable rule on `PAY_0` to tuned gradient boosting buys +0.1399 ROC-AUC, yet 58% of that climb arrives by the linear rung and the final step from library defaults to a tuned model is worth only +0.0222. The loss of interpretability is therefore justified *conditionally*: it is worth paying when the downstream decision needs calibrated probabilities — as the cost-threshold result in §3.2 does — because a single rule cannot produce them at all. On Q2, SHAP bridges the gap partially. It is exactly additive (max error 3.7×10^{-6}) and independently recovers the Phase 1 `PAY_0` non-monotonicity from the model's internals, but attribution inside the collinear `BILL_AMT` block is unstable (rank SD 1.73), so those six columns are auditable only as a group.

What worked. The complexity ladder was the right instrument; it converted the accuracy–explainability trade-off from an assertion into a measured axis with an explicit cost on each side. Freezing $\tau = 0.1341$ from out-of-fold probabilities and applying it untouched gave a 33.5% hold-out cost reduction against the 33.2% predicted in CV — the agreement between those two numbers is the clearest evidence the leakage protocol held.

What didn't work. Every intervention aimed at class imbalance failed, informatively. Class reweighting moved ROC-AUC by +0.0003 against a fold-SD of 0.0071 while degrading ECE from 0.0212 to 0.2088. SMOTE (Chawla et al., 2002) was worse still, at 0.7681 ROC-AUC with Brier 0.2200. Isotonic calibration also failed, worsening hold-out Brier on a model that was already near-calibrated. The common cause is that the imbalance here is mild (22%) and the threshold, not the training distribution, was the correct lever all along.

What surprised me, in descending order of impact on my view:

1. *Threshold selection is worth more than the entire hyperparameter search.* 150 Optuna trials bought +0.0222 ROC-AUC; choosing τ from the cost matrix cut cost by a third.
2. *A model that cannot see sex still reproduces most of the sex disparity.* Deleting the four demographic columns costs 0.0013 ROC-AUC, yet 58.0% of the selection-rate gap survives — and the model amplifies it, 3.44 pp observed becoming 8.72 pp selected.
3. *`PAY_0 = 0` is the strongest negative signal, not `PAY_0 = -2`.* Mean SHAP by code recovered the Phase 1 non-monotonicity from the model's own internals, without being told to look for it.
4. *The Bayes-floor argument had to be withdrawn.* I expected Phase 1's 21 contradictory duplicate groups to set an irreducible error floor; after de-duplication that contribution is exactly zero, so the ceiling claim now rests on the learning curve instead. Plateauing near 0.78 is a feature-set limitation, not a model deficiency — six months of billing history cannot observe a job loss.
5. *XGBoost's default weight importance ranks `PAY_0` twelfth while SHAP ranks it first.* The out-of-the-box importance plot would have inverted this report's central finding.

Limitations and further work. The dataset is a single 2005 Taiwanese cohort, so temporal and geographic transfer is untested; the 6:1 cost ratio is an industry-typical estimate rather than a figure from a specific lender's book; and the fairness audit is diagnostic only — no mitigation was applied. The natural extensions are a cost-weighted objective using `LIMIT_BAL` as an exposure proxy, and a constrained model that enforces monotonicity in `PAY_0` to make the decision logic defensible as well as explainable.

5. References

- Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (pp. 2623–2631). Association for Computing Machinery. <https://doi.org/10.1145/3292500.3330701>
- Barocas, S., & Selbst, A. D. (2016). Big data's disparate impact. *California Law Review*, 104(3), 671–732. <https://doi.org/10.15779/Z38BG31>
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). Association for Computing Machinery. <https://doi.org/10.1145/2939672.2939785>
- Davis, J., & Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning* (pp. 233–240). Association for Computing Machinery. <https://doi.org/10.1145/1143844.1143874>

- DeLong, E. R., DeLong, D. M., & Clarke-Pearson, D. L. (1988). Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach. *Biometrics*, 44(3), 837–845. <https://doi.org/10.2307/2531595>
- Elkan, C. (2001). The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference on Artificial Intelligence* (pp. 973–978). Morgan Kaufmann.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232. <https://doi.org/10.1214/aos/1013203451>
- Johnson, R., & Zhang, T. (2014). Learning nonlinear functions using regularized greedy forest. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 36(5), 942–954. <https://doi.org/10.1109/TPAMI.2013.159>
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30* (pp. 3146–3154). Curran Associates.
- Lessmann, S., Baesens, B., Seow, H.-V., & Thomas, L. C. (2015). Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research. *European Journal of Operational Research*, 247(1), 124–136. <https://doi.org/10.1016/j.ejor.2015.05.030>
- Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., & Lee, S.-I. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1), 56–67. <https://doi.org/10.1038/s42256-019-0138-9>
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30* (pp. 4765–4774). Curran Associates.
- National Consumer Credit Protection Act 2009 (Cth) (Austl.).
- Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning* (pp. 625–632). Association for Computing Machinery. <https://doi.org/10.1145/1102351.1102430>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems 31* (pp. 6638–6648). Curran Associates.
- Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3), e0118432. <https://doi.org/10.1371/journal.pone.0118432>
- Shwartz-Ziv, R., & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84–90. <https://doi.org/10.1016/j.inffus.2021.11.011>
- XGBoost Developers. (2024). *XGBoost: Scalable and flexible gradient boosting*. <https://xgboost.ai/>
- Yeh, I.-C., & Lien, C.-H. (2009). The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications*, 36(2), 2473–2480. <https://doi.org/10.1016/j.eswa.2007.12.020>

Appendix

Does not count toward the 10-page limit.

Appendix A — Reproduction manifest

Environment: Python 3.14, xgboost 3.x, scikit-learn, shap 0.52, optuna, imbalanced-learn. Full pins in requirements.txt. Global seed 42 (src/config.py); repeated-CV seed 7.

Module map.

File	Responsibility
src/config.py	Seeds, paths, cost constants, SEARCH_SPACE, imbalance-arm registry
src/data.py	Loading, cleaning, de-duplication, feature engineering, splits, fold generation
src/models.py	Model factory — every estimator in the report is built here
src/runner.py	cv_evaluate, the single evaluation path; result-file writer
src/tuning.py	Optuna objective and study runner
src/metrics.py	evaluate, Wilson intervals, Brier, reliability curves, fast DeLong, paired fold test
src/costs.py	Cost matrix, OOF threshold selection, Elkan analytic optimum
src/plotting.py, src/report.py	Figure and table generation
experiments/exp01–exp14	One work package each; each writes results/<id>.json
experiments/verify.py	Re-checks every headline number in this report against the result files

Execution order (≈48 min serial): exp02_tuning → exp01_ladder, exp04–exp08 → exp03_imbalance → exp09–exp13 → exp14_final. Only exp14_final.py is licensed to read the hold-out.

```
python -m experiments.exp02_tuning      # Optuna search; writes artifacts/best_model.json
python -m experiments.exp01_ladder      # complexity ladder (Table 4)
python -m experiments.exp10_cost        # threshold sweep (Table 6)
python -m experiments.exp11_shap        # SHAP attribution (Table 8)
python -m experiments.exp14_final       # hold-out, read once (Table 7)
python -m experiments.verify            # re-derive every number quoted above
python -m experiments.figures           # regenerate all figures
```

Appendix B — Source code

Full listings of src/*.py and experiments/*.py follow. Generate with: python report/build_appendix.py

src/

src/config.py

```
"""Single source of truth for seeds, paths, cost constants and search spaces."""

from __future__ import annotations

from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
FIGDIR = ROOT / "figures"
RESULTS = ROOT / "results"
ARTIFACTS = ROOT / "artifacts"

for _d in (FIGDIR, RESULTS, ARTIFACTS):
    _d.mkdir(exist_ok=True)

# ----- randomness

SEED = 42
# Distinct from SEED so the repeated-CV folds used for variance reporting are not
# correlated with the folds the tuner selected against.
CV_SEED = 7

N_FOLDS = 5
N_REPEATS = 5
```

```

TEST_SIZE = 0.20
# Fraction carved out of each training fold as an early-stopping eval_set, so the
# CV validation fold is never seen by early stopping.
ES_SLICE = 0.15
ES_ROUNDS = 50

# ----- cost model
# FN (missed default) = LGD x EAD. Retail unsecured loss-given-default ~0.70.
LGD = 0.70
# FP (rejected good customer) = forgone net interest margin ~0.12 x exposure.
NIM = 0.12
# Headline cost ratio C_FN / C_FP = 0.70 / 0.12 = 5.83 -> reported as 6:1.
R_HEADLINE = 6
R_GRID = [1, 2, 5, 6, 10, 20]
THRESHOLD_GRID_N = 501

# ----- xgboost

# enable_categorical defaults to True in xgboost 3.4, and shap 0.52 reads that flag
# alone to decide a model has categorical splits -- which blocks interventional
# TreeSHAP. Numeric models must set it False explicitly. See SETUP.md.
XGB_FIXED = {
    "tree_method": "hist",
    "enable_categorical": False,
    "eval_metric": "auc",
    "n_jobs": -1,
    "random_state": SEED,
}

# Declarative so tuning.py can suggest from it and the report can print it.
# (kind, low, high, log)
SEARCH_SPACE = {
    "max_depth": ("int", 3, 8, False),
    "eta": ("float", 0.01, 0.30, True),
    "min_child_weight": ("float", 1.0, 20.0, True),
    "subsample": ("float", 0.60, 1.0, False),
    "colsample_bytree": ("float", 0.50, 1.0, False),
    "gamma": ("float", 0.0, 5.0, False),
    "reg_lambda": ("float", 1e-2, 100.0, True),
    "reg_alpha": ("float", 1e-3, 10.0, True),
}

# Early stopping decides the real count; this is only a ceiling.
N_ESTIMATORS_CAP = 2000
# The plan budgeted 60-80 trials against an estimated 20-40 min. Measured cost is ~2.3 s
# per trial, so the budget was raised: more TPE trials is strictly better and 150 still
# finishes in ~6 min. Recorded in the exp02 notes as a deviation.
N_TRIALS = 150

# ----- imbalance arms
# scale_pos_weight is fixed per arm rather than tuned: ROC-AUC is invariant to
# monotone score transformations, so the tuner would be blind to it and pick
# arbitrarily. Fixing it per arm turns the imbalance study into a controlled
# comparison instead of a confound.
IMBALANCE_ARMS = {
    "A_baseline": {"spw": 1.0, "threshold": "fixed_0.5", "sampler": None},
    "B_threshold": {"spw": 1.0, "threshold": "oof_cost", "sampler": None},
    "C_spw": {"spw": None, "threshold": "fixed_0.5", "sampler": None},
    "C_spw_threshold": {"spw": None, "threshold": "oof_cost", "sampler": None},
    "D_smote": {"spw": 1.0, "threshold": "oof_cost", "sampler": "smote"},
}

TUNING_METRIC = "roc_auc"

```

src/data.py

```

"""Data loading, cleaning, feature engineering and splitting.

`load_raw` and `clean` are ported from Phase 1 Cells 4 and 24 with the assertions intact,
so Phase 2 provably operates on the same 29,944-row frame the proposal described.
"""

from __future__ import annotations

```

```

import numpy as np
import pandas as pd
from sklearn.model_selection import (
    RepeatedStratifiedKFold,
    StratifiedKFold,
    train_test_split,
)

from .config import ARTIFACTS, CV_SEED, N_FOLDS, N_REPEATS, SEED, TEST_SIZE

TARGET = "DEFAULT"

DEMOGRAPHIC = ["SEX", "EDUCATION", "MARRIAGE", "AGE"]
PAY_COLS = ["PAY_0", "PAY_2", "PAY_3", "PAY_4", "PAY_5", "PAY_6"]
PAY_CHRONO = ["PAY_6", "PAY_5", "PAY_4", "PAY_3", "PAY_2", "PAY_0"] # April -> September
BILL_COLS = [f"BILL_AMT{i}" for i in range(1, 7)]
BILL_CHRONO = BILL_COLS[::-1]
PAYAMT_COLS = [f"PAY_AMT{i}" for i in range(1, 7)]
CONTINUOUS = ["LIMIT_BAL", "AGE"] + BILL_COLS + PAYAMT_COLS
MONTH_LABEL = ["Apr", "May", "Jun", "Jul", "Aug", "Sep"]

RAW_FEATURES = ["LIMIT_BAL"] + DEMOGRAPHIC + PAY_COLS + BILL_COLS + PAYAMT_COLS

# WP10 ablates over this dict.
FEATURE_GROUPS = {
    "demographic": ["SEX", "EDUCATION", "MARRIAGE", "AGE"],
    "limit": ["LIMIT_BAL"],
    "pay_status": PAY_COLS,
    "bill": BILL_COLS,
    "pay_amt": PAYAMT_COLS,
}

SEX_LABEL = {1: "1 male", 2: "2 female"}
CLEAN_EDU = {1: "1 graduate school", 2: "2 university", 3: "3 high school", 4: "4 others"}
CLEAN_MAR = {1: "1 married", 2: "2 single", 3: "3 others"}
AGE_BANDS = [(0, 29, "<30"), (30, 39, "30-39"), (40, 49, "40-49"), (50, 200, "50+")]

_RAW_CACHE = ARTIFACTS / "raw.csv"

def load_raw(use_cache=True):
    """Fetch UCI id=350, apply the programmatic rename, attach the target."""
    if use_cache and _RAW_CACHE.exists():
        raw = pd.read_csv(_RAW_CACHE)
    else:
        from ucimlrepo import fetch_ucirepo

        repo = fetch_ucirepo(id=350)
        name_map = {
            r["name"]: r["description"]
            for _, r in repo.variables.iterrows()
            if isinstance(r.get("description"), str) and r["description"].strip()
        }
        raw = repo.data.features.rename(columns=name_map).copy()
        raw[TARGET] = repo.data.targets.iloc[:, 0].values
        raw.to_csv(_RAW_CACHE, index=False)

    assert list(raw.columns[:5]) == ["LIMIT_BAL", "SEX", "EDUCATION", "MARRIAGE", "AGE"], \
        "Programmatic rename failed - check variables metadata"
    assert raw.shape == (30000, 24), f"Unexpected raw shape {raw.shape}"
    return raw

def clean(raw):
    """Phase 1 Cell 24: de-duplicate on features, fold undocumented codes into 'others'.

    De-duplication happens before any split, so identical feature-rows cannot straddle
    train and test and inflate every score.
    """
    feat_cols = [c for c in raw.columns if c != TARGET]

```

```

df = raw.copy()
df = df.drop_duplicates(subset=feat_cols, keep="first").reset_index(drop=True)
df["EDUCATION"] = df["EDUCATION"].replace({0: 4, 5: 4, 6: 4})
df["MARRIAGE"] = df["MARRIAGE"].replace({0: 3})

assert len(df) == 29944, f"Expected 29,944 rows after de-duplication, got {len(df):,}"
assert int(df[TARGET].sum()) == 6622, "Unexpected default count after cleaning"
assert set(df["EDUCATION"].unique()) <= {1, 2, 3, 4}, "EDUCATION outside documented domain"
assert set(df["MARRIAGE"].unique()) <= {1, 2, 3}, "MARRIAGE outside documented domain"
assert int(df.isna().sum().sum()) == 0, "Nulls introduced by cleaning"
assert not df[feat_cols].duplicated().any(), "Duplicates survived cleaning"
return df

# ----- WP3: engineering

# Evenly spaced chronological month index; the OLS slope denominator is therefore
# a constant and the whole trend reduces to one dot product.
_X_MONTH = np.arange(6, dtype=float)
_X_CENTRED = _X_MONTH - _X_MONTH.mean()
_X_SS = float((_X_CENTRED**2).sum()) # 17.5

def _ols_slope(values):
    """Least-squares slope of each row against the chronological month index."""
    v = np.asarray(values, dtype=float)
    return (v - v.mean(axis=1, keepdims=True)) @ _X_CENTRED / _X_SS

ENGINEERED = (
    [f"UTIL_{i}" for i in range(1, 7)]
    + [f"PAY_RATIO_{i}" for i in range(1, 7)]
    + ["N_DELIQ", "MAX_DELIQ", "DELIQ_TREND", "BILL_TREND", "MONTHS_DORMANT"]
)

def engineer(df):
    """Add the 17 derived features WP9 benchmarks against the raw 23.

    Phase 1 predicted aggregation may *lose* signal by collapsing the recency gradient;
    WP9 tests that prediction rather than assuming these help.

    PAY_RATIO is undefined where the statement balance is zero or negative (1,930 rows
    carry a negative BILL_AMT, per Phase 1 section 1.6). Those entries become NaN rather
    than a sentinel: XGBoost learns a default direction for missing values natively, so
    NaN carries strictly more information than an arbitrary fill.
    """
    out = df.copy()
    limit = out["LIMIT_BAL"].to_numpy(dtype=float)
    assert (limit > 0).all(), "LIMIT_BAL must be positive for UTIL_n to be defined"

    bills = out[BILL_COLS].to_numpy(dtype=float)
    pays = out[PAYAMT_COLS].to_numpy(dtype=float)
    status = out[PAY_COLS].to_numpy(dtype=float)

    for i, col in enumerate(BILL_COLS, start=1):
        out[f"UTIL_{i}"] = bills[:, i - 1] / limit

    # Index-aligned with the bill of the same month, matching the column naming.
    safe_bills = np.where(bills > 0, bills, np.nan)
    ratios = pays / safe_bills
    for i in range(1, 7):
        out[f"PAY_RATIO_{i}"] = ratios[:, i - 1]

    out["N_DELIQ"] = (status >= 1).sum(axis=1)
    out["MAX_DELIQ"] = status.max(axis=1)
    out["MONTHS_DORMANT"] = (status == -2).sum(axis=1)
    out["DELIQ_TREND"] = _ols_slope(out[PAY_CHRONO].to_numpy(dtype=float))
    out["BILL_TREND"] = _ols_slope(out[BILL_CHRONO].to_numpy(dtype=float))

```

```

added = [c for c in out.columns if c not in df.columns]
assert set(added) == set(ENGINEERED), f"Engineered column set drifted: {added}"
assert len(added) == 17, f"Expected 17 engineered columns, got {len(added)}"
assert not np.isinf(out[added].to_numpy(dtype=float)).any(), "engineer() emitted inf"
return out

def age_band(series):
    labels = pd.Series(index=series.index, dtype="object")
    for lo, hi, name in AGE_BANDS:
        labels[(series >= lo) & (series <= hi)] = name
    return labels

def xy(df, features=None):
    """Feature matrix / target split. Defaults to the raw 23 predictors."""
    cols = list(features) if features is not None else list(RAW_FEATURES)
    return df[cols].copy(), df[TARGET].to_numpy(dtype=int)

# ----- WP4: splits

SPLITS_PATH = ARTIFACTS / "splits.npz"

def make_splits(df=None, force=False):
    """Build and cache the 80/20 hold-out plus both fold structures.

    Written once by WP4 and read by every experiment, so all fourteen share identical
    indices and no experiment can silently reshuffle.
    """
    if SPLITS_PATH.exists() and not force:
        return load_splits()

    if df is None:
        df = clean(load_raw())
    y = df[TARGET].to_numpy(dtype=int)
    idx = np.arange(len(df))

    train_idx, test_idx = train_test_split(
        idx, test_size=TEST_SIZE, stratify=y, random_state=SEED
    )
    train_idx = np.sort(train_idx)
    test_idx = np.sort(test_idx)
    y_train = y[train_idx]

    # Tuning / model-selection folds.
    skf = StratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_state=SEED)
    fold_id = np.empty(len(train_idx), dtype=np.int8)
    for f, (_, va) in enumerate(skf.split(train_idx, y_train)):
        fold_id[va] = f

    # Variance-reporting folds: distinct seed so repeats are not correlated with
    # the folds the tuner optimised against.
    rskf = RepeatedStratifiedKFold(
        n_splits=N_FOLDS, n_repeats=N_REPEATS, random_state=CV_SEED
    )
    rep_fold_id = np.empty((N_REPEATS, len(train_idx)), dtype=np.int8)
    for k, (_, va) in enumerate(rskf.split(train_idx, y_train)):
        rep_fold_id[k // N_FOLDS, va] = k % N_FOLDS

    np.savez_compressed(
        SPLITS_PATH,
        train_idx=train_idx,
        test_idx=test_idx,
        fold_id=fold_id,
        rep_fold_id=rep_fold_id,
    )
    print(f"saved -> {SPLITS_PATH}")
    return load_splits()

```

```

def load_splits():
    if not SPLITS_PATH.exists():
        raise FileNotFoundError(
            f"{SPLITS_PATH} is missing. Produce it with WP4: python -m src.data"
        )
    z = np.load(SPLITS_PATH)
    return {k: z[k] for k in z.files}

def cv_folds(splits):
    """Tuning folds as (train_positions, val_positions) pairs into the training frame."""
    fold_id = splits["fold_id"]
    return [(np.where(fold_id != f)[0], np.where(fold_id == f)[0])
            for f in range(N_FOLDS)]

def repeated_cv_folds(splits):
    """The 25 variance-reporting folds, as positions into the training frame."""
    rep = splits["rep_fold_id"]
    out = []
    for r in range(rep.shape[0]):
        for f in range(N_FOLDS):
            out.append((np.where(rep[r] != f)[0], np.where(rep[r] == f)[0]))
    return out

def frame(engineered=False):
    df = clean(load_raw())
    return engineer(df) if engineered else df

def training_context(features=None, engineered=False, df=None):
    """Training portion only, with both fold structures attached.

    Experiments WP5-WP13 use this exclusively. It cannot return test rows, which is what
    makes the leakage audit in section 7.5 a one-line grep: `X_test` should appear only
    in the final-evaluation and SHAP modules.
    """
    df = frame(engineered) if df is None else df
    s = load_splits()
    X, y = xy(df, features)
    tr = s["train_idx"]
    return {
        "df": df,
        "splits": s,
        "X": X.iloc[tr].reset_index(drop=True),
        "y": y[tr],
        "folds": cv_folds(s),
        "repeated_folds": repeated_cv_folds(s),
        "features": list(X.columns),
    }

def test_context(features=None, engineered=False, df=None):
    """Held-out rows. Licensed for WP14/WP15 (SHAP explains decisions, selects nothing)
    and WP18 (the single final-evaluation gate). Nothing else may import this."""
    df = frame(engineered) if df is None else df
    s = load_splits()
    X, y = xy(df, features)
    te = s["test_idx"]
    return {
        "df": df,
        "splits": s,
        "X_test": X.iloc[te].reset_index(drop=True),
        "y_test": y[te],
        "features": list(X.columns),
    }

if __name__ == "__main__":

```

```

frame = clean(load_raw())
frame = engineer(frame)
s = make_splits(frame, force=True)
ytr = frame[TARGET].to_numpy()[s["train_idx"]]
yte = frame[TARGET].to_numpy()[s["test_idx"]]
print(f"train {len(s['train_idx']):,} ({ytr.mean():.4%} default) "
      f"test {len(s['test_idx']):,} ({yte.mean():.4%} default)")
assert not set(s["train_idx"]) & set(s["test_idx"]), "train/test index overlap"
print("splits ok")

```

src/models.py

```

"""Model zoo. Every rung of the complexity ladder and every XGBoost variant is built here,
so no experiment constructs an estimator of its own and configurations cannot drift apart.
"""

```

```

from __future__ import annotations

```

```

import numpy as np
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from xgboost import XGBClassifier

```

```

from .config import ES_ROUNDS, ES_SLICE, N_ESTIMATORS_CAP, SEED, XGB_FIXED
from .data import PAY_COLS, RAW_FEATURES

```

```

def xgb_classifier(params=None, spw=1.0, early_stopping=True,
                  enable_categorical=False, seed=SEED):
    """XGBoost with the project's fixed infrastructure settings.

    `early_stopping_rounds` and `eval_metric` go in the constructor, not `.fit()` --
    the XGBoost 2.x/3.x API moved them. `enable_categorical` is pinned False because
    xgboost 3.4 defaults it True and shap then refuses interventional TreeSHAP.
    """

```

```

    p = dict(XGB_FIXED)
    p["enable_categorical"] = enable_categorical
    p["random_state"] = seed
    p["scale_pos_weight"] = spw
    if early_stopping:
        p["n_estimators"] = N_ESTIMATORS_CAP
        p["early_stopping_rounds"] = ES_ROUNDS
    if params:
        p.update(params)
    return XGBClassifier(**p)

```

```

def xgb_library_defaults(seed=SEED):
    """Rung 5: no tuning and no early stopping, so the tuning gain is isolated."""
    return XGBClassifier(**{**XGB_FIXED, "random_state": seed})

```

```

def majority():
    return DummyClassifier(strategy="most_frequent")

```

```

def single_rule():
    """Rung 1: one split on PAY_0. Recovers the Phase 1 benchmark that was lost with
    the uncommitted multivariate notebook."""
    return DecisionTreeClassifier(max_depth=1, random_state=SEED)

```

```

def logistic():
    """Rung 2: linear anchor.

```

Standardisation is a convergence requirement for a gradient learner on monetary

columns spanning six orders of magnitude -- not a tuned hyperparameter. That trees need no such step is itself part of the Phase 1 scaling argument.

```
return Pipeline([
    ("scale", StandardScaler()),
    ("lr", LogisticRegression(max_iter=1000, random_state=SEED)),
])
```

```
def tree_depth_tuned(seed=SEED):
```

"""Rung 3: depth selected by inner CV, so the rung is not handicapped by an arbitrary depth. The search is nested inside each outer fold."""

```
return GridSearchCV(
    DecisionTreeClassifier(random_state=seed),
    {"max_depth": [2, 3, 4, 5, 6, 8, 10, 12]},
    scoring="roc_auc",
    cv=StratifiedKFold(3, shuffle=True, random_state=seed),
    n_jobs=-1,
)
```

```
def random_forest(seed=SEED):
```

"""Rung 4: bagging at library defaults, isolating it from boosting."""

```
return RandomForestClassifier(random_state=seed, n_jobs=-1)
```

```
def ladder_specs(best_params=None):
```

"""The seven rungs of WP5, in increasing capability and decreasing interpretability.

Rungs 2 and 3 are untuned internal reference implementations used to locate XGBoost on a capability axis. They are NOT the group's Decision Tree or Logistic Regression submissions, which are separately tuned by other members.

```
specs = [
    {"label": "rung0_majority", "model": "Majority class",
     "factory": majority, "features": RAW_FEATURES,
     "interpretability": "total (1 constant)", "interp_cost": 1},
    {"label": "rung1_pay0_rule", "model": "PAY_0 only, depth-1 tree",
     "factory": single_rule, "features": ["PAY_0"],
     "interpretability": "single rule", "interp_cost": 1},
    {"label": "rung2_logistic", "model": "Logistic regression (defaults)",
     "factory": logistic, "features": RAW_FEATURES,
     "interpretability": "23 coefficients + intercept", "interp_cost": 24},
    {"label": "rung3_tree", "model": "Decision tree (depth-tuned)",
     "factory": tree_depth_tuned, "features": RAW_FEATURES,
     "interpretability": "rule set", "interp_cost": None},
    {"label": "rung4_forest", "model": "Random forest (defaults)",
     "factory": random_forest, "features": RAW_FEATURES,
     "interpretability": "not directly available - see SHAP", "interp_cost": None},
    {"label": "rung5_xgb_default", "model": "XGBoost (library defaults)",
     "factory": xgb_library_defaults, "features": RAW_FEATURES,
     "interpretability": "not directly available - see SHAP", "interp_cost": None},
]
if best_params is not None:
    specs.append({
        "label": "rung6_xgb_tuned", "model": "XGBoost (tuned)",
        "factory": lambda: xgb_classifier(best_params), "features": RAW_FEATURES,
        "interpretability": "not directly available - see SHAP", "interp_cost": None,
    })
return specs
```

```
def scale_pos_weight(y):
```

"""neg/pos, the ratio XGBoost expects. Confirm polarity: 1 = default."""

```
y = np.asarray(y).astype(int)
pos = int(y.sum())
return float((len(y) - pos) / pos)
```

```
def categorical_frame(X, cols=None):
```

"""Cast PAY_n to pandas `category` for the enable_categorical arm of WP8."""

```
out = X.copy()
```

```

for c in (cols if cols is not None else PAY_COLS):
    out[c] = out[c].astype("category")
return out

```

```

class SmoteXGB(BaseEstimator, ClassifierMixin):
    """SMOTE + XGBoost, resampling strictly inside `fit`.

    Equivalent to an imblearn Pipeline for leakage purposes -- oversampling only ever
    sees the rows passed to `fit`, which cv_evaluate restricts to the fold's training
    half -- with one deliberate refinement: the early-stopping slice is carved off
    *before* resampling, so it retains the natural 22% prevalence. Validating against
    synthetically balanced data would make the stopping criterion measure the wrong
    distribution.
    """

    def __init__(self, params=None, seed=SEED):
        self.params = params
        self.seed = seed

    def fit(self, X, y):
        from imblearn.over_sampling import SMOTE
        from sklearn.model_selection import train_test_split

        y = np.asarray(y).astype(int)
        i_fit, i_es = train_test_split(
            np.arange(len(y)), test_size=ES_SLICE, stratify=y, random_state=self.seed
        )
        X_res, y_res = SMOTE(random_state=self.seed).fit_resample(
            X.iloc[i_fit], y[i_fit]
        )
        self.model_ = xgb_classifier(self.params, spw=1.0, seed=self.seed)
        self.model_.fit(X_res, y_res,
            eval_set=[(X.iloc[i_es], y[i_es])], verbose=False)
        self.classes_ = np.array([0, 1])
        return self

    def predict_proba(self, X):
        return self.model_.predict_proba(X)

    def predict(self, X):
        return (self.predict_proba(X)[: , 1] >= 0.5).astype(int)

    @property
    def best_iteration(self):
        return self.model_.best_iteration

    def count_tree_nodes(fitted):
        """Interpretability cost for the tree rungs: nodes a human must read."""
        est = fitted.best_estimator_ if isinstance(fitted, GridSearchCV) else fitted
        if isinstance(est, DecisionTreeClassifier):
            return int(est.tree_.node_count)
        if isinstance(est, RandomForestClassifier):
            return int(sum(t.tree_.node_count for t in est.estimators_))
        if isinstance(est, XGBClassifier):
            return int(len(est.get_booster().get_dump()))
        return None

```

src/runner.py

```

"""The one evaluation path every experiment calls, plus the result-file writer.

Divergent CV loops are the main source of numbers that cannot be reconciled later, so
no experiment implements its own.
"""

from __future__ import annotations

import json
import platform
import sys

```

```

import time
from datetime import datetime, timezone

import numpy as np
from sklearn.metrics import average_precision_score, brier_score_loss, roc_auc_score
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier

from .config import ES_SLICE, RESULTS, SEED

def versions():
    import sklearn

    v = {"python": platform.python_version(), "numpy": np.__version__,
         "sklearn": sklearn.__version__}
    for mod in ("xgboost", "shap", "optuna", "imblearn", "pandas"):
        try:
            v[mod] = __import__(mod).__version__
        except Exception:
            v[mod] = None
    return v

def _uses_early_stopping(est):
    return isinstance(est, XGBClassifier) and getattr(est, "early_stopping_rounds", None)

def fit_one(est, X_tr, y_tr, seed=SEED):
    """Fit a single estimator, carving the early-stopping slice out of the training data.

    The eval_set comes from an inner slice of this fold's *training* rows, so the outer
    validation fold is never seen by early stopping and the test set never at all.
    """
    if _uses_early_stopping(est):
        i_fit, i_es = train_test_split(
            np.arange(len(y_tr)), test_size=ES_SLICE, stratify=y_tr, random_state=seed
        )
        est.fit(X_tr.iloc[i_fit], y_tr[i_fit],
                eval_set=[(X_tr.iloc[i_es], y_tr[i_es])], verbose=False)
    else:
        est.fit(X_tr, y_tr)
    return est

def _fold_metrics(y_true, proba):
    return {
        "roc_auc": float(roc_auc_score(y_true, proba)),
        "pr_auc": float(average_precision_score(y_true, proba)),
        "brier": float(brier_score_loss(y_true, proba)),
    }

def cv_evaluate(X, y, factory, folds, seed=SEED, collect_oof=True, return_models=False):
    """Fit `factory()` on every fold and report mean +/- SD across fits.

    `X` is a DataFrame and `y` a 1-D array, both already restricted to the training
    portion. `folds` are (train_positions, val_positions) pairs from data.cv_folds or
    data.repeated_cv_folds.

    The SD returned is across fits, not the standard error -- it is the quantity the
    report uses to decide whether two arms are distinguishable.
    """
    y = np.asarray(y).astype(int)
    per_fold, models = [], []
    oof_sum = np.zeros(len(y)) if collect_oof else None
    oof_n = np.zeros(len(y)) if collect_oof else None
    best_iters = []

    for k, (tr, va) in enumerate(folds):

```

```

        est = fit_one(factory(), X.iloc[tr], y[tr], seed=seed)
        proba = est.predict_proba(X.iloc[va])[0, 1]
        per_fold.append(_fold_metrics(y[va], proba))
    if collect_oof:
        oof_sum[va] += proba
        oof_n[va] += 1
    if isinstance(est, XGBClassifier) and getattr(est, "best_iteration", None) is not None:
        best_iters.append(int(est.best_iteration))
    if return_models:
        models.append(est)

summary = {}
for m in ("roc_auc", "pr_auc", "brier"):
    vals = np.array([f[m] for f in per_fold], dtype=float)
    summary[f"{m}_mean"] = float(vals.mean())
    summary[f"{m}_sd"] = float(vals.std(ddof=1)) if len(vals) > 1 else 0.0
summary["n_fits"] = len(folds)

out = {"cv": summary, "per_fold": per_fold}
if best_iters:
    out["best_iteration_mean"] = float(np.mean(best_iters))
if collect_oof:
    with np.errstate(invalid="ignore"):
        out["oof"] = np.where(oof_n > 0, oof_sum / np.maximum(oof_n, 1), np.nan)
if return_models:
    out["models"] = models
return out

def per_fold_metric(cv_out, metric="roc_auc"):
    """Per-fold values, for paired arm-vs-arm comparisons."""
    return np.array([f[metric] for f in cv_out["per_fold"]], dtype=float)

def make_row(label, cv_summary, params=None, n_features=None, test=None, extra=None):
    """Build one `rows[]` entry of the result contract.

    Any metric that does not apply is explicitly null, never omitted and never zero.
    """
    row = {
        "label": label,
        "params": params or {},
        "n_features": n_features,
        "cv": {
            "roc_auc_mean": cv_summary.get("roc_auc_mean"),
            "roc_auc_sd": cv_summary.get("roc_auc_sd"),
            "pr_auc_mean": cv_summary.get("pr_auc_mean"),
            "pr_auc_sd": cv_summary.get("pr_auc_sd"),
            "brier_mean": cv_summary.get("brier_mean"),
            "brier_sd": cv_summary.get("brier_sd"),
            "n_fits": cv_summary.get("n_fits"),
        } if cv_summary is not None else None,
        "test": test,
    }
    if extra:
        row.update(extra)
    return row

def save_result(experiment_id, title, rows, notes, runtime_sec, config=None, seed=SEED):
    """Write results/<experiment_id>.json against the section 3.2 contract."""
    if not notes or not str(notes).strip():
        raise ValueError("notes is mandatory; 'Nothing unusual' is acceptable, empty is not")
    payload = {
        "experiment_id": experiment_id,
        "title": title,
        "run_utc": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
        "seed": seed,
        "versions": versions(),
        "runtime_sec": round(float(runtime_sec), 1),
        "config": config or {},
        "rows": rows,
    }

```

```

        "notes": notes,
    }
    path = RESULTS / f"{experiment_id}.json"
    path.write_text(json.dumps(payload, indent=2, default=_jsonable), encoding="utf-8")
    print(f"saved -> {path}")
    return path

def _jsonable(o):
    if isinstance(o, (np.integer,)):
        return int(o)
    if isinstance(o, (np.floating,)):
        return float(o)
    if isinstance(o, np.ndarray):
        return o.tolist()
    if isinstance(o, (np.bool_,)):
        return bool(o)
    raise TypeError(f"{type(o)} is not JSON serialisable")

def load_result(experiment_id):
    path = RESULTS / f"{experiment_id}.json"
    if not path.exists():
        raise FileNotFoundError(f"{path} is missing - run python -m experiments.{experiment_id}")
    return json.loads(path.read_text(encoding="utf-8"))

class Timer:
    def __enter__(self):
        self.t0 = time.perf_counter()
        return self

    def __exit__(self, *a):
        self.elapsed = time.perf_counter() - self.t0

    @property
    def seconds(self):
        return time.perf_counter() - self.t0

def require_artifact(path, wp):
    """Fail loudly rather than silently recomputing -- that is how seeds drift."""
    if not path.exists():
        sys.exit(f"MISSING ARTIFACT: {path}\nProduced by {wp}. Run that first.")
    return path

```

src/tuning.py

```

"""Optuna TPE search.

ROC-AUC is the objective: it is the declared primary metric (Lessmann et al., 2015),
prevalence-independent so it stays comparable across the complexity ladder, and
lower-variance across folds than PR-AUC. `aucpr` is logged every trial but not optimised.

`scale_pos_weight` is deliberately absent from the space. ROC-AUC is invariant to
monotone score transformations, so the tuner would be blind to it and pick arbitrarily;
fixing it per arm turns the imbalance study into a controlled comparison instead.
"""

from __future__ import annotations

import numpy as np
import optuna
from sklearn.metrics import average_precision_score, roc_auc_score

from .config import N_TRIALS, SEARCH_SPACE, SEED
from .models import xgb_classifier
from .runner import fit_one

optuna.logging.set_verbosity(optuna.logging.WARNING)

```

```

def suggest(trial, space=None):
    space = space or SEARCH_SPACE
    params = {}
    for name, (kind, lo, hi, log) in space.items():
        if kind == "int":
            params[name] = trial.suggest_int(name, int(lo), int(hi), log=log)
        else:
            params[name] = trial.suggest_float(name, lo, hi, log=log)
    return params

def make_objective(X, y, folds, spw=1.0, seed=SEED, space=None, model_fn=None):
    """`model_fn(params, spw, seed) -> estimator` lets each imbalance arm be tuned
    independently while sharing one search space and one objective."""
    y = np.asarray(y).astype(int)
    if model_fn is None:
        def model_fn(params, spw, seed):
            return xgb_classifier(params, spw=spw, seed=seed)

    def objective(trial):
        params = suggest(trial, space)
        aucs, aps, iters = [], [], []
        for k, (tr, va) in enumerate(folds):
            est = fit_one(model_fn(params, spw, seed), X.iloc[tr], y[tr], seed=seed)
            proba = est.predict_proba(X.iloc[va][:, 1])
            aucs.append(roc_auc_score(y[va], proba))
            aps.append(average_precision_score(y[va], proba))
            if getattr(est, "best_iteration", None) is not None:
                iters.append(int(est.best_iteration))
            # Report after each fold so MedianPruner can abandon hopeless trials early.
            trial.report(float(np.mean(aucs)), step=k)
            if trial.should_prune():
                raise optuna.TrialPruned()
            trial.set_user_attr("pr_auc_mean", float(np.mean(aps)))
            trial.set_user_attr("roc_auc_sd", float(np.std(aucs, ddof=1)))
            trial.set_user_attr("best_iteration_mean", float(np.mean(iters)) if iters else None)
            return float(np.mean(aucs))

        return objective

def run_study(X, y, folds, storage_path, study_name="xgb_roc_auc", n_trials=N_TRIALS,
             spw=1.0, seed=SEED, space=None, model_fn=None):
    """TPE with median pruning, persisted to SQLite so the study resumes and the
    optimisation-history and param-importance plots survive for the appendix."""
    study = optuna.create_study(
        study_name=study_name,
        storage=f"sqlite:////{storage_path}",
        direction="maximize",
        sampler=optuna.samplers.TPESampler(seed=seed, multivariate=True),
        pruner=optuna.pruners.MedianPruner(n_startup_trials=10, n_warmup_steps=2),
        load_if_exists=True,
    )
    done = len([t for t in study.trials
                if t.state in (optuna.trial.TrialState.COMPLETE,
                              optuna.trial.TrialState.PRUNED)])
    remaining = max(0, n_trials - done)
    if remaining:
        # XGBoost already parallelises across threads; Optuna stays single-process to
        # avoid thread oversubscription.
        study.optimize(make_objective(X, y, folds, spw=spw, seed=seed, space=space,
                                     model_fn=model_fn),
                      n_trials=remaining, n_jobs=1, show_progress_bar=False)

    return study

```

src/metrics.py

```

"""Metric helpers. `wilson` is ported verbatim from Phase 1 Cell 20."""

from __future__ import annotations

import numpy as np

```

```

from scipy import stats
from sklearn.metrics import (
    average_precision_score,
    brier_score_loss,
    confusion_matrix,
    roc_auc_score,
)

def wilson(k, n, z=1.96):
    """Wilson score interval for a binomial proportion; valid at small n and p near 0 or 1."""
    if n == 0:
        return np.nan, np.nan
    p = k / n
    den = 1 + z**2 / n
    centre = (p + z**2 / (2 * n)) / den
    half = z * np.sqrt(p * (1 - p) / n + z**2 / (4 * n**2)) / den
    return max(0.0, centre - half) * 100, min(1.0, centre + half) * 100

def brier(y_true, proba):
    return float(brier_score_loss(y_true, proba))

def evaluate(y_true, proba, threshold=0.5):
    """Ranking, calibration and thresholded-decision metrics for one prediction vector.

    `threshold` must already be frozen from out-of-fold data (see costs.pick_threshold_oof);
    this function never searches for one.
    """
    y_true = np.asarray(y_true)
    proba = np.asarray(proba, dtype=float)
    pred = (proba >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, pred, labels=[0, 1]).ravel()
    recall = tp / (tp + fn) if (tp + fn) else 0.0
    precision = tp / (tp + fp) if (tp + fp) else 0.0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) else 0.0
    return {
        "roc_auc": float(roc_auc_score(y_true, proba)),
        "pr_auc": float(average_precision_score(y_true, proba)),
        "brier": brier(y_true, proba),
        "threshold": float(threshold),
        "recall": float(recall),
        "precision": float(precision),
        "f1": float(f1),
        "tp": int(tp), "fp": int(fp), "fn": int(fn), "tn": int(tn),
    }

def reliability(y_true, proba, n_bins=10):
    """Equal-width reliability curve, serialised so figures rebuild without refitting."""
    y_true = np.asarray(y_true)
    proba = np.asarray(proba, dtype=float)
    edges = np.linspace(0.0, 1.0, n_bins + 1)
    idx = np.clip(np.digitize(proba, edges[1:-1], right=False), 0, n_bins - 1)
    out = []
    for b in range(n_bins):
        m = idx == b
        out.append({
            "bin": b,
            "lo": float(edges[b]),
            "hi": float(edges[b + 1]),
            "n": int(m.sum()),
            "mean_pred": float(proba[m].mean()) if m.any() else None,
            "frac_pos": float(y_true[m].mean()) if m.any() else None,
        })
    return out

# ----- DeLong

```

```

def _midrank(x):
    order = np.argsort(x)
    z = x[order]
    n = len(x)
    t = np.zeros(n, dtype=float)
    i = 0
    while i < n:
        j = i
        while j < n and z[j] == z[i]:
            j += 1
        t[i:j] = 0.5 * (i + j - 1) + 1
        i = j
    out = np.empty(n, dtype=float)
    out[order] = t
    return out

def _fast_delong(scores, n_pos):
    """Sun & Xu (2014) O(n log n) DeLong covariance. `scores` is (k, n) with positives first."""
    m = n_pos
    n = scores.shape[1] - m
    k = scores.shape[0]
    pos, neg = scores[:, :m], scores[:, m:]
    tx = np.empty((k, m)); ty = np.empty((k, n)); tz = np.empty((k, m + n))
    for r in range(k):
        tx[r] = _midrank(pos[r])
        ty[r] = _midrank(neg[r])
        tz[r] = _midrank(scores[r])
    aucs = tz[:, :m].sum(axis=1) / m / n - (m + 1.0) / 2.0 / n
    v01 = (tz[:, :m] - tx) / n
    v10 = 1.0 - (tz[:, m:] - ty) / m
    sx = np.cov(v01)
    sy = np.cov(v10)
    cov = sx / m + sy / n
    return aucs, np.atleast_2d(cov)

def delong_test(y_true, proba_a, proba_b):
    """Paired DeLong test for two correlated ROC curves on the same sample.

    Valid only on a single held-out set -- never across CV folds, where the samples
    are not independent.
    """
    y_true = np.asarray(y_true).astype(int)
    order = np.argsort(-y_true, kind="mergesort") # positives first, stable
    y_sorted = y_true[order]
    n_pos = int(y_sorted.sum())
    scores = np.vstack([np.asarray(proba_a, float)[order],
                        np.asarray(proba_b, float)[order]])
    aucs, cov = _fast_delong(scores, n_pos)
    var_diff = float(cov[0, 0] + cov[1, 1] - 2 * cov[0, 1])
    diff = float(aucs[0] - aucs[1])
    if var_diff <= 0:
        return {"auc_a": float(aucs[0]), "auc_b": float(aucs[1]), "diff": diff,
                "se": 0.0, "ci95": [diff, diff], "z": None, "p_value": None}
    se = float(np.sqrt(var_diff))
    z = diff / se
    p = float(2 * stats.norm.sf(abs(z)))
    return {
        "auc_a": float(aucs[0]),
        "auc_b": float(aucs[1]),
        "diff": diff,
        "se": se,
        "ci95": [diff - 1.96 * se, diff + 1.96 * se],
        "z": float(z),
        "p_value": p,
    }

def paired_fold_test(deltas):
    """Paired t-test on per-fold differences.

```

```

Nadeau & Bengio (2003): repeated-CV t-tests are anti-conservative because the
training folds overlap, so the p-value is a directional indicator, not a
calibrated error rate.
"""
d = np.asarray(deltas, dtype=float)
if len(d) < 2 or np.allclose(d.std(ddof=1), 0):
    return {"mean_delta": float(d.mean()), "sd": 0.0, "t": None, "p_value": None,
            "n": int(len(d))}
t, p = stats.ttest_1samp(d, 0.0)
return {"mean_delta": float(d.mean()), "sd": float(d.std(ddof=1)),
        "t": float(t), "p_value": float(p), "n": int(len(d))}

```

src/costs.py

```

"""Cost matrix, threshold selection and Elkan's analytic optimum.

The cost ratio is anchored in credit-risk parameters rather than chosen for convenience:
a false negative costs LGD x EAD (the unrecovered balance) and a false positive costs the
forgone net interest margin on a customer who would have repaid.
"""

from __future__ import annotations

import numpy as np

from .config import LGD, NIM, R_HEADLINE, THRESHOLD_GRID_N

def elkan_p_star(r):
    """Elkan (2001) cost-optimal decision threshold for a calibrated classifier.

    Acting is optimal when  $p * C_{FN} > (1 - p) * C_{FP}$ , i.e.  $p > 1 / (1 + r)$ .
    """
    return 1.0 / (1.0 + r)

def cost_ratio_from_credit_params(lgd=LGD, nim=NIM):
    return lgd / nim

def cost_at_threshold(y_true, proba, tau, r=R_HEADLINE, ead=None):
    """Total misclassification cost at a fixed threshold.

    With `ead=None` every error costs the same (count x unit cost). With `ead` supplied
    -- LIMIT_BAL is the proxy used in WP14 -- cost is summed per client, so a missed
    default on a large limit costs more than one on a small limit.
    """
    y_true = np.asarray(y_true).astype(int)
    pred = (np.asarray(proba, dtype=float) >= tau).astype(int)
    fn = (y_true == 1) & (pred == 0)
    fp = (y_true == 0) & (pred == 1)
    if ead is None:
        return float(r * fn.sum() + fp.sum())
    ead = np.asarray(ead, dtype=float)
    return float(LGD * ead[fn].sum() + NIM * ead[fp].sum())

def threshold_grid(n=THRESHOLD_GRID_N):
    # Open interval: tau=0 predicts everyone default, tau=1 nobody. Neither is a decision.
    return np.linspace(1e-4, 1 - 1e-4, n)

def pick_threshold_oof(y_oof, proba_oof, r=R_HEADLINE, ead=None, grid=None):
    """Choose the cost-minimising threshold on out-of-fold training predictions.

    The returned scalar is frozen and applied unchanged to the test set. Sweeping the
    threshold against test labels is the single most common fatal flaw in this task.
    """
    grid = threshold_grid() if grid is None else np.asarray(grid, dtype=float)
    costs = np.array([cost_at_threshold(y_oof, proba_oof, t, r=r, ead=ead) for t in grid])

```

```

tau = float(grid[int(np.argmin(costs))])
return tau, {"grid": grid.tolist(), "cost": costs.tolist(),
            "min_cost": float(costs.min()), "tau": tau}

```

src/plotting.py

```

"""House style ported verbatim from Phase 1 Cell 2, plus Phase 2 figure helpers."""

from __future__ import annotations

import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.ticker import FuncFormatter

from .config import FIGDIR

SURFACE = "#fcbcfb"
INK = "#0b0b0b"
INK_2 = "#52514e"
MUTED = "#898781"
GRID = "#e1e0d9"
AXIS = "#c3c2b7"

CAT = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100",
        "#e87ba4", "#008300", "#4a3aa7", "#e34948"]

SEQ = ["#cde2fb", "#b7d3f6", "#9ec5f4", "#86b6ef", "#6da7ec", "#5598e7",
        "#3987e5", "#2a78d6", "#256abf", "#1c5cab", "#184f95", "#104281", "#0d366b"]
SEQ_CMAP = mpl.colors.LinearSegmentedColormap.from_list("seq_blue", SEQ)

CRITICAL = "#d03b3b"

mpl.rcParams.update({
    "figure.facecolor": SURFACE, "axes.facecolor": SURFACE, "savefig.facecolor": SURFACE,
    "axes.edgecolor": AXIS, "axes.linewidth": 0.8, "axes.labelcolor": INK_2,
    "axes.titlecolor": INK, "axes.titlesize": 11, "axes.titleweight": "bold",
    "axes.titlelocation": "left", "axes.titlepad": 10, "axes.labelsize": 9.5,
    "axes.spines.top": False, "axes.spines.right": False, "axes.grid": True,
    "grid.color": GRID, "grid.linewidth": 0.8, "grid.linestyle": "-",
    "xtick.color": MUTED, "ytick.color": MUTED, "xtick.labelsize": 8.5,
    "ytick.labelsize": 8.5, "text.color": INK, "font.size": 9.5,
    "font.family": "sans-serif", "legend.frameon": False, "legend.fontsize": 9,
    "figure.dpi": 110, "savefig.dpi": 150, "savefig.bbox": "tight",
})

def save_fig(fig, name):
    """Write a figure to figures/<name>.png at 150 dpi for the report document."""
    path = FIGDIR / f"{name}.png"
    fig.savefig(path)
    print(f"saved -> {path}")
    return path

def thousands(x, _pos=None):
    return f"{x:,0f}"

KFMT = FuncFormatter(thousands)

def rateBars(ax, rates, los, his, counts, labels, title, *,
             colours=None, overall=None, xlabel=None, rotate=0, pct_fmt="{:.2f}%"):
    """Standard default-rate bar panel: Wilson CIs, counts in the tick labels."""
    rates, los, his = np.asarray(rates), np.asarray(los), np.asarray(his)
    x = np.arange(len(rates))
    ax.bar(x, rates, color=(colours if colours is not None else CAT[0]), width=0.62, zorder=3)
    ax.errorbar(x, rates, yerr=[rates - los, his - rates], fmt="none",
               ecolor=INK_2, elinewidth=1.1, capsize=4, zorder=4)
    top = float(his.max())
    ax.set_ylim(0, top * 1.24)

```

```

for xi, r, h in zip(x, rates, his):
    ax.text(xi, h + top * 0.045, pct_fmt.format(r), ha="center", va="bottom",
            fontsize=8.5, color=INK, zorder=5)
if overall is not None:
    ax.axhline(overall, color=AXIS, lw=1.1, zorder=2, label=f"overall {overall:.2f}%")
ax.set_xticks(x, [f"{labels[i]}\nn={c:,}" for i, c in enumerate(counts)], rotation=rotate)
if rotate:
    for lb in ax.get_xticklabels():
        lb.set_ha("right")
ax.set_ylabel("Default rate (%)")
if xlabel:
    ax.set_xlabel(xlabel)
ax.set_title(title)
ax.grid(axis="x", visible=False)
ax.set_axisbelow(True)

def errorbar_ladder(ax, labels, means, sds, title, *, xlabel="ROC-AUC", ref=None):
    """Horizontal mean +/- SD panel, used for the complexity ladder and ablations."""
    y = np.arange(len(labels))[:-1]
    ax.errorbar(means, y, xerr=sds, fmt="o", color=CAT[0], ecolor=INK_2,
                elinewidth=1.1, capsize=4, markersize=5, zorder=4)
    if ref is not None:
        ax.axvline(ref, color=AXIS, lw=1.1, zorder=2)
    ax.set_yticks(y, labels)
    ax.set_xlabel(xlabel)
    ax.set_title(title)
    ax.grid(axis="y", visible=False)
    ax.set_axisbelow(True)

```

src/report.py

```

"""Table builders shared by the narrative notebook and the report.

Every table is assembled from results/*.json, so a number cannot appear in the report
without existing in a committed result file. `pm` formats mean +/- SD to a fixed number
of digits, which keeps the "do not bold a difference smaller than one SD" rule honest:
if the difference does not survive the formatting, it does not survive the argument.
"""

from __future__ import annotations

import json

import pandas as pd

from src.config import RESULTS

def load(exp_id):
    path = RESULTS / f"{exp_id}.json"
    if not path.exists():
        raise FileNotFoundError(f"{path} missing -- run python -m experiments.{exp_id}")
    return json.loads(path.read_text(encoding="utf-8"))

def notes(exp_id):
    return load(exp_id)["notes"]

def pm(mean, sd, digits=4):
    if mean is None:
        return "--"
    if sd is None:
        return f"mean:.{digits}f"
    return f"mean:.{digits}f +/- {sd:.{digits}f}"

def num(value, spec=".4f"):
    """Format a metric, or `--` when it is null.

    The section 3.2 schema requires an inapplicable metric to be null rather than zero, so

```

```

every table cell has to tolerate one. Rung 1b, for instance, scores `PAY_0` as an ordinal
rather than fitting a model, so it has no accuracy at a 0.5 threshold.
"""
return "--" if value is None else format(value, spec)

def frame(exp_id):
    """Raw rows as a DataFrame, for ad-hoc inspection."""
    return pd.json_normalize(load(exp_id)["rows"])

def _cv(r, metric="roc_auc", digits=4):
    cv = r.get("cv") or {}
    return pm(cv.get(f"{metric}_mean"), cv.get(f"{metric}_sd"), digits)

# ----- body tables

def table1_ladder():
    """Table 1: the complexity ladder, which is research question 1 quantified."""
    rows = []
    for r in load("exp01_ladder")["rows"]:
        rows.append({
            "Model": r["model"],
            "Features": r["n_features"],
            "ROC-AUC": _cv(r),
            "PR-AUC": _cv(r, "pr_auc"),
            "Brier": _cv(r, "brier"),
            "Accuracy @0.5": num(r['accuracy_at_0.5'], ".4f"),
            "Recall @0.5": num(r['recall_at_0.5'], ".4f"),
            "Interpretability": r["interpretability"],
            "Cost to read": r["interp_cost"] if r["interp_cost"] else "n/a",
        })
    return pd.DataFrame(rows)

def table2_imbalance():
    """Table 2: imbalance arms. Brier is the column that actually separates them."""
    rows = []
    for r in load("exp03_imbalance")["rows"]:
        rows.append({
            "Arm": r["label"],
            "spw": num(r['spw'], ".2f"),
            "Sampler": r["sampler"] or "--",
            "tau": num(r['oof_threshold'], ".4f"),
            "ROC-AUC": _cv(r),
            "PR-AUC": _cv(r, "pr_auc"),
            "Brier": _cv(r, "brier", 5),
            "OOF recall": num(r['oof_recall'], ".4f"),
            "OOF precision": num(r['oof_precision'], ".4f"),
            "OOF cost (r=6)": num(r['oof_cost_r6'], ".0f"),
        })
    return pd.DataFrame(rows)

def table3_cost(scheme="flat"):
    """Table 3: the cost sweep against Elkan's analytic optimum."""
    rows = []
    for r in load("exp10_cost")["rows"]:
        if r["scheme"] != scheme:
            continue
        rows.append({
            "r = C_FN/C_FP": r["r"],
            "Elkan p* = 1/(1+r)": num(r['elkan_p_star'], ".4f"),
            "Empirical OOF tau": num(r['tau_empirical'], ".4f"),
            "tau - p*": num(r['tau_minus_p_star'], "+.4f"),
            "Recall": num(r['recall'], ".4f"),
            "Precision": num(r['precision'], ".4f"),
            "Min cost": num(r['min_cost'], ".0f"),
            "Cost @0.5": num(r['cost_at_0.5'], ".0f"),
        })

```

```

        "Saving": num(r['saving_vs_0.5'], ",.0f"),
    })
    return pd.DataFrame(rows)

def table4_shap(top=10):
    """Table 4: attribution against rank agreement, replacing the lossy SHAP bar plot."""
    shap_rows = {r["label"]: r for r in load("exp11_shap")["rows"]}
    try:
        stab = {r["label"]: r for r in load("exp12_stability")["rows"]}
    except FileNotFoundError:
        stab = {}
    ordered = sorted(shap_rows.values(), key=lambda r: r["rank"][:top])
    rows = []
    for r in ordered:
        s = stab.get(r["label"], {})
        rows.append({
            "Rank": r["rank"],
            "Feature": r["label"],
            "mean|SHAP| (log-odds)": num(r['mean_abs_shap'], ".4f"),
            "Share": num(r['share_of_total'], ".1%"),
            "Mean SHAP": num(r['mean_shap'], "+.4f"),
            "Group": r["group"],
            "Rank SD (5 seeds)": (num(s['rank_sd_across_seeds'], ".2f")
                                if "rank_sd_across_seeds" in s else "--"),
            "Gain rank": s.get("rank_gain", "--"),
            "Permutation rank": s.get("rank_permutation", "--"),
        })
    return pd.DataFrame(rows)

def table5_fairness(model="full"):
    """Table 5: subgroup summary. The full per-group detail goes to the appendix."""
    rows = []
    for r in load("exp13_fairness")["rows"]:
        if r["model"] != model:
            continue
        rows.append({
            "Attribute": r["attribute"],
            "Groups": r["n_groups"],
            "Observed rate gap (pp)": num(r['observed_rate_gap_pp'], ".2f"),
            "Recall gap": num(r['recall_gap'], ".4f"),
            "FPR gap": num(r['fpr_gap'], ".4f"),
            "ROC-AUC gap": num(r['roc_auc_gap'], ".4f"),
            "Selection-rate ratio": num(r['selection_rate_ratio'], ".3f"),
            "Four-fifths rule": "pass" if r["selection_rate_ratio"] >= 0.8 else "FAIL",
        })
    return pd.DataFrame(rows)

def table_fairness_detail(model="full", attribute=None):
    """Appendix: every subgroup with its Wilson interval."""
    detail = load("exp13_fairness")["config"]["subgroup_detail"]
    rows = []
    for key, groups in detail.items():
        m, attr = key.split("|")
        if m != model or (attribute and attr != attribute):
            continue
        for g in groups:
            rows.append({
                "Attribute": attr,
                "Group": g["group"],
                "n": num(g['n'], ""),
                "Observed rate (%)": num(g['observed_default_rate_pct'], ".2f"),
                "Wilson 95% CI": (f"[{num(g['wilson_lo_pct'], '.2f')}, {num(g['wilson_hi_pct'], '.2f')}]"),
                "ROC-AUC": num(g['roc_auc'], ".4f"),
                "PR-AUC": num(g['pr_auc'], ".4f"),
                "Recall": num(g['recall'], ".4f"),
                "FPR": num(g['fpr'], ".4f"),
                "Flag rate": num(g['selection_rate'], ".4f"),
            })
    return pd.DataFrame(rows)

```

```

def table_test():
    """The hold-out table. Only exp14 is licensed to fill a test block."""
    rows = []
    for r in load("exp14_final")["rows"]:
        t = r["test"]
        rows.append({
            "Configuration": r["label"],
            "Features": r["n_features"],
            "Frozen tau": num(t['threshold'], ".4f"),
            "ROC-AUC": num(t['roc_auc'], ".5f"),
            "PR-AUC": num(t['pr_auc'], ".5f"),
            "Brier": num(t['brier'], ".5f"),
            "Recall": num(t['recall'], ".4f"),
            "Precision": num(t['precision'], ".4f"),
            "F1": num(t['f1'], ".4f"),
            "TP/FP/FN/TN": f"{t['tp']}/{t['fp']}/{t['fn']}/{t['tn']}",
            "Cost (r=6)": num(t['total_cost_r6'], ".0f"),
        })
    return pd.DataFrame(rows)

# ----- appendix tables

def table_ablation(exp_id, extra_cols=None):
    """Generic CV-only ablation table, used for exp04 through exp08."""
    rows = []
    for r in load(exp_id)["rows"]:
        rec = {"Arm": r["label"], "Features": r["n_features"],
            "ROC-AUC": _cv(r), "PR-AUC": _cv(r, "pr_auc"),
            "Brier": _cv(r, "brier", 5)}
        for c in (extra_cols or []):
            if c in r:
                v = r[c]
                rec[c] = num(v, "+.4f") if isinstance(v, float) else v
        rows.append(rec)
    return pd.DataFrame(rows)

def table_hyperparams():
    """Appendix: the selected configuration, with its search range beside it."""
    from src.config import SEARCH_SPACE

    p = load("exp02_tuning")
    best = next(r for r in p["rows"] if r["label"] == "tuned")["params"]
    rows = []
    for name, (kind, lo, hi, log) in SEARCH_SPACE.items():
        v = best.get(name)
        rows.append({
            "Hyperparameter": name,
            "Searched range": f"{lo} to {hi}" + (" (log)" if log else ""),
            "Type": kind,
            "Selected": num(v, ".5g") if isinstance(v, float) else v,
        })
    return pd.DataFrame(rows)

def runtimes():
    """Every experiment's runtime, for the reproduction note in SETUP.md."""
    rows = []
    for path in sorted(RESULTS.glob("exp*.json")):
        p = json.loads(path.read_text(encoding="utf-8"))
        rows.append({"Experiment": p["experiment_id"], "Title": p["title"],
            "Rows": len(p["rows"]),
            "Runtime (s)": round(p["runtime_sec"], 1),
            "Run (UTC)": p["run_utc"]})
    df = pd.DataFrame(rows)
    return df

```

experiments/

experiments/exp01_ladder.py

```
"""WP5 / exp01 -- the complexity ladder. Answers research question 1.
```

Each rung adds modelling capability at a measurable interpretability cost, so the accuracy-versus-explainability trade-off becomes a measured axis rather than an assertion. Rung 1 also recovers the PAY_0-alone ROC-AUC benchmark that was lost with the uncommitted Phase 1 multivariate notebook.

Rungs 2 and 3 are untuned internal reference implementations used to locate XGBoost on a capability axis. They are NOT the group's Logistic Regression or Decision Tree submissions, which other members tune separately.

```
python -m experiments.exp01_ladder
```

```
from __future__ import annotations
```

```
import json
```

```
import numpy as np
```

```
from sklearn.metrics import average_precision_score, roc_auc_score
```

```
from src.config import ARTIFACTS
```

```
from src.data import RAW_FEATURES, training_context
```

```
from src.models import count_tree_nodes, ladder_specs
```

```
from src.runner import Timer, cv_evaluate, fit_one, make_row, save_result
```

```
BEST_PARAMS = ARTIFACTS / "best_params.json"
```

```
def main():
```

```
    with Timer() as t:
```

```
        if not BEST_PARAMS.exists():
```

```
            raise SystemExit(
```

```
                f"MISSING ARTIFACT: {BEST_PARAMS}\n"
```

```
                "Produced by WP6. Run: python -m experiments.exp02_tuning"
```

```
            )
```

```
        best = json.loads(BEST_PARAMS.read_text(encoding="utf-8"))["params"]
```

```
        ctx = training_context(RAW_FEATURES)
```

```
        X_all, y, rfolds = ctx["X"], ctx["y"], ctx["repeated_folds"]
```

```
        prevalence = float(y.mean())
```

```
        rows = []
```

```
        for spec in ladder_specs(best_params=best):
```

```
            X = X_all[spec["features"]]
```

```
            out = cv_evaluate(X, y, spec["factory"], rfolds)
```

```
            oof = out["oof"]
```

```
            # Decisions at the naive 0.5 threshold, to make the accuracy trap explicit.
```

```
            pred = (oof >= 0.5).astype(int)
```

```
            acc = float((pred == y).mean())
```

```
            rec = float(pred[y == 1].mean())
```

```
            # Interpretability cost: the number of parameters or rules a human must read
```

```
            # to reproduce one decision. None means not directly available.
```

```
            cost = spec["interp_cost"]
```

```
            if cost is None:
```

```
                fitted = fit_one(spec["factory"](), X, y)
```

```
                cost = count_tree_nodes(fitted)
```

```
        rows.append(make_row(
```

```
            spec["label"], out["cv"],
```

```
            params=best if spec["label"] == "rung6_xgb_tuned" else {},
```

```
            n_features=X.shape[1],
```

```
            extra={
```

```
                "model": spec["model"],
```

```
                "interpretability": spec["interpretability"],
```

```
                "interp_cost": cost,
```

```

        "accuracy_at_0.5": acc,
        "recall_at_0.5": rec,
    },
))
print(f"{spec['label']:<20} roc_auc {out['cv']['roc_auc_mean']:.4f} "
      f"+/- {out['cv']['roc_auc_sd']:.4f} acc@0.5 {acc:.4f} "
      f"recall@0.5 {rec:.4f} interp_cost {cost}")

# Rung 1b: PAY_0 used directly as an ordinal score, with no model fitted at all.
# The depth-1 tree above binarises PAY_0 into two groups and so throws the
# ordinal gradient away; ranking on the raw code keeps it. The gap between the
# two is the price of insisting on a single readable rule, and it is this row --
# not the tree -- that reproduces the Phase 1 benchmark.
pay0 = X_all["PAY_0"].to_numpy(dtype=float)
score_auc = [roc_auc_score(y[va], pay0[va]) for _, va in rfolds]
score_pr = [average_precision_score(y[va], pay0[va]) for _, va in rfolds]
rows.insert(2, make_row(
    "rung1b_pay0_score",
    {"roc_auc_mean": float(np.mean(score_auc)),
     "roc_auc_sd": float(np.std(score_auc, ddof=1)),
     "pr_auc_mean": float(np.mean(score_pr)),
     "pr_auc_sd": float(np.std(score_pr, ddof=1)),
     "brier_mean": None, "brier_sd": None, "n_fits": len(rfolds)},
    params={}, n_features=1,
    extra={"model": "PAY_0 as an ordinal score (no model fitted)",
           "interpretability": "rank on one column",
           "interp_cost": 1,
           "accuracy_at_0.5": None, "recall_at_0.5": None}))
print(f"{ 'rung1b_pay0_score':<20} roc_auc {np.mean(score_auc):.4f} "
      f"+/- {np.std(score_auc, ddof=1):.4f} (no model: ranking on the raw code)")

by = {r["label"]: r for r in rows}
rule_auc = by["rung1_pay0_rule"]["cv"]["roc_auc_mean"]
score_mean = by["rung1b_pay0_score"]["cv"]["roc_auc_mean"]
tuned_auc = by["rung6_xgb_tuned"]["cv"]["roc_auc_mean"]
tuned_sd = by["rung6_xgb_tuned"]["cv"]["roc_auc_sd"]
forest_auc = by["rung4_forest"]["cv"]["roc_auc_mean"]
default_auc = by["rung5_xgb_default"]["cv"]["roc_auc_mean"]

notes = (
    f"Rung 0 reaches {by['rung0_majority']['accuracy_at_0.5']:.4%} accuracy with "
    f"zero recall, which is the accuracy trap in one line: the majority-class rate "
    f"is 1 - prevalence = {1 - prevalence:.4%}. "
    f"Rung 1, a depth-1 tree on PAY_0, reaches only {rule_auc:.4f} -- short of the "
    f"0.690 Phase 1 reported from the notebook that was never committed. The "
    f"discrepancy is informative rather than an error: a single split binarises "
    f"PAY_0 into two groups and discards the ordinal gradient, so rung 1b ranks on "
    f"the raw code with no model at all and reaches {score_mean:.4f}, recovering "
    f"the benchmark. The {score_mean - rule_auc:+.4f} between them is the measured "
    f"price of insisting on one readable rule, and it is the first rung of the "
    f"interpretability cost this experiment exists to quantify. "
    f"Tuned XGBoost reaches {tuned_auc:.4f} +/- {tuned_sd:.4f}, a gain of "
    f"{tuned_auc - rule_auc:+.4f} over a single readable rule and "
    f"{tuned_auc - default_auc:+.4f} over untuned XGBoost. Bagging alone "
    f"(rung 4) reaches {forest_auc:.4f}, so boosting contributes "
    f"{tuned_auc - forest_auc:+.4f} beyond bagging. "
    f"Rungs 2 and 3 are untuned internal references, not the group's separately "
    f"tuned Logistic Regression and Decision Tree submissions. "
    f"Interpretability cost counts parameters or nodes a human must read to "
    f"reproduce one decision; for the ensembles it is total node count across all "
    f"trees, which is why it is reported as 'not directly available - see SHAP'.")
)
save_result("exp01_ladder", "Complexity ladder: capability against interpretability",
            rows, notes, t.seconds,
            config={"prevalence": prevalence, "n_rungs": len(rows),
                  "cv": "RepeatedStratifiedKFold(5x5, random_state=7)})})

if __name__ == "__main__":
    main()

```

experiments/exp02_tuning.py

""WP6 / exp02 -- Optuna TPE hyperparameter search.

Produces artifacts/best_params.json and artifacts/final_model.json, which every downstream experiment reads. Reports the tuning gain against XGBoost's library defaults (rung 5 of the complexity ladder) on identical folds.

```
""" python -m experiments.exp02_tuning

from __future__ import annotations

import json

import numpy as np

from src.config import ARTIFACTS, N_TRIALS, SEARCH_SPACE, SEED
from src.data import RAW_FEATURES, training_context
from src.models import xgb_classifier, xgb_library_defaults
from src.runner import Timer, cv_evaluate, fit_one, make_row, save_result
from src.tuning import run_study

BEST_PARAMS = ARTIFACTS / "best_params.json"
FINAL_MODEL = ARTIFACTS / "final_model.json"
STUDY_DB = ARTIFACTS / "optuna_study.db"

def main():
    with Timer() as t:
        ctx = training_context(RAW_FEATURES)
        X, y, folds, rfolds = ctx["X"], ctx["y"], ctx["folds"], ctx["repeated_folds"]

        print(f"tuning on {len(X):,} training rows, {X.shape[1]} features, "
              f"{N_TRIALS} trials, {len(folds)}-fold ROC-AUC objective")
        study = run_study(X, y, folds, STUDY_DB, n_trials=N_TRIALS, seed=SEED)
        best = study.best_trial
        params = dict(best.params)
        print(f"best trial #{best.number}: ROC-AUC {best.value:.5f}")
        print(json.dumps(params, indent=2))

        # Refit the selected configuration over the 25 repeated-CV folds for the
        # mean +/- SD the report quotes. The tuner itself never saw these folds.
        tuned = cv_evaluate(X, y, lambda: xgb_classifier(params), rfolds, seed=SEED)
        default = cv_evaluate(X, y, xgb_library_defaults, rfolds, seed=SEED)
        gain = tuned["cv"]["roc_auc_mean"] - default["cv"]["roc_auc_mean"]
        pooled_sd = float(np.hypot(tuned["cv"]["roc_auc_sd"], default["cv"]["roc_auc_sd"]))

        # Final model: one fit on the whole training portion, early stopping against an
        # inner slice of it. This is the model WP14-WP18 explain and evaluate.
        final = fit_one(xgb_classifier(params), X, y, seed=SEED)
        n_trees = int(final.best_iteration) + 1
        final.get_booster().save_model(FINAL_MODEL)

        BEST_PARAMS.write_text(json.dumps({
            "params": params,
            "n_estimators_final": n_trees,
            "best_iteration": int(final.best_iteration),
            "cv_roc_auc_tuning_folds": float(best.value),
            "trial_number": int(best.number),
            "seed": SEED,
            "features": RAW_FEATURES,
        }, indent=2), encoding="utf-8")
        print(f"saved -> {BEST_PARAMS}\nsaved -> {FINAL_MODEL} ({n_trees} trees)")

        n_complete = len([tr for tr in study.trials if tr.state.name == "COMPLETE"])
        n_pruned = len([tr for tr in study.trials if tr.state.name == "PRUNED"])

        rows = [
            make_row("tuned", tuned["cv"], params=params, n_features=X.shape[1],
                    extra={"n_trees": n_trees,
                          "best_iteration_mean": tuned.get("best_iteration_mean")}),
            make_row("library_defaults", default["cv"], params={},
                    n_features=X.shape[1], extra={"n_trees": 100}),
```

```

]

notes = (
    f"Optuna TPE, {n_complete} complete and {n_pruned} pruned trials, MedianPruner. "
    f"Trial budget raised from the planned 60-80 to {N_TRIALS}: measured cost was "
    f"~2.3 s per trial rather than the estimated 15-30 s, so a wider search was free. "
    f"Tuning gain over library defaults on the 25 repeated-CV fits: "
    f"{gain:+.5f} ROC-AUC against a pooled fold-SD of {pooled_sd:.5f} "
    f"({'larger than' if abs(gain) > pooled_sd else 'smaller than'}) one pooled SD). "
    f"scale_pos_weight was excluded from the search space by design: ROC-AUC is "
    f"invariant to monotone score transformations, so the tuner cannot see it. "
    f"Early stopping used an inner 15% slice of each training fold, never the "
    f"validation fold and never the test set."
)
save_result("exp02_tuning", "Hyperparameter search (Optuna TPE)", rows, notes,
            t.seconds,
            config={"n_trials": N_TRIALS, "objective": "roc_auc",
                    "search_space": {k: list(v) for k, v in SEARCH_SPACE.items()},
                    "tuning_gain_roc_auc": gain, "pooled_fold_sd": pooled_sd,
                    "n_complete": n_complete, "n_pruned": n_pruned})

if __name__ == "__main__":
    main()

```

experiments/exp03_imbalance.py

```

"""WP7 / exp03 -- class-imbalance strategy comparison.

Four strategies, each tuned independently so `scale_pos_weight` is a controlled factor
rather than a confound (see src/tuning.py for why it is excluded from the search space).

Arm A is the spw=1 configuration already tuned by exp02; arm B is that same fitted model
read at a cost-minimising threshold instead of 0.5. Arms C and D get their own studies.

Writes artifacts/oof_proba_{label}.npy for WP13 (calibration) and WP14 (cost).

"""
python -m experiments.exp03_imbalance

from __future__ import annotations

import json

import numpy as np

from src.config import ARTIFACTS, N_TRIALS, R_HEADLINE, SEED
from src.costs import cost_at_threshold, pick_threshold_oof
from src.data import RAW_FEATURES, training_context
from src.metrics import evaluate
from src.models import SmoteXGB, scale_pos_weight, xgb_classifier
from src.runner import Timer, cv_evaluate, make_row, save_result
from src.tuning import run_study

BEST_PARAMS = ARTIFACTS / "best_params.json"
SMOTE_TRIALS = 60 # each trial resamples to ~2x rows, so the budget is trimmed

def _oof(X, y, factory, folds, label):
    out = cv_evaluate(X, y, factory, folds)
    np.save(ARTIFACTS / f"oof_proba_{label}.npy", out["oof"])
    return out

def main():
    with Timer() as t:
        if not BEST_PARAMS.exists():
            raise SystemExit(
                f"MISSING ARTIFACT: {BEST_PARAMS}\n"
                "Produced by WP6. Run: python -m experiments.exp02_tuning"
            )
        params_a = json.loads(BEST_PARAMS.read_text(encoding="utf-8"))["params"]

```

```

ctx = training_context(RAW_FEATURES)
X, y, folds, rfolds = ctx["X"], ctx["y"], ctx["folds"], ctx["repeated_folds"]
spw = scale_pos_weight(y)
print(f"scale_pos_weight (neg/pos) = {spw:.4f}")

# --- arm C: independently tuned at spw = neg/pos
study_c = run_study(X, y, folds, ARTIFACTS / "optuna_study_spw.db",
                    study_name="xgb_spw", n_trials=N_TRIALS, spw=spw, seed=SEED)
params_c = dict(study_c.best_trial.params)
print(f"arm C best tuning-fold ROC-AUC {study_c.best_value:.5f}")

# --- arm D: independently tuned with SMOTE inside fit
study_d = run_study(
    X, y, folds, ARTIFACTS / "optuna_study_smote.db",
    study_name="xgb_smote", n_trials=SMOTE_TRIALS, seed=SEED,
    model_fn=lambda p, s, sd: SmoteXGB(params=p, seed=sd),
)
params_d = dict(study_d.best_trial.params)
print(f"arm D best tuning-fold ROC-AUC {study_d.best_value:.5f}")

# WP13 (calibration) and WP17 (fairness) read the reweighted arm's parameters.
(ARTIFACTS / "best_params_spw.json").write_text(json.dumps(
    {"params": params_c, "spw": spw,
     "cv_roc_auc_tuning_folds": float(study_c.best_value)}, indent=2),
    encoding="utf-8")
(ARTIFACTS / "best_params_smote.json").write_text(json.dumps(
    {"params": params_d, "sampler": "SMOTE",
     "cv_roc_auc_tuning_folds": float(study_d.best_value)}, indent=2),
    encoding="utf-8")

configs = {
    "A": (lambda: xgb_classifier(params_a), params_a),
    "C": (lambda: xgb_classifier(params_c, spw=spw), params_c),
    "D": (lambda: SmoteXGB(params=params_d), params_d),
}

# Variance reporting over the 25 repeated-CV fits, and OOF probabilities from
# the 5 tuning folds (which partition the training rows exactly).
cv, oof = {}, {}
for key, (factory, _) in configs.items():
    cv[key] = cv_evaluate(X, y, factory, rfolds, collect_oof=False)
    oof[key] = _oof(X, y, factory, folds, {"A": "A_baseline",
                                           "C": "C_spw",
                                           "D": "D_smote"}[key])["oof"]
    print(f"arm {key}: roc_auc {cv[key]['cv']['roc_auc_mean']:.4f} "
          f"+/- {cv[key]['cv']['roc_auc_sd']:.4f} "
          f"brier {cv[key]['cv']['brier_mean']:.4f}")

tau = {}
for key in configs:
    tau[key], _ = pick_threshold_oof(y, oof[key], r=R_HEADLINE)
print("cost-minimising OOF thresholds:",
      {k: round(v, 4) for k, v in tau.items()})

def decision_extras(key, threshold):
    m = evaluate(y, oof[key], threshold)
    return {
        "oof_threshold": threshold,
        "oof_recall": m["recall"],
        "oof_precision": m["precision"],
        "oof_f1": m["f1"],
        "oof_cost_r6": cost_at_threshold(y, oof[key], threshold, r=R_HEADLINE),
        "spw": 1.0 if key in ("A", "D") else spw,
        "sampler": "SMOTE" if key == "D" else None,
    }

rows = [
    make_row("A_baseline", cv["A"]["cv"], params=params_a, n_features=X.shape[1],
            extra=decision_extras("A", 0.5)),
    make_row("B_threshold", cv["A"]["cv"], params=params_a, n_features=X.shape[1],
            extra={**decision_extras("A", tau["A"]),

```

```

        "note": "identical fitted model to A_baseline; only the "
                "decision threshold differs, so CV ranking and "
                "calibration metrics are the same by construction"}),
make_row("C_spw", cv["C"]["cv"], params=params_c, n_features=X.shape[1],
        extra=decision_extras("C", 0.5)),
make_row("C_spw_threshold", cv["C"]["cv"], params=params_c,
        n_features=X.shape[1], extra=decision_extras("C", tau["C"])),
make_row("D_smote", cv["D"]["cv"], params=params_d, n_features=X.shape[1],
        extra=decision_extras("D", tau["D"])),
]

auc_a = cv["A"]["cv"]["roc_auc_mean"]
auc_c = cv["C"]["cv"]["roc_auc_mean"]
auc_d = cv["D"]["cv"]["roc_auc_mean"]
sd_a = cv["A"]["cv"]["roc_auc_sd"]
br_a = cv["A"]["cv"]["brier_mean"]
br_c = cv["C"]["cv"]["brier_mean"]

notes = (
    f"Reweighting changes ranking by {auc_c - auc_a:+.5f} ROC-AUC against a "
    f"fold-SD of {sd_a:.5f}, so A and C are not distinguishable on ranking. "
    f"Calibration does move: Brier {br_a:.5f} (spw=1) versus {br_c:.5f} "
    f"(spw={spw:.2f}), a {br_c - br_a:+.5f} change. This is a mechanistic result, "
    f"not a null one: reweighting and threshold-moving are two parameterisations "
    f"of the same decision shift, but only threshold-moving preserves probability "
    f"semantics, and the cost analysis in WP14 consumes probabilities. "
    f"SMOTE reaches {auc_d:.5f} ({auc_d - auc_a:+.5f} versus A) and is rejected on "
    f"dataset-specific grounds rather than generic ones: interpolating two clients "
    f"produces values such as PAY_0 = 0.37, a repayment code that does not exist, "
    f"and Phase 1 established PAY_n is non-monotonic and categorical-like. "
    f"max_delta_step was considered and dismissed -- it addresses imbalance beyond "
    f"roughly 100:1, and this dataset is {spw:.1f}:1. "
    f"Thresholds were frozen from out-of-fold training predictions at r=6; the test "
    f"set is not touched by this experiment."
)
save_result("exp03_imbalance", "Class-imbalance strategy comparison", rows, notes,
            t.seconds,
            config={"scale_pos_weight": spw, "r": R_HEADLINE,
                  "thresholds": {k: tau[k] for k in tau},
                  "n_trials_spw": N_TRIALS, "n_trials_smote": SMOTE_TRIALS,
                  "arm_A_params_source": "exp02 (spw=1 study)"})

if __name__ == "__main__":
    main()

```

experiments/exp04_encoding.py

```

"""WP8 / exp04 -- encoding ablation for the repayment-status columns.

```

Native integers versus one-hot versus XGBoost's native categorical support, plus a signed-log transform of the monetary columns to demonstrate empirically that trees are scale-invariant -- the Phase 1 scaling recommendation applied to distance and gradient learners, not to this one.

```

""" python -m experiments.exp04_encoding

```

```

from __future__ import annotations

```

```

import json

```

```

import numpy as np
import pandas as pd

```

```

from src.config import ARTIFACTS
from src.data import BILL_COLS, PAY_COLS, PAYAMT_COLS, RAW_FEATURES, training_context
from src.models import xgb_classifier
from src.runner import Timer, cv_evaluate, make_row, per_fold_metric, save_result
from src.metrics import paired_fold_test

```

```

BEST_PARAMS = ARTIFACTS / "best_params.json"
MONETARY = ["LIMIT_BAL"] + BILL_COLS + PAYAMT_COLS

```

```

def main():
    with Timer() as t:
        if not BEST_PARAMS.exists():
            raise SystemExit(f"MISSING ARTIFACT: {BEST_PARAMS}\nProduced by WP6: "
                             "python -m experiments.exp02_tuning")
        params = json.loads(BEST_PARAMS.read_text(encoding="utf-8"))["params"]

        ctx = training_context(RAW_FEATURES)
        X, y, rfolds = ctx["X"], ctx["y"], ctx["repeated_folds"]

        # The dummy vocabulary is built from the full training frame, so folds share a
        # column set. That is structural encoding, not target leakage -- no fold sees
        # another fold's labels.
        X_onehot = pd.get_dummies(X, columns=PAY_COLS, dtype=np.int8)
        X_cat = X.copy()
        for c in PAY_COLS:
            X_cat[c] = X_cat[c].astype("category")
        X_log = X.copy()
        for c in MONETARY:
            X_log[c] = np.sign(X_log[c]) * np.log1p(np.abs(X_log[c]))

        arms = [
            ("native_int", X, dict(params), False,
             "PAY_n as signed integers, as loaded"),
            ("one_hot", X_onehot, dict(params), False,
             f"PAY_n expanded to {X_onehot.shape[1] - X.shape[1] + len(PAY_COLS)} dummies"),
            ("native_categorical", X_cat, dict(params), True,
             "PAY_n as pandas category dtype, enable_categorical=True"),
            ("signed_log_monetary", X_log, dict(params), False,
             "sign(x)*log1p(|x|) on LIMIT_BAL, BILL_AMT and PAY_AMT"),
        ]

        rows, folds_auc = [], {}
        for label, Xa, p, cat, desc in arms:
            out = cv_evaluate(Xa, y, lambda p=p, cat=cat: xgb_classifier(p, enable_categorical=cat),
                              rfolds, collect_oof=False)
            folds_auc[label] = per_fold_metric(out)
            rows.append(make_row(label, out["cv"], params=p, n_features=Xa.shape[1],
                                extra={"description": desc}))
            print(f"{label:<22} n_feat {Xa.shape[1]:>3} roc_auc "
                  f"{out['cv']['roc_auc_mean']:.4f} +/- {out['cv']['roc_auc_sd']:.4f}")

        base = folds_auc["native_int"]
        tests = {k: paired_fold_test(v - base) for k, v in folds_auc.items()
                  if k != "native_int"}
        spread = max(r["cv"]["roc_auc_mean"] for r in rows) - \
            min(r["cv"]["roc_auc_mean"] for r in rows)
        sd = rows[0]["cv"]["roc_auc_sd"]

        notes = (
            f"All four encodings sit within {spread:.5f} ROC-AUC of each other against a "
            f"fold-SD of {sd:.5f}, so none is distinguishable from native integers. "
            f"Native integers are carried forward. Trees make axis-aligned splits, so a "
            f"non-monotonic response to PAY_n is recoverable by stacking splits "
            f"(PAY_0 < 0.5, then PAY_0 < -1.5) -- that costs depth, not expressible "
            f"functions. Integers also keep the six PAY_n columns distinct, protecting the "
            f"recency gradient Phase 1 measured (Spearman rho 0.143 rising to 0.292). "
            f"One-hot expands to {rows[1]['n_features']} columns for no measurable gain. "
            f"The signed-log arm confirms scale invariance: a monotone transform of every "
            f"monetary column moves ROC-AUC by "
            f"{tests['signed_log_monetary']['mean_delta']:.5f}, because axis-aligned splits "
            f"depend only on rank order. Paired per-fold t-tests are reported, with the "
            f"Nadeau-Bengio (2003) caveat that repeated-CV tests are anti-conservative "
            f"because the training folds overlap. "
            f>Note that enable_categorical=True blocks interventional TreeSHAP in shap "
            f"0.52, which is a second reason to prefer integers for a model that must be "
            f"explained."
        )
    save_result("exp04_encoding", "Encoding ablation for repayment-status columns",
               rows, notes, t.seconds,

```

```

config={"paired_fold_tests_vs_native_int": tests,
       "auc_spread": spread})

```

```

if __name__ == "__main__":
    main()

```

experiments/exp05_engineered.py

```

"""WP9 / exp05 -- engineered-feature benchmark. Promised in Phase 1 section 2.2.2.

```

Phase 1 committed to benchmarking aggregate features rather than assuming they help, and predicted that aggregation may *lose* signal by collapsing the recency gradient. This tests that prediction.

```

python -m experiments.exp05_engineered
"""

```

```

from __future__ import annotations

```

```

import json

```

```

import numpy as np

```

```

from src.config import ARTIFACTS
from src.data import ENGINEERED, RAW_FEATURES, training_context
from src.metrics import paired_fold_test
from src.models import xgb_classifier
from src.runner import Timer, cv_evaluate, make_row, per_fold_metric, save_result

```

```

BEST_PARAMS = ARTIFACTS / "best_params.json"

```

```

def main():
    with Timer() as t:
        if not BEST_PARAMS.exists():
            raise SystemExit(f"MISSING ARTIFACT: {BEST_PARAMS}\nProduced by WP6: "
                             "python -m experiments.exp02_tuning")
        params = json.loads(BEST_PARAMS.read_text(encoding="utf-8"))["params"]

        ctx = training_context(RAW_FEATURES + ENGINEERED, engineered=True)
        X_all, y, folds, rfolds = ctx["X"], ctx["y"], ctx["folds"], ctx["repeated_folds"]

        # RFE to 23 features, so the "raw + engineered" arm is compared at equal width
        # rather than simply being given more columns.
        from sklearn.feature_selection import RFE

        selector = RFE(xgb_classifier(params, early_stopping=False),
                       n_features_to_select=len(RAW_FEATURES), step=3)
        selector.fit(X_all, y)
        rfe_cols = list(X_all.columns[selector.support_])
        print(f"RFE kept {len(rfe_cols)} of {X_all.shape[1]}: {rfe_cols}")

        arms = [
            ("raw_23", RAW_FEATURES),
            ("raw_plus_engineered", RAW_FEATURES + ENGINEERED),
            ("engineered_only", ENGINEERED),
            ("rfe_top23", rfe_cols),
        ]

        rows, folds_auc = [], {}
        for label, cols in arms:
            Xa = X_all[cols]
            out = cv_evaluate(Xa, y, lambda: xgb_classifier(params), rfolds,
                              collect_oof=False)
            folds_auc[label] = per_fold_metric(out)
            rows.append(make_row(label, out["cv"], params=params, n_features=len(cols),
                                 extra={"features": cols,
                                         "n_engineered": len([c for c in cols
                                                                if c in ENGINEERED])}))

        print(f"{label:<22} n_feat {len(cols):>3} roc_auc "
              f"{out['cv']['roc_auc_mean']:.4f} +/- {out['cv']['roc_auc_sd']:.4f}")

```

```

base = folds_auc["raw_23"]
tests = {k: paired_fold_test(v - base) for k, v in folds_auc.items() if k != "raw_23"}
by = {r["label"]: r["cv"]["roc_auc_mean"] for r in rows}
sd = rows[0]["cv"]["roc_auc_sd"]
d_add = by["raw_plus_engineered"] - by["raw_23"]
d_only = by["engineered_only"] - by["raw_23"]
verdict = ("confirmed" if d_only < -sd else
           "refuted" if d_only > sd else "neither confirmed nor refuted")

notes = (
    f"Adding the 17 engineered features to the raw 23 moves ROC-AUC by "
    f"{d_add:+.5f} against a fold-SD of {sd:.5f}. Replacing the raw columns "
    f"entirely with the engineered ones moves it by {d_only:+.5f}. "
    f"Phase 1 predicted aggregation would lose the recency gradient; on this "
    f"evidence that prediction is {verdict} at the one-SD level. "
    f"RFE to {len(rfe_cols)} features from the combined pool provides an "
    f"equal-width comparison, so the raw-plus-engineered arm is not simply "
    f"rewarded for having more columns. "
    f"PAY_RATIO_n is NaN where the statement balance is zero or negative "
    f"(1,930 rows carry a negative BILL_AMT per Phase 1 section 1.6); XGBoost "
    f"learns a default direction for missing values natively rather than "
    f"requiring imputation, which is why NaN was preferred to a sentinel."
)
save_result("exp05_engineered", "Engineered-feature benchmark", rows, notes,
            t.seconds,
            config={"engineered": ENGINEERED, "rfe_selected": rfe_cols,
                  "paired_fold_tests_vs_raw_23": tests})

if __name__ == "__main__":
    main()

```

experiments/exp06_groups.py

```

"""WP10 / exp06 -- feature-group ablation.

```

Leave-one-group-out and only-one-group over FEATURE_GROUPS, quantifying where the signal actually lives and testing the Phase 1 conclusion that repayment status dominates the monetary columns.

The no-demographics arm is kept as a named row because WP17 reuses it to test whether removing demographic columns removes demographic influence or merely hides it.

```

""" python -m experiments.exp06_groups

```

```

from __future__ import annotations

```

```

import json

```

```

from src.config import ARTIFACTS
from src.data import FEATURE_GROUPS, RAW_FEATURES, training_context
from src.models import xgb_classifier
from src.runner import Timer, cv_evaluate, make_row, save_result

```

```

BEST_PARAMS = ARTIFACTS / "best_params.json"

```

```

def main():
    with Timer() as t:
        if not BEST_PARAMS.exists():
            raise SystemExit(f"MISSING ARTIFACT: {BEST_PARAMS}\nProduced by WP6: "
                             f"python -m experiments.exp02_tuning")
        params = json.loads(BEST_PARAMS.read_text(encoding="utf-8"))["params"]

        ctx = training_context(RAW_FEATURES)
        X_all, y, rfold = ctx["X"], ctx["y"], ctx["repeated_folds"]

        arms = [("all_groups", RAW_FEATURES, "all five groups")]
        for g, cols in FEATURE_GROUPS.items():
            arms.append((f"drop_{g}", [c for c in RAW_FEATURES if c not in cols],

```

```

        f"leave-one-out: {g} removed"))
for g, cols in FEATURE_GROUPS.items():
    arms.append((f"only_{g}", list(cols), f"only-one: {g} alone"))

rows = []
for label, cols, desc in arms:
    out = cv_evaluate(X_all[cols], y, lambda: xgb_classifier(params), rfolds,
                      collect_oof=False)
    rows.append(make_row(label, out["cv"], params=params, n_features=len(cols),
                        extra={"description": desc, "features": cols}))
    print(f"{label:<22} n_feat {len(cols):>3} roc_auc "
          f"{out['cv']['roc_auc_mean']:.4f} +/- {out['cv']['roc_auc_sd']:.4f}")

by = {r["label"]: r["cv"]["roc_auc_mean"] for r in rows}
sd = rows[0]["cv"]["roc_auc_sd"]
full = by["all_groups"]
drops = {g: full - by[f"drop_{g}"] for g in FEATURE_GROUPS}
onlys = {g: by[f"only_{g}"] for g in FEATURE_GROUPS}
worst_drop = max(drops, key=drops.get)
best_only = max(onlys, key=onlys.get)
demo_cost = drops["demographic"]

notes = (
    f"Removing {worst_drop} costs {drops[worst_drop]:.5f} ROC-AUC, the largest "
    f"leave-one-out loss, against a fold-SD of {sd:.5f}. "
    f"{best_only} alone reaches {onlys[best_only]:.5f} versus {full:.5f} for the "
    f"full feature set, so it carries most of the available signal on its own. "
    f"This confirms the Phase 1 conclusion that repayment status dominates the "
    f"monetary columns. "
    f"Leave-one-out and only-one disagree in the usual way: a group can be "
    f"individually predictive yet redundant once the others are present, which is "
    f"exactly what collinear BILL_AMT columns produce. Both views are reported "
    f"rather than one. "
    f"Dropping the demographic group costs {demo_cost:+.5f} ROC-AUC; the "
    f"drop_demographic arm is consumed by WP17, which tests whether the financial "
    f"columns proxy for the removed demographics (Barocas & Selbst, 2016).")
)
save_result("exp06_groups", "Feature-group ablation", rows, notes, t.seconds,
            config={"groups": FEATURE_GROUPS,
                  "leave_one_out_loss": drops, "only_one_auc": onlys})

if __name__ == "__main__":
    main()

```

experiments/exp07_recency.py

```

"""WP11 / exp07 -- recency ablation.

Most-recent-k-months only, k in 1..6, using the PAY_CHRONO ordering. Tests the Phase 1
recency gradient directly and answers the practical question a lender would ask: how
much billing history do we actually need to collect?

"""
python -m experiments.exp07_recency

from __future__ import annotations

import json

from src.config import ARTIFACTS
from src.data import (
    BILL_COLS,
    DEMOGRAPHIC,
    PAY_CHRONO,
    PAYAMT_COLS,
    RAW_FEATURES,
    training_context,
)
from src.models import xgb_classifier
from src.runner import Timer, cv_evaluate, make_row, save_result

BEST_PARAMS = ARTIFACTS / "best_params.json"

```

```

# Chronological, April -> September. Index -k: gives the most recent k months.
BILL_CHRONO = BILL_COLS[::-1]
PAYAMT_CHRONO = PAYAMT_COLS[::-1]
STATIC = ["LIMIT_BAL"] + DEMOGRAPHIC

def main():
    with Timer() as t:
        if not BEST_PARAMS.exists():
            raise SystemExit(f"MISSING ARTIFACT: {BEST_PARAMS}\nProduced by WP6: "
                             "python -m experiments.exp02_tuning")
        params = json.loads(BEST_PARAMS.read_text(encoding="utf-8"))["params"]

        ctx = training_context(RAW_FEATURES)
        X_all, y, rfolds = ctx["X"], ctx["y"], ctx["repeated_folds"]

        rows = []
        for k in range(1, 7):
            cols = (STATIC + PAY_CHRONO[-k:] + BILL_CHRONO[-k:] + PAYAMT_CHRONO[-k:])
            out = cv_evaluate(X_all[cols], y, lambda: xgb_classifier(params), rfolds,
                             collect_oof=False)
            rows.append(make_row(f"k{k}_months", out["cv"], params=params,
                                 n_features=len(cols),
                                 extra={"k_months": k, "features": cols,
                                         "months": PAY_CHRONO[-k:])))
            print(f"k={k} n_feat {len(cols):>3} roc_auc "
                  f"{out['cv']['roc_auc_mean']:.4f} +/- {out['cv']['roc_auc_sd']:.4f}")

        by = {r["label"]: r["cv"]["roc_auc_mean"] for r in rows}
        sd = rows[-1]["cv"]["roc_auc_sd"]
        k1, k6 = by["k1_months"], by["k6_months"]
        # Smallest k whose ROC-AUC is within one fold-SD of the full six months.
        enough = next(k for k in range(1, 7) if by[f"k{k}_months"] >= k6 - sd)
        gains = {k: by[f"k{k}_months"] - by[f"k{k - 1}_months"] for k in range(2, 7)}

        notes = (
            f"One month of history (September only) already reaches ROC-AUC {k1:.5f}, "
            f"against {k6:.5f} for all six months -- a total gain of {k6 - k1:+.5f} from "
            f"five extra months, against a fold-SD of {sd:.5f}. "
            f"k={enough} is the smallest history window that lands within one fold-SD of "
            f"the full six months, so most of the collectable signal is in the most recent "
            f"statement. This is the recency gradient of Phase 1 (Spearman rho rising from "
            f"0.143 at April to 0.292 at September) restated as a data-collection cost. "
            f"Marginal gain per additional month: "
            f"{', '.join(f'k{k}: {v:+.5f}' for k, v in gains.items())}. "
            f"Practical reading for a lender: the fifth and sixth months of history are "
            f"close to free of predictive value here, which matters because history depth "
            f"is an acquisition cost and a privacy exposure, not just a modelling choice."
        )
        save_result("exp07_recency", "Recency ablation: months of history required",
                    rows, notes, t.seconds,
                    config={"chronological_order": PAY_CHRONO,
                           "static_features": STATIC,
                           "smallest_k_within_one_sd": enough,
                           "marginal_gain_per_month": gains})

if __name__ == "__main__":
    main()

```

experiments/exp08_learning_curve.py

```

"""WP12 / exp08 -- learning curve.

```

Training and validation ROC-AUC against training-set fraction, five subsample seeds per fraction. This is the evidence for the feature-set-ceiling claim: a validation curve that has plateaued means more of the same data will not help, which is a stronger statement than asserting it.

```

""" python -m experiments.exp08_learning_curve

```

```

from __future__ import annotations

import json

import numpy as np
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split

from src.config import ARTIFACTS, SEED
from src.data import RAW_FEATURES, training_context
from src.models import xgb_classifier
from src.runner import Timer, fit_one, make_row, save_result

BEST_PARAMS = ARTIFACTS / "best_params.json"
FRACTIONS = [0.10, 0.25, 0.50, 0.75, 1.00]
SEEDS = [SEED + i for i in range(5)]

def main():
    with Timer() as t:
        if not BEST_PARAMS.exists():
            raise SystemExit(f"MISSING ARTIFACT: {BEST_PARAMS}\nProduced by WP6: "
                             "python -m experiments.exp02_tuning")
        params = json.loads(BEST_PARAMS.read_text(encoding="utf-8"))["params"]

        ctx = training_context(RAW_FEATURES)
        X, y, folds = ctx["X"], ctx["y"], ctx["folds"]

        rows = []
        for frac in FRACTIONS:
            tr_auc, va_auc, n_used = [], [], []
            for seed in SEEDS:
                for tr, va in folds:
                    if frac < 1.0:
                        sub, _ = train_test_split(tr, train_size=frac, stratify=y[tr],
                                                  random_state=seed)
                    else:
                        sub = tr
                    est = fit_one(xgb_classifier(params, seed=seed), X.iloc[sub], y[sub],
                                  seed=seed)
                    tr_auc.append(roc_auc_score(y[sub], est.predict_proba(X.iloc[sub]))[:, 1]))
                    va_auc.append(roc_auc_score(y[va], est.predict_proba(X.iloc[va]))[:, 1]))
                    n_used.append(len(sub))
            cv = {
                "roc_auc_mean": float(np.mean(va_auc)),
                "roc_auc_sd": float(np.std(va_auc, ddof=1)),
                "pr_auc_mean": None, "pr_auc_sd": None,
                "brier_mean": None, "brier_sd": None,
                "n_fits": len(va_auc),
            }
            rows.append(make_row(f"frac_{frac:.2f}", cv, params=params,
                                n_features=X.shape[1],
                                extra={"fraction": frac,
                                       "n_train_rows_mean": float(np.mean(n_used)),
                                       "train_roc_auc_mean": float(np.mean(tr_auc)),
                                       "train_roc_auc_sd": float(np.std(tr_auc, ddof=1)),
                                       "gap_train_minus_val": float(np.mean(tr_auc) -
                                                                           np.mean(va_auc))}))
            print(f"frac {frac:.2f}  n~{int(np.mean(n_used)):>6,}  "
                  f"val {np.mean(va_auc):.4f} +/- {np.std(va_auc, ddof=1):.4f}  "
                  f"train {np.mean(tr_auc):.4f}  gap {np.mean(tr_auc)-np.mean(va_auc):+.4f}")

        by = {r["fraction"]: r for r in rows}
        full = by[1.00]["cv"]["roc_auc_mean"]
        half = by[0.50]["cv"]["roc_auc_mean"]
        sd = by[1.00]["cv"]["roc_auc_sd"]
        last_step = full - by[0.75]["cv"]["roc_auc_mean"]
        gap = by[1.00]["gap_train_minus_val"]

        notes = (
            f"Validation ROC-AUC rises from {by[0.10]['cv']['roc_auc_mean']:.5f} at 10% of "

```

```

        f"the training data to {full:.5f} at 100%. Half the data already reaches "
        f"{half:.5f}, and the final step from 75% to 100% adds {last_step:+.5f} against "
        f"a fold-SD of {sd:.5f}. The curve has therefore flattened: more rows of the "
        f"same six-month billing snapshot would not materially help. "
        f"The train-validation gap at full data is {gap:+.5f}, so the plateau is a "
        f"feature-set ceiling rather than a high-variance overfit that more data would "
        f"close. Together with the small tuned-versus-untuned gap from exp02, this is "
        f"the evidence behind the WP18 argument that performance is limited by what a "
        f"billing snapshot can observe -- it cannot see job loss, illness or income "
        f"shocks -- not by model capacity."
    )
    save_result("exp08_learning_curve", "Learning curve against training-set size",
               rows, notes, t.seconds,
               config={"fractions": FRACTIONS, "seeds": SEEDS,
                      "fits_per_fraction": len(SEEDS) * len(folds)})

if __name__ == "__main__":
    main()

```

experiments/exp09_calibration.py

```

"""WP13 / exp09 -- calibration.

```

Mandatory rather than optional: WP14's cost analysis converts probabilities into decisions through Elkan's threshold, and uncalibrated probabilities invalidate that.

Each fold splits its own training half into a fit portion and a calibration portion, so the calibrator never sees the fold's validation rows and the test set is untouched. Reliability curves are serialised so WP19 can plot without refitting.

```

""" python -m experiments.exp09_calibration

from __future__ import annotations

import json

import numpy as np
from sklearn.calibration import CalibratedClassifierCV
from sklearn.frozen import FrozenEstimator
from sklearn.metrics import average_precision_score, brier_score_loss, roc_auc_score
from sklearn.model_selection import train_test_split

from src.config import ARTIFACTS, SEED
from src.data import RAW_FEATURES, training_context
from src.metrics import reliability
from src.models import scale_pos_weight, xgb_classifier
from src.runner import Timer, fit_one, make_row, save_result

BEST_PARAMS = ARTIFACTS / "best_params.json"
SPW_PARAMS = ARTIFACTS / "best_params_spw.json"
CALIB_SIZE = 0.20
N_BINS = 10

def main():
    with Timer() as t:
        if not BEST_PARAMS.exists():
            raise SystemExit(f"MISSING ARTIFACT: {BEST_PARAMS}\nProduced by WP6: "
                             f"python -m experiments.exp02_tuning")
        params_a = json.loads(BEST_PARAMS.read_text(encoding="utf-8"))["params"]
        # exp03 writes the spw-arm parameters; fall back to arm A's if it has not run.
        params_c = (json.loads(SPW_PARAMS.read_text(encoding="utf-8"))["params"]
                     if SPW_PARAMS.exists() else params_a)

        ctx = training_context(RAW_FEATURES)
        X, y, folds = ctx["X"], ctx["y"], ctx["folds"]
        spw = scale_pos_weight(y)

        arms = {
            "A_spw1": (params_a, 1.0),

```

```

    "C_spw": (params_c, spw),
}
methods = ["raw", "sigmoid", "isotonic"]

# oof[arm][method] accumulates one probability per training row.
oof = {a: {m: np.full(len(y), np.nan) for m in methods} for a in arms}

for arm, (params, arm_spw) in arms.items():
    for tr, va in folds:
        i_fit, i_cal = train_test_split(tr, test_size=CALIB_SIZE,
                                       stratify=y[tr], random_state=SEED)
        base = fit_one(xgb_classifier(params, spw=arm_spw),
                      X.iloc[i_fit], y[i_fit], seed=SEED)
        oof[arm]["raw"][va] = base.predict_proba(X.iloc[va])[:, 1]
        for m in ("sigmoid", "isotonic"):
            # sklearn 1.9 removed cv="prefit"; FrozenEstimator is the replacement,
            # so `fit` below trains the calibration map only, never the booster.
            cal = CalibratedClassifierCV(FrozenEstimator(base), method=m)
            cal.fit(X.iloc[i_cal], y[i_cal])
            oof[arm][m][va] = cal.predict_proba(X.iloc[va])[:, 1]

rows, curves = [], {}
for arm in arms:
    for m in methods:
        p = oof[arm][m]
        assert not np.isnan(p).any(), f"{arm}/{m} left rows unpredicted"
        label = f"{arm}_{m}"
        cv = {
            "roc_auc_mean": float(roc_auc_score(y, p)),
            "roc_auc_sd": None,
            "pr_auc_mean": float(average_precision_score(y, p)),
            "pr_auc_sd": None,
            "brier_mean": float(brier_score_loss(y, p)),
            "brier_sd": None,
            "n_fits": len(folds),
        }
        curves[label] = reliability(y, p, n_bins=N_BINS)
        # Expected calibration error over the same bins.
        ece = float(sum(b["n"] * abs(b["mean_pred"] - b["frac_pos"])
                       for b in curves[label] if b["n"] > 0) / len(y))
        rows.append(make_row(label, cv, params=arms[arm][0],
                             n_features=X.shape[1],
                             extra={"arm": arm, "method": m, "ece": ece,
                                    "spw": arms[arm][1],
                                    "mean_predicted": float(p.mean())}))
        print(f"{label}<20} brier {cv['brier_mean']:.5f} ece {ece:.5f} "
              f"roc_auc {cv['roc_auc_mean']:.5f} mean_p {p.mean():.4f}")
        np.save(ARTIFACTS / f"oof_calib_{label}.npy", p)

by = {r["label"]: r for r in rows}
prevalence = float(y.mean())
a_raw, c_raw = by["A_spw1_raw"], by["C_spw_raw"]
best = min(rows, key=lambda r: r["cv"]["brier_mean"])

notes = (
    f"Prevalence is {prevalence:.4f}. The spw=1 model predicts a mean probability "
    f"of {a_raw['mean_predicted']:.4f} -- essentially the base rate -- with Brier "
    f"{a_raw['cv']['brier_mean']:.5f} and ECE {a_raw['ece']:.5f}. Reweighting to "
    f"spw={spw:.2f} inflates the mean prediction to {c_raw['mean_predicted']:.4f} "
    f"and worsens Brier to {c_raw['cv']['brier_mean']:.5f} (ECE "
    f"{c_raw['ece']:.5f}), because scale_pos_weight shifts the score distribution "
    f"without changing the underlying event rate. That is the concrete cost of "
    f"reweighting that ROC-AUC cannot see. "
    f"Best Brier overall: {best['label']} at {best['cv']['brier_mean']:.5f}. "
    f"Arm B of exp03 is omitted here because it is arm A's fitted model read at a "
    f"different threshold -- its probabilities are identical, which is precisely "
    f"why threshold-moving is preferable to reweighting when probabilities are "
    f"consumed downstream. "
    f"Reliability curves ({N_BINS} equal-width bins) are serialised in config for "
    f"WP19."
)
save_result("exp09_calibration", "Calibration: Brier, ECE and reliability curves",
            rows, notes, t.seconds,

```

```

        config={"n_bins": N_BINS, "calib_size": CALIB_SIZE,
               "prevalence": prevalence, "scale_pos_weight": spw,
               "reliability_curves": curves,
               "params_c_source": ("exp03" if SPW_PARAMS.exists()
                                  else "fallback to arm A params"))}

```

```

if __name__ == "__main__":
    main()

```

experiments/exp10_cost.py

```

"""WP14 / exp10 -- cost-sensitive threshold selection.

```

The cost ratio is anchored in credit-risk parameters rather than chosen for convenience: a false negative costs LGD x EAD and a false positive costs the forgone net interest margin, giving $r = 0.70 / 0.12 = 5.8$, reported as 6:1.

The sweep over r is overlaid with Elkan's (2001) analytic optimum $p^* = 1/(1+r)$. If the empirical out-of-fold optimum tracks the analytic curve, that is direct evidence the probabilities are calibrated -- which ties calibration, cost and threshold selection into one result.

All thresholds are chosen on out-of-fold *training* predictions and frozen. The test set is not touched here.

```

""" python -m experiments.exp10_cost

```

```

from __future__ import annotations

```

```

import numpy as np

```

```

from src.config import ARTIFACTS, LGD, NIM, R_GRID, R_HEADLINE
from src.costs import cost_at_threshold, elkan_p_star, pick_threshold_oof
from src.data import RAW_FEATURES, training_context
from src.metrics import evaluate
from src.runner import Timer, make_row, save_result

```

```

SCHEMES = {
    "flat": "raw probabilities, unit cost per error",
    "flat_isotonic": "isotonic-calibrated probabilities, unit cost per error",
    "instance": "raw probabilities, per-client cost using LIMIT_BAL as an EAD proxy",
}

```

```

def _load(name, wp):
    path = ARTIFACTS / name
    if not path.exists():
        raise SystemExit(f"MISSING ARTIFACT: {path}\nProduced by {wp}.")
    return np.load(path)

```

```

def main():
    with Timer() as t:
        oof_raw = _load("oof_proba_A_baseline.npy",
                       "WP7: python -m experiments.exp03_imbalance")
        iso_path = ARTIFACTS / "oof_calib_A_spw1_isotonic.npy"
        oof_iso = np.load(iso_path) if iso_path.exists() else None
        if oof_iso is None:
            print("note: isotonic OOF not found, skipping the calibrated scheme "
                  "(run exp09_calibration first for the full sweep)")

```

```

        ctx = training_context(RAW_FEATURES)
        y, X = ctx["y"], ctx["X"]
        ead = X["LIMIT_BAL"].to_numpy(dtype=float)

```

```

        rows, curves = [], {}
        for r in R_GRID:
            p_star = elkan_p_star(r)
            for scheme, desc in SCHEMES.items():
                if scheme == "flat_isotonic" and oof_iso is None:

```

```

        continue
    proba = oof_iso if scheme == "flat_isotonic" else oof_raw
    use_ead = ead if scheme == "instance" else None
    tau, sweep = pick_threshold_oof(y, proba, r=r, ead=use_ead)
    curves[f"r{r}_{scheme}"] = {
        "grid": sweep["grid"][:,10], "cost": sweep["cost"][:,10],
    }
    m = evaluate(y, proba, tau)
    cost_star = cost_at_threshold(y, proba, p_star, r=r, ead=use_ead)
    cost_half = cost_at_threshold(y, proba, 0.5, r=r, ead=use_ead)
    rows.append(make_row(
        f"r{r}_{scheme}", None, n_features=X.shape[1],
        extra={
            "r": r, "scheme": scheme, "description": desc,
            "tau_empirical": tau,
            "elkan_p_star": p_star,
            "tau_minus_p_star": tau - p_star,
            "min_cost": sweep["min_cost"],
            "cost_at_elkan": cost_star,
            "cost_at_0.5": cost_half,
            "saving_vs_0.5": cost_half - sweep["min_cost"],
            "recall": m["recall"], "precision": m["precision"],
            "tp": m["tp"], "fp": m["fp"], "fn": m["fn"], "tn": m["tn"],
        })
    )
    print(f"r={r:<3} {scheme:<14} tau {tau:.4f} elkan {p_star:.4f} "
          f"diff {tau - p_star:+.4f} recall {m['recall']:.4f} "
          f"cost {sweep['min_cost']:, .0f}")

def track(scheme):
    d = [r["tau_minus_p_star"] for r in rows if r["scheme"] == scheme]
    return float(np.mean(np.abs(d))) if d else None

mae_flat = track("flat")
mae_iso = track("flat_isotonic")
head = next(r for r in rows if r["r"] == R_HEADLINE and r["scheme"] == "flat")
head_inst = next(r for r in rows if r["r"] == R_HEADLINE
                  and r["scheme"] == "instance")

notes = (
    f"Cost ratio derived from credit-risk parameters: LGD={LGD} and NIM={NIM} give "
    f"r = {LGD / NIM:.2f}, reported as {R_HEADLINE}:1. At r={R_HEADLINE}, Elkan's "
    f"analytic optimum is  $p^* = 1/(1+r) = \{elkan\_p\_star(R\_HEADLINE):.4f\}$  and the "
    f"empirical out-of-fold optimum is {head['tau_empirical']:.4f}, a difference of "
    f"{head['tau_minus_p_star']:+.4f}. Mean absolute deviation from the analytic "
    f"curve across the whole sweep is {mae_flat:.4f} for raw probabilities"
    + (f" and {mae_iso:.4f} after isotonic calibration, so calibration tightens the "
       f"agreement." if mae_iso is not None else ".")
    + f" That the empirical optimum tracks  $1/(1+r)$  at all is direct evidence the "
    f"probabilities carry the right scale, not merely the right ranking. "
    f"Moving from the default 0.5 to the frozen threshold at r={R_HEADLINE} saves "
    f"{head['saving_vs_0.5']:, .0f} cost units and lifts recall from "
    f"{next(r for r in rows if r['r'] == 1 and r['scheme'] == 'flat')['recall']:.4f} "
    f"at r=1 to {head['recall']:.4f}. "
    f"Instance-specific costing using LIMIT_BAL as an EAD proxy selects "
    f"{head_inst['tau_empirical']:.4f} instead of {head['tau_empirical']:.4f}: "
    f"weighting each error by exposure shifts the optimum because large-limit "
    f"defaults dominate the loss. Both schemes are reported. "
    f"The 9.8M-customer and $1.17bn figures from the proposal are motivation for "
    f"caring about cost asymmetry; they are not presented as a result of this model."
)
save_result("exp10_cost", "Cost-sensitive threshold selection with Elkan overlay",
            rows, notes, t.seconds,
            config={
                "r_grid": R_GRID, "LGD": LGD, "NIM": NIM,
                "r_headline": R_HEADLINE,
                "elkan": {r: elkan_p_star(r) for r in R_GRID},
                "mae_vs_elkan": {"flat": mae_flat, "flat_isotonic": mae_iso},
                "cost_curves": curves})

if __name__ == "__main__":
    main()

```

experiments/exp11_shap.py

```
"""WP15 / exp11 -- TreeSHAP attribution. Answers research question 2.
```

Explanations are computed on held-out rows, because "can post-hoc explanation provide regulatory-compliant decision auditability" is a question about decisions the model was not fitted on. SHAP explains the model's output; it does not establish a causal mechanism, and that distinction is stated in the report.

Attribution is in log-odds (margin) space, where TreeSHAP is exactly additive. The probability transform is non-additive, so it is used only for the single-client waterfall.

```
""" python -m experiments.exp11_shap

from __future__ import annotations

import json

import numpy as np
import shap
from scipy import stats

from src.config import ARTIFACTS, SEED
from src.data import BILL_COLS, PAY_COLS, RAW_FEATURES, test_context, training_context
from src.models import xgb_classifier
from src.runner import Timer, fit_one, make_row, save_result

BEST_PARAMS = ARTIFACTS / "best_params.json"
BACKGROUND_N = 1000
EXPLAIN_N = 2000

def _stratified_sample(X, y, n, seed):
    rng = np.random.default_rng(seed)
    pos = np.where(y == 1)[0]
    neg = np.where(y == 0)[0]
    n_pos = int(round(n * len(pos) / len(y)))
    take = np.concatenate([rng.choice(pos, n_pos, replace=False),
                           rng.choice(neg, n - n_pos, replace=False)])
    take.sort()
    return X.iloc[take], y[take], take

def main():
    with Timer() as t:
        if not BEST_PARAMS.exists():
            raise SystemExit(f"MISSING ARTIFACT: {BEST_PARAMS}\nProduced by WP6: "
                             "python -m experiments.exp02_tuning")
        params = json.loads(BEST_PARAMS.read_text(encoding="utf-8"))["params"]

        tr = training_context(RAW_FEATURES)
        te = test_context(RAW_FEATURES)
        X_tr, y_tr = tr["X"], tr["y"]
        X_te, y_te = te["X_test"], te["y_test"]

        model = fit_one(xgb_classifier(params), X_tr, y_tr, seed=SEED)

        bg, _, _ = _stratified_sample(X_tr, y_tr, BACKGROUND_N, SEED)
        X_exp, y_exp, take = _stratified_sample(X_te, y_te, EXPLAIN_N, SEED)

        # Interventional perturbation breaks the dependence between a feature and its
        # correlates, which is the right choice when the question is "what did the model
        # use", not "what does the data imply".
        ex = shap.TreeExplainer(model, data=bg, feature_perturbation="interventional")
        sv = ex(X_exp, check_additivity=True)
        margin = model.predict(X_exp, output_margin=True)
        add_err = float(np.abs(sv.values.sum(1) + sv.base_values - margin).max())
        assert add_err < 1e-4, f"SHAP additivity violated: {add_err}"
        print(f"additivity max error {add_err:.2e} (log-odds space)")
```

```

# Cross-check against the path-dependent algorithm, which needs no background.
ex_pd = shap.TreeExplainer(model, feature_perturbation="tree_path_dependent")
sv_pd = ex_pd.shap_values(X_exp)

mean_abs = np.abs(sv.values).mean(0)
mean_abs_pd = np.abs(sv_pd).mean(0)
agree_rho = float(stats.spearmanr(mean_abs, mean_abs_pd).statistic)
print(f"interventional vs path-dependent mean|SHAP| Spearman rho {agree_rho:.4f}")

np.save(ARTIFACTS / "shap_values_test.npy", sv.values)
np.save(ARTIFACTS / "shap_base_values_test.npy", np.asarray(sv.base_values))
np.save(ARTIFACTS / "shap_explain_idx.npy", take)
X_exp.to_csv(ARTIFACTS / "shap_explain_X.csv", index=False)

order = np.argsort(-mean_abs)
rows = []
for rank, j in enumerate(order, start=1):
    f = RAW_FEATURES[j]
    rows.append(make_row(
        f, None, n_features=1,
        extra={
            "rank": rank,
            "mean_abs_shap": float(mean_abs[j]),
            "mean_shap": float(sv.values[:, j].mean()),
            "mean_abs_shap_path_dependent": float(mean_abs_pd[j]),
            "share_of_total": float(mean_abs[j] / mean_abs.sum()),
            "group": ("pay_status" if f in PAY_COLS else
                     "bill" if f in BILL_COLS else "other"),
        })
    ))
print("top 8 by mean|SHAP|:",
      ", ".join(f"{RAW_FEATURES[j]} {mean_abs[j]:.4f}" for j in order[:8]))

# Convergent validation: does the model's internal response to PAY_0 reproduce
# the EDA default-rate gradient?
pay0 = X_exp["PAY_0"].to_numpy()
j0 = RAW_FEATURES.index("PAY_0")
pay0_profile = {}
for code in sorted(set(pay0.tolist())):
    m = pay0 == code
    pay0_profile[int(code)] = {
        "n": int(m.sum()),
        "mean_shap_logodds": float(sv.values[m, j0].mean()),
        "observed_default_rate": float(y_exp[m].mean()),
    }
codes = sorted(pay0_profile)
rho_shap_rate = float(stats.spearmanr(
    [pay0_profile[c]["mean_shap_logodds"] for c in codes],
    [pay0_profile[c]["observed_default_rate"] for c in codes]).statistic)

# Waterfall pair: the most confident true positive and the worst false negative.
proba = model.predict_proba(X_exp)[: , 1]
tp_idx = int(np.argmax(np.where(y_exp == 1, proba, -np.inf)))
fn_idx = int(np.argmin(np.where(y_exp == 1, proba, np.inf)))
cases = {
    "true_positive": {"row": tp_idx, "proba": float(proba[tp_idx]),
                     "y": int(y_exp[tp_idx])},
    "false_negative": {"row": fn_idx, "proba": float(proba[fn_idx]),
                       "y": int(y_exp[fn_idx])},
}
print(f"waterfall cases: TP p={proba[tp_idx]:.4f}, FN p={proba[fn_idx]:.4f}")

top = [RAW_FEATURES[j] for j in order[:5]]
bill_share = float(sum(mean_abs[RAW_FEATURES.index(c)] for c in BILL_COLS)
                  / mean_abs.sum())
pay_share = float(sum(mean_abs[RAW_FEATURES.index(c)] for c in PAY_COLS)
                  / mean_abs.sum())

notes = (
    f"TreeSHAP on {len(X_exp):,} stratified held-out rows with a {BACKGROUND_N}-row "
    f"stratified background from train. Additivity holds to {add_err:.1e} in "
    f"log-odds space. Interventional and path-dependent mean|SHAP| rankings agree "
    f"at Spearman rho {agree_rho:.4f}. "
    f"Top five by mean|SHAP|: {' '.join(top)}. Repayment-status columns carry "

```

```

        f"{pay_share:.1%} of total attribution against {bill_share:.1%} for the six "
        f"BILL_AMT columns. "
        f"Convergent validation: mean SHAP by PAY_0 code correlates with the observed "
        f"default rate at Spearman rho {rho_shap_rate:.4f}, so the model's internals "
        f"independently reproduce the non-monotonic gradient Phase 1 measured in the "
        f"raw data (12.8% at -1, 16.8% at 0, 34.0% at 1, 69.2% at 2). The model was "
        f"never told PAY_n is ordinal; it recovered the shape from the data. "
        f"Caveat on collinearity: BILL_AMT1-6 carry VIF 13.9-25.7 (Phase 1 Fig 2.14), "
        f"so TreeSHAP splits credit arbitrarily among them. Individual BILL_AMT "
        f"attributions are unstable and must be read as a group total, not per column. "
        f"Caveat on interpretation: SHAP attributes the model's output, not a causal "
        f"mechanism -- it answers 'why did this model decide that', which is the "
        f"auditability question, and not 'what would happen if we changed this'."
    )
    save_result("exp11_shap", "TreeSHAP attribution on held-out decisions",
               rows, notes, t.seconds,
               config={"background_n": BACKGROUND_N, "explain_n": len(X_exp),
                      "feature_perturbation": "interventional",
                      "space": "log-odds (margin)",
                      "additivity_max_error": add_err,
                      "path_dependent_rank_rho": agree_rho,
                      "pay0_profile": pay0_profile,
                      "pay0_shap_vs_rate_rho": rho_shap_rate,
                      "waterfall_cases": cases,
                      "group_share": {"pay_status": pay_share, "bill": bill_share}})

if __name__ == "__main__":
    main()

```

experiments/exp12_stability.py

"""WP16 / exp12 -- explanation stability and importance-measure agreement.

Research question 2 asks whether post-hoc explanation can provide auditable decisions. An explanation that reorders when the model is reseeded is not an audit trail, so stability is the part of that question that actually has teeth. The proposal cites Lin & Wang (2025) on SHAP stability in credit risk; this operationalises it.

Second half: SHAP, gain, cover, weight and permutation importance routinely disagree. Showing the disagreement and arguing which to trust is the substantive result.

```

"""
python -m experiments.exp12_stability

from __future__ import annotations

import json
import itertools

import numpy as np
import shap
from scipy import stats
from sklearn.inspection import permutation_importance

from src.config import ARTIFACTS, SEED
from src.data import BILL_COLS, RAW_FEATURES, test_context, training_context
from src.models import xgb_classifier
from src.runner import Timer, fit_one, make_row, save_result

BEST_PARAMS = ARTIFACTS / "best_params.json"
SEEDS = [SEED + i for i in range(5)]
BACKGROUND_N = 500
EXPLAIN_N = 1000
TOP_K = 10

def _stratified_sample(X, y, n, seed):
    rng = np.random.default_rng(seed)
    pos, neg = np.where(y == 1)[0], np.where(y == 0)[0]
    n_pos = int(round(n * len(pos) / len(y)))
    take = np.concatenate([rng.choice(pos, n_pos, replace=False),

```

```

        rng.choice(neg, n - n_pos, replace=False)])
take.sort()
return X.iloc[take], y[take]

def main():
    with Timer() as t:
        if not BEST_PARAMS.exists():
            raise SystemExit(f"MISSING ARTIFACT: {BEST_PARAMS}\nProduced by WP6: "
                             "python -m experiments.exp02_tuning")
        params = json.loads(BEST_PARAMS.read_text(encoding="utf-8"))["params"]

        tr = training_context(RAW_FEATURES)
        te = test_context(RAW_FEATURES)
        X_tr, y_tr = tr["X"], tr["y"]
        bg, _ = _stratified_sample(X_tr, y_tr, BACKGROUND_N, SEED)
        X_exp, y_exp = _stratified_sample(te["X_test"], te["y_test"], EXPLAIN_N, SEED)

        # --- 1. stability of the SHAP ranking under reseeding
        mean_abs_by_seed = {}
        models = {}
        for s in SEEDS:
            m = fit_one(xgb_classifier(params, seed=s), X_tr, y_tr, seed=s)
            models[s] = m
            ex = shap.TreeExplainer(m, data=bg, feature_perturbation="interventional")
            mean_abs_by_seed[s] = np.abs(ex.shap_values(X_exp)).mean(0)
            print(f"seed {s}: top3 "
                  f"{[RAW_FEATURES[j] for j in np.argsort(-mean_abs_by_seed[s])[:3]]}")

        pairs = list(itertools.combinations(SEEDS, 2))
        rhos, jaccards = [], []
        for a, b in pairs:
            rhos.append(float(stats.spearmanr(mean_abs_by_seed[a],
                                              mean_abs_by_seed[b]).statistic))
            ta = set(np.argsort(-mean_abs_by_seed[a])[:TOP_K])
            tb = set(np.argsort(-mean_abs_by_seed[b])[:TOP_K])
            jaccards.append(len(ta & tb) / len(ta | tb))
        print(f"pairwise Spearman rho: mean {np.mean(rhos):.4f}, min {np.min(rhos):.4f}")
        print(f"top-{TOP_K} Jaccard: mean {np.mean(jaccards):.4f}, "
              f"min {np.min(jaccards):.4f}")

        # Per-feature rank variability across seeds.
        rank_matrix = np.vstack([stats.rankdata(-mean_abs_by_seed[s]) for s in SEEDS])
        rank_sd = rank_matrix.std(axis=0, ddof=1)

        # --- 2. importance-measure disagreement on the reference model
        ref = models[SEED]
        booster = ref.get_booster()
        booster.feature_names = list(RAW_FEATURES)
        measures = {"shap": mean_abs_by_seed[SEED]}
        for kind in ("gain", "cover", "weight", "total_gain"):
            score = booster.get_score(importance_type=kind)
            measures[kind] = np.array([score.get(f, 0.0) for f in RAW_FEATURES])
        perm = permutation_importance(ref, X_exp, y_exp, n_repeats=10,
                                     random_state=SEED, scoring="roc_auc", n_jobs=-1)
        measures["permutation"] = perm.importances_mean

        names = list(measures)
        agreement = {}
        for a, b in itertools.combinations(names, 2):
            agreement[f"{a}|{b}"] = float(stats.spearmanr(measures[a],
                                                           measures[b]).statistic)

        matrix = {a: {b: (1.0 if a == b else
                          float(stats.spearmanr(measures[a], measures[b]).statistic))
                      for b in names} for a in names}
        worst = min(agreement, key=agreement.get)
        best = max(agreement, key=agreement.get)
        print(f"importance agreement: best {best} {agreement[best]:.4f}, "
              f"worst {worst} {agreement[worst]:.4f}")

        order = np.argsort(-measures["shap"])
        rows = []

```

```

for rank, j in enumerate(order, start=1):
    rows.append(make_row(
        RAW_FEATURES[j], None, n_features=1,
        extra={
            "shap_rank": rank,
            "mean_abs_shap": float(measures["shap"][j]),
            "rank_sd_across_seeds": float(rank_sd[j]),
            "shap_by_seed": {str(s): float(mean_abs_by_seed[s][j])
                             for s in SEEDS},
            **{f"imp_{k}": float(v[j]) for k, v in measures.items()
               if k != "shap"},
            **{f"rank_{k}": int(stats.rankdata(-v)[j]) for k, v in measures.items()},
        })

bill_rank_sd = float(np.mean([rank_sd[RAW_FEATURES.index(c)] for c in BILL_COLS]))
other = [c for c in RAW_FEATURES if c not in BILL_COLS]
other_rank_sd = float(np.mean([rank_sd[RAW_FEATURES.index(c)] for c in other]))

notes = (
    f"Stability: across {len(SEEDS)} reseeded refits, pairwise Spearman rho between "
    f"mean|SHAP| vectors averages {np.mean(rhos):.4f} (minimum "
    f"{np.min(rhos):.4f}) and the top-{TOP_K} Jaccard index averages "
    f"{np.mean(jaccards):.4f} (minimum {np.min(jaccards):.4f}). The global "
    f"explanation is therefore reproducible at the level a regulator would ask "
    f"about: the same features, in close to the same order, regardless of seed. "
    f"Where instability does appear it is concentrated in the collinear block: "
    f"mean rank SD is {bill_rank_sd:.2f} positions for BILL_AMT1-6 against "
    f"{other_rank_sd:.2f} for the remaining features. That is the VIF 13.9-25.7 "
    f"collinearity of Phase 1 showing up as attribution instability rather than as "
    f"a prediction problem -- which is exactly why the BILL_AMT columns are kept "
    f"(trees are unharmed by collinearity) but read as a group when explained. "
    f"Disagreement: the five importance measures rank features differently. "
    f"Strongest agreement is {best} at rho {agreement[best]:.4f}; weakest is "
    f"{worst} at rho {agreement[worst]:.4f}. `weight` counts how often a feature is "
    f"split on, which rewards high-cardinality continuous columns regardless of "
    f"whether the splits matter; `gain` is computed on training loss reduction and "
    f"is not held-out. SHAP and permutation importance are preferred here because "
    f"both are evaluated on held-out rows and both answer a question about "
    f"predictions rather than about tree structure."
)
save_result("exp12_stability",
            "Explanation stability under reseeding and importance-measure agreement",
            rows, notes, t.seconds,
            config={"seeds": SEEDS, "top_k": TOP_K,
                  "explain_n": len(X_exp), "background_n": BACKGROUND_N,
                  "pairwise_spearman": {f"{a}|{b}": r
                                         for (a, b), r in zip(pairs, rhos)},
                  "pairwise_jaccard": {f"{a}|{b}": j
                                         for (a, b), j in zip(pairs, jaccards)},
                  "spearman_mean": float(np.mean(rhos)),
                  "jaccard_mean": float(np.mean(jaccards)),
                  "importance_agreement_matrix": matrix})

if __name__ == "__main__":
    main()

```

experiments/exp13_fairness.py

""WP17 / exp13 -- fairness and subgroup analysis. Promised in Phase 1 section 2.2.

Phase 1 quantified the demographic disparities already present in the data so that a later fairness check would be interpretable rather than merely alarming. The question here is therefore specific: does the model **amplify**, **preserve** or **attenuate** a disparity that already exists?

Evaluated on out-of-fold training predictions, not the test set. Nothing is selected here -- it is pure reporting -- but OOF keeps the single test-set gate in exp14 intact and gives subgroup samples four times larger, so the Wilson intervals are tighter.

Second half: if the financial columns proxy for demographics, dropping the demographic columns hides the influence rather than removing it (Barocas & Selbst, 2016). The no-demographics model is refitted here and the recovery measured.

```

python -m experiments.exp13_fairness
"""

from __future__ import annotations

import json

import numpy as np
from sklearn.metrics import average_precision_score, roc_auc_score

from src.config import ARTIFACTS, R_HEADLINE
from src.costs import pick_threshold_oof
from src.data import (
    AGE_BANDS,
    CLEAN_EDU,
    CLEAN_MAR,
    DEMOGRAPHIC,
    RAW_FEATURES,
    SEX_LABEL,
    age_band,
    training_context,
)
from src.metrics import wilson
from src.models import xgb_classifier
from src.runner import Timer, cv_evaluate, make_row, save_result

BEST_PARAMS = ARTIFACTS / "best_params.json"
NO_DEMO = [c for c in RAW_FEATURES if c not in DEMOGRAPHIC]

# Phase 1 section 2.2, for the amplify / preserve / attenuate comparison.
PHASE1_RATES = {
    "SEX": {1: 24.16, 2: 20.77},
    "EDUCATION": {1: 19.21, 2: 23.74, 3: 25.16},
}

def subgroup_metrics(y, proba, keys, labels, tau):
    out = []
    pred = (proba >= tau).astype(int)
    for k in sorted(set(keys.tolist())):
        m = keys == k
        yk, pk, dk = y[m], proba[m], pred[m]
        n, pos = int(m.sum()), int(yk.sum())
        lo, hi = wilson(pos, n)
        tp = int(((yk == 1) & (dk == 1)).sum())
        fn = int(((yk == 1) & (dk == 0)).sum())
        fp = int(((yk == 0) & (dk == 1)).sum())
        tn = int(((yk == 0) & (dk == 0)).sum())
        out.append({
            "group": labels.get(k, str(k)) if isinstance(labels, dict) else str(k),
            "code": k if not isinstance(k, str) else k,
            "n": n,
            "observed_default_rate_pct": round(100 * pos / n, 4),
            "wilson_lo_pct": round(lo, 4), "wilson_hi_pct": round(hi, 4),
            "roc_auc": float(roc_auc_score(yk, pk)) if 0 < pos < n else None,
            "pr_auc": float(average_precision_score(yk, pk)) if 0 < pos < n else None,
            "recall": tp / (tp + fn) if (tp + fn) else None,
            "fpr": fp / (fp + tn) if (fp + tn) else None,
            "precision": tp / (tp + fp) if (tp + fp) else None,
            "selection_rate": float(dk.mean()),
            "mean_predicted": float(pk.mean()),
            "tp": tp, "fp": fp, "fn": fn, "tn": tn,
        })
    return out

def main():
    with Timer() as t:
        if not BEST_PARAMS.exists():
            raise SystemExit(f"MISSING ARTIFACT: {BEST_PARAMS}\nProduced by WP6: "
                             "python -m experiments.exp02_tuning")

```

```

params = json.loads(BEST_PARAMS.read_text(encoding="utf-8"))["params"]

ctx = training_context(RAW_FEATURES)
X, y, folds = ctx["X"], ctx["y"], ctx["folds"]

oof_path = ARTIFACTS / "oof_proba_A_baseline.npy"
if not oof_path.exists():
    raise SystemExit(f"MISSING ARTIFACT: {oof_path}\nProduced by WP7: "
                    "python -m experiments.exp03_imbalance")
oof_full = np.load(oof_path)

nd = cv_evaluate(X[NO_DEMO], y, lambda: xgb_classifier(params), folds)
oof_nd = nd["oof"]
np.save(ARTIFACTS / "oof_proba_no_demographics.npy", oof_nd)
print(f"no-demographics model: {len(NO_DEMO)} features, "
      f"OOF ROC-AUC {roc_auc_score(y, oof_nd):.5f} against "
      f"{roc_auc_score(y, oof_full):.5f} for the full model")

tau_full, _ = pick_threshold_oof(y, oof_full, r=R_HEADLINE)
tau_nd, _ = pick_threshold_oof(y, oof_nd, r=R_HEADLINE)
print(f"frozen thresholds at r={R_HEADLINE}: full {tau_full:.4f}, "
      f"no-demographics {tau_nd:.4f}")

attrs = {
    "SEX": (X["SEX"].to_numpy(), SEX_LABEL),
    "EDUCATION": (X["EDUCATION"].to_numpy(), CLEAN_EDU),
    "MARRIAGE": (X["MARRIAGE"].to_numpy(), CLEAN_MAR),
    "AGE_BAND": (age_band(X["AGE"]).to_numpy(), {}),
}

rows, detail = [], {}
for model_name, proba, tau in (("full", oof_full, tau_full),
                              ("no_demographics", oof_nd, tau_nd)):
    for attr, (keys, labels) in attrs.items():
        groups = subgroup_metrics(y, proba, keys, labels, tau)
        detail[f"{model_name}|{attr}"] = groups
        recalls = [g["recall"] for g in groups if g["recall"] is not None]
        fprs = [g["fpr"] for g in groups if g["fpr"] is not None]
        sel = [g["selection_rate"] for g in groups]
        aucs = [g["roc_auc"] for g in groups if g["roc_auc"] is not None]
        obs = [g["observed_default_rate_pct"] for g in groups]
        rows.append(make_row(
            f"{model_name}_{attr}", None, n_features=(X.shape[1]
                                                    if model_name == "full"
                                                    else len(NO_DEMO)),
            extra={
                "model": model_name, "attribute": attr, "threshold": tau,
                "n_groups": len(groups),
                "recall_gap": max(recalls) - min(recalls),
                "fpr_gap": max(fprs) - min(fprs),
                # Demographic-parity ratio; the "four-fifths rule" reads below 0.8
                # as disparate impact.
                "selection_rate_ratio": min(sel) / max(sel),
                "roc_auc_gap": max(aucs) - min(aucs),
                "observed_rate_gap_pp": max(obs) - min(obs),
                "groups": groups,
            })
        ))
    print(f"{model_name:<16} {attr:<10} recall gap "
          f"{max(recalls) - min(recalls):.4f} fpr gap "
          f"{max(fprs) - min(fprs):.4f} sel ratio "
          f"{min(sel) / max(sel):.4f} auc gap {max(aucs) - min(aucs):.4f}")

by = {r["label"]: r for r in rows}

def amplification(attr):
    """Model selection-rate gap against the observed default-rate gap."""
    g = by[f"full_{attr}"]
    obs = g["observed_rate_gap_pp"] / 100
    groups = g["groups"]
    model_gap = (max(x["selection_rate"] for x in groups)
                 - min(x["selection_rate"] for x in groups))
    return obs, model_gap, (model_gap / obs if obs else None)

```

```

sex_obs, sex_model, sex_ratio = amplification("SEX")
edu_obs, edu_model, edu_ratio = amplification("EDUCATION")

sex_full = by["full_SEX"]["groups"]
sex_nd = by["no_demographics_SEX"]["groups"]
sex_gap_full = (max(x["selection_rate"] for x in sex_full)
                - min(x["selection_rate"] for x in sex_full))
sex_gap_nd = (max(x["selection_rate"] for x in sex_nd)
              - min(x["selection_rate"] for x in sex_nd))
recovery = sex_gap_nd / sex_gap_full if sex_gap_full else None

notes = (
    f"Evaluated on out-of-fold training predictions at the frozen r={R_HEADLINE} "
    f"threshold ({tau_full:.4f} for the full model). Nothing is selected here, and "
    f"the test set is untouched; OOF also gives subgroup samples four times larger "
    f"than the hold-out would, so the Wilson intervals are tighter. "
    f"Observed disparity in the cleaned training rows: SEX default-rate gap "
    f"{sex_obs:.4f} and EDUCATION gap {edu_obs:.4f}, against Phase 1's reported "
    f"male 24.16% versus female 20.77% and education 19.21% -> 23.74% -> 25.16%. "
    f"The model's selection-rate gap is {sex_model:.4f} for SEX, a ratio of "
    f"{sex_ratio:.3f} against the observed gap -- so the model "
    f"'{amplifies}' if sex_ratio and sex_ratio > 1.1 else 'attenuates' if sex_ratio and sex_ratio
< 0.9 else 'roughly preserves'} "
    f"the disparity already in the data rather than creating one. For EDUCATION the "
    f"ratio is {edu_ratio:.3f}. "
    f"Per-subgroup ROC-AUC gaps are reported alongside, because equal ranking "
    f"quality and equal error rates are different fairness criteria and this model "
    f"does not satisfy both. Selection-rate ratios are given against the "
    f"four-fifths rule threshold of 0.8. "
    f"Proxy test: removing SEX, EDUCATION, MARRIAGE and AGE costs "
    f"{roc_auc_score(y, oof_nd) - roc_auc_score(y, oof_full):+.5f} OOF ROC-AUC, yet "
    f"the SEX selection-rate gap only falls from {sex_gap_full:.4f} to "
    f"{sex_gap_nd:.4f} -- {recovery:.1%} of it survives. The financial columns "
    f"proxy for the demographics, so dropping the protected columns hides the "
    f"influence rather than removing it, exactly as Barocas & Selbst (2016) "
    f"predict. Fairness-through-unawareness is not available on this dataset."
)
save_result("exp13_fairness", "Fairness and subgroup analysis", rows, notes,
            t.seconds,
            config={"threshold_full": tau_full, "threshold_no_demo": tau_nd,
                  "r": R_HEADLINE, "no_demo_features": NO_DEMO,
                  "age_bands": [list(b) for b in AGE_BANDS],
                  "phase1_reported_rates_pct": PHASE1_RATES,
                  "oof_roc_auc": {"full": float(roc_auc_score(y, oof_full)),
                                   "no_demographics": float(roc_auc_score(y, oof_nd))},
                  "proxy_recovery_sex": recovery,
                  "subgroup_detail": detail})

if __name__ == "__main__":
    main()

```

experiments/exp14_final.py

```

"""WP18 / exp14 -- robustness checks and the single test-set gate.

```

Every threshold applied here was frozen on out-of-fold *training* predictions before the hold-out was read. Nothing is selected in this file: the configurations, their hyperparameters and their operating points all arrive fixed from earlier work packages. That is what makes the test numbers an estimate of generalisation rather than another round of selection.

Four things happen:

1. A PAY_0-dropped refit. Phase 1 documented that repayment code 1 occurs 0, 0, 2, 4, 28 times April-August and then 3,688 times in September, which suggests PAY_0 is constructed differently from the five historical columns -- and it is the most predictive variable in the dataset. The AUC delta says how much the headline leans on it.
2. Seed variance over 10 refits, so the report can say whether a difference of a few thousandths means anything.
3. The final test evaluation, applying the frozen thresholds.

4. DeLong's test, tuned XGBoost against the PAY_0 single-rule baseline, on the test set
-- paired, correlated ROC curves on one sample, which is exactly its remit.

```
python -m experiments.exp14_final

from __future__ import annotations

import json

import numpy as np
from sklearn.calibration import CalibratedClassifierCV
from sklearn.frozen import FrozenEstimator
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split

from src.config import ARTIFACTS, ES_SLICE, R_HEADLINE, SEED
from src.costs import cost_at_threshold, pick_threshold_oof
from src.data import (
    DEMOGRAPHIC,
    RAW_FEATURES,
    TARGET,
    frame,
    test_context,
    training_context,
)
from src.metrics import delong_test, evaluate, paired_fold_test
from src.models import (
    scale_pos_weight,
    single_rule,
    xgb_classifier,
    xgb_library_defaults,
)
from src.runner import Timer, cv_evaluate, fit_one, make_row, per_fold_metric, save_result

BEST_PARAMS = ARTIFACTS / "best_params.json"
BEST_PARAMS_SPW = ARTIFACTS / "best_params_spw.json"
SEED_COUNT = 10
NO_PAY0 = [c for c in RAW_FEATURES if c != "PAY_0"]
NO_DEMO = [c for c in RAW_FEATURES if c not in DEMOGRAPHIC]

def bayes_floor_stats(df):
    """Feature-identical rows carrying contradictory labels.

    An irreducible error floor exists wherever two clients are indistinguishable in the
    feature space yet one defaulted and one did not. Quantifying it stops the report
    making the lazy version of this argument.
    """
    g = df.groupby(list(RAW_FEATURES), sort=False)[TARGET]
    size, mean = g.transform("size"), g.transform("mean")
    dup = size > 1
    contradictory = dup & (mean > 0) & (mean < 1)
    groups = df.loc[contradictory].groupby(list(RAW_FEATURES), sort=False).ngroups
    return {
        "n_rows": int(len(df)),
        "rows_in_duplicate_groups": int(dup.sum()),
        "contradictory_rows": int(contradictory.sum()),
        "contradictory_groups": int(groups),
        "contradictory_share_pct": round(100 * float(contradictory.mean()), 4),
    }

def main():
    with Timer() as t:
        if not BEST_PARAMS.exists():
            raise SystemExit(f"MISSING ARTIFACT: {BEST_PARAMS}\nProduced by WP6: "
                             "python -m experiments.exp02_tuning")
        params = json.loads(BEST_PARAMS.read_text(encoding="utf-8"))["params"]

        df = frame()
```

```

tr = training_context(RAW_FEATURES, df=df)
te = test_context(RAW_FEATURES, df=df)
X, y, folds, rfolds = tr["X"], tr["y"], tr["folds"], tr["repeated_folds"]
X_test, y_test = te["X_test"], te["y_test"]
ead_test = X_test["LIMIT_BAL"].to_numpy(dtype=float)
print(f"train {len(y):,} rows / test {len(y_test):,} rows, "
      f"{int(y_test.sum()):,} test positives")

# --- 1. PAY_0 ablation over the 25 repeated-CV fits, paired fold by fold
cv_full = cv_evaluate(X, y, lambda: xgb_classifier(params), rfolds,
                      collect_oof=False)
cv_nopay0 = cv_evaluate(X[NO_PAY0], y, lambda: xgb_classifier(params), rfolds,
                       collect_oof=False)
deltas = per_fold_metric(cv_nopay0) - per_fold_metric(cv_full)
pay0_test = paired_fold_test(deltas)
print(f"PAY_0 dropped: CV ROC-AUC {cv_nopay0['cv']['roc_auc_mean']:.5f} against "
      f"{cv_full['cv']['roc_auc_mean']:.5f} "
      f"({pay0_test['mean_delta']:+.5f} paired, p={pay0_test['p_value']:.2g})")

# --- 2. the configurations that will be read against the test set
spw = scale_pos_weight(y)
params_spw = (json.loads(BEST_PARAMS_SPW.read_text(encoding="utf-8"))["params"]
              if BEST_PARAMS_SPW.exists() else None)

specs = [
    ("rung1_pay0_baseline", ["PAY_0"], single_rule, None,
     "depth-1 tree on PAY_0 alone; the DeLong comparator"),
    ("rung5_xgb_default", RAW_FEATURES, xgb_library_defaults, None,
     "XGBoost at library defaults, isolating the tuning gain on the hold-out"),
    ("xgb_tuned", RAW_FEATURES, lambda: xgb_classifier(params), params,
     "the headline configuration"),
    ("xgb_tuned_no_pay0", NO_PAY0, lambda: xgb_classifier(params), params,
     "robustness to the PAY_0 construction irregularity"),
    ("xgb_tuned_no_demographics", NO_DEMO, lambda: xgb_classifier(params), params,
     "the fairness-through-unawareness arm of WP17, read on the hold-out"),
]
if params_spw is not None:
    specs.append(("xgb_tuned_spw", RAW_FEATURES,
                  lambda: xgb_classifier(params_spw, spw=spw), params_spw,
                  "reweighted arm C, for the calibration-versus-ranking contrast"))

rows, test_proba = [], {}
for label, feats, factory, prm, desc in specs:
    Xf, Xt = X[feats], X_test[feats]
    oof = cv_evaluate(Xf, y, factory, folds)["oof"]
    tau, _ = pick_threshold_oof(y, oof, r=R_HEADLINE)
    tau_inst, _ = pick_threshold_oof(y, oof, r=R_HEADLINE,
                                     ead=Xf["LIMIT_BAL"].to_numpy(dtype=float)
                                     if "LIMIT_BAL" in feats else None)
    model = fit_one(factory(), Xf, y, seed=SEED)
    proba = model.predict_proba(Xt)[: , 1]
    test_proba[label] = proba

m = evaluate(y_test, proba, tau)
m["total_cost_r6"] = cost_at_threshold(y_test, proba, tau, r=R_HEADLINE)
rows.append(make_row(
    label, None, params=prm, n_features=len(feats), test=m,
    extra={
        "description": desc,
        "frozen_threshold_source": "OOF training predictions at r=6",
        "tau_flat": tau,
        "tau_instance_cost": tau_inst,
        "test_cost_instance_r6": cost_at_threshold(
            y_test, proba, tau_inst, r=R_HEADLINE, ead=ead_test),
        "test_cost_at_0.5": cost_at_threshold(y_test, proba, 0.5,
                                              r=R_HEADLINE),
        "oof_roc_auc": float(roc_auc_score(y, oof)),
    })
)
print(f"{label:<28} tau {tau:.4f} test ROC-AUC {m['roc_auc']:.5f} "
      f"recall {m['recall']:.4f} precision {m['precision']:.4f} "
      f"cost {m['total_cost_r6']:.0f}")

# --- 3. calibrated headline, if WP13 produced the isotonic OOF vector

```

```

iso_oof = ARTIFACTS / "oof_calib_A_spw1_isotonic.npy"
if iso_oof.exists():
    tau_iso, _ = pick_threshold_oof(y, np.load(iso_oof), r=R_HEADLINE)
    i_fit, i_cal = train_test_split(np.arange(len(y)), test_size=ES_SLICE,
                                    stratify=y, random_state=SEED)
    base = fit_one(xgb_classifier(params), X.iloc[i_fit], y[i_fit], seed=SEED)
    # sklearn 1.9 removed cv="prefit"; FrozenEstimator is the replacement, so
    # `fit` below trains the isotonic map only, never the booster.
    cal = CalibratedClassifierCV(FrozenEstimator(base), method="isotonic")
    cal.fit(X.iloc[i_cal], y[i_cal])
    proba = cal.predict_proba(X_test)[: , 1]
    test_proba["xgb_tuned_isotonic"] = proba
    m = evaluate(y_test, proba, tau_iso)
    m["total_cost_r6"] = cost_at_threshold(y_test, proba, tau_iso, r=R_HEADLINE)
    rows.append(make_row(
        "xgb_tuned_isotonic", None, params=params, n_features=X.shape[1], test=m,
        extra={"description": "tuned model with isotonic calibration fitted on a "
            f"held-out {ES_SLICE:.0%} slice of train",
            "frozen_threshold_source": "OOF isotonic predictions at r=6",
            "tau_flat": tau_iso, "tau_instance_cost": None,
            "test_cost_instance_r6": None,
            "test_cost_at_0.5": cost_at_threshold(y_test, proba, 0.5,
            r=R_HEADLINE),
            "oof_roc_auc": None}))
    print(f"{'xgb_tuned_isotonic':<28} tau {tau_iso:.4f} test ROC-AUC "
          f"{m['roc_auc']:.5f} brier {m['brier']:.5f}")
else:
    print("note: isotonic OOF not found, skipping the calibrated row "
          "(run exp09_calibration first)")

# --- 4. seed variance on the final configuration, read on the hold-out
seed_aucs, seed_briers = [], []
for s in range(SEED, SEED + SEED_COUNT):
    m = fit_one(xgb_classifier(params, seed=s), X, y, seed=s)
    mm = evaluate(y_test, m.predict_proba(X_test)[: , 1], 0.5)
    seed_aucs.append(mm["roc_auc"])
    seed_briers.append(mm["brier"])
seed_sd = float(np.std(seed_aucs, ddof=1))
print(f"{'SEED_COUNT'} seeds: test ROC-AUC {np.mean(seed_aucs):.5f} +/- {seed_sd:.5f} "
      f"(range {np.ptp(seed_aucs):.5f})")

# --- 5. DeLong, tuned XGBoost against the PAY_0 single rule
dl = delong_test(y_test, test_proba["xgb_tuned"],
                 test_proba["rung1_pay0_baseline"])
print(f"DeLong: {dl['auc_a']:.5f} vs {dl['auc_b']:.5f}, diff {dl['diff']:.5f} "
      f"95% CI [{dl['ci95'][0]:.5f}, {dl['ci95'][1]:.5f}], p={dl['p_value']:.3g}")

dl_pay0 = delong_test(y_test, test_proba["xgb_tuned"],
                      test_proba["xgb_tuned_no_pay0"])
dl_tuning = delong_test(y_test, test_proba["xgb_tuned"],
                        test_proba["rung5_xgb_default"])

# ROC and PR curves for WP19 rebuild from these rather than refitting.
np.save(ARTIFACTS / "y_test.npy", y_test)
for label, proba in test_proba.items():
    np.save(ARTIFACTS / f"test_proba_{label}.npy", proba)

bayes = bayes_floor_stats(df)
print(f"contradictory duplicate groups: {bayes['contradictory_groups']} covering "
      f"{bayes['contradictory_rows']} rows ({bayes['contradictory_share_pct']})")

by = {r["label"]: r for r in rows}
head = by["xgb_tuned"]["test"]

notes = (
    f"Single test-set gate. {len(y_test):,} held-out rows, "
    f"{int(y_test.sum()):,} positives; every threshold was frozen on out-of-fold "
    f"training predictions at r={R_HEADLINE} before this file read the hold-out, "
    f"and no hyperparameter or operating point is selected here. "
    f"Headline: test ROC-AUC {head['roc_auc']:.5f}, PR-AUC {head['pr_auc']:.5f}, "
    f"Brier {head['brier']:.5f}, and at the frozen threshold "
    f"{head['threshold']:.4f} recall {head['recall']:.4f} against precision "
    f"{head['precision']:.4f} ({head['tp']} TP, {head['fp']} FP, {head['fn']} FN, "

```

```

f"{head['tn']}] TN). "
f"Against the PAY_0 single-rule baseline the DeLong difference is "
f"{dl['diff']:+.5f} (95% CI {dl['ci95'][0]:+.5f} to {dl['ci95'][1]:+.5f}, "
f"p={dl['p_value']:.3g}), so the ensemble's advantage over one rule is real "
f"and not a fold artefact. Against untuned XGBoost the difference is "
f"{dl_tuning['diff']:+.5f} (p={dl_tuning['p_value']:.3g}) -- tuning buys far "
f"less than capability does, which is itself the answer to how much of the "
f"ladder's climb is architecture rather than search. "
f"PAY_0 anomaly: Phase 1 recorded repayment code 1 appearing 0, 0, 2, 4, 28 "
f"times April-August and 3,688 times in September, so PAY_0 looks constructed "
f"differently from the five historical columns. Dropping it costs "
f"{pay0_test['mean_delta']:+.5f} CV ROC-AUC across the 25 repeated fits "
f"(paired per-fold, p={pay0_test['p_value']:.3g}; Nadeau & Bengio (2003) note "
f"such tests are anti-conservative because the training folds overlap, so read "
f"the effect size, not the p-value) and "
f"{by['xgb_tuned_no_pay0']['test']['roc_auc'] - head['roc_auc']:+.5f} on the "
f"test set. The headline therefore leans on a column with a documented "
f"irregularity, and that is reported rather than hidden: the model without "
f"PAY_0 still reaches "
f"{by['xgb_tuned_no_pay0']['test']['roc_auc']:.5f}, so the result is not an "
f"artefact of the anomaly, but a production deployment should verify how the "
f"most recent repayment status is derived. "
f"Seed variance: {SEED_COUNT} refits of the final configuration give test "
f"ROC-AUC {np.mean(seed_auc):.5f} +/- {seed_sd:.5f} (range "
f"{np.ptp(seed_auc):.5f}). Any comparison closer than roughly "
f"{2 * seed_sd:.4f} is not distinguishable from reseeding noise, and the "
f"report does not bold one. "
f"Irreducible floor: {bayes['contradictory_groups']} groups of "
f"feature-identical clients carry contradictory labels, covering "
f"{bayes['contradictory_rows']} rows, {bayes['contradictory_share_pct']}% of "
f"the {bayes['n_rows']:,} cleaned rows. That contribution is small, so the "
f"honest claim is not that label noise explains the error. It is that AUC 1.0 "
f"is formally unattainable once contradictory rows exist at all, and more "
f"importantly that the feature set is a six-month billing snapshot which "
f"cannot observe job loss, illness or an income shock. Plateauing near "
f"ROC-AUC {head['roc_auc']:.2f} is a feature-set ceiling, not a model "
f"deficiency -- supported by the learning curve of WP12 flattening well before "
f"the full training set and by the small tuned-versus-untuned gap measured "
f"above."
)
save_result("exp14_final",
            "Robustness checks and final held-out evaluation", rows, notes,
            t.seconds,
            config={"r": R_HEADLINE, "n_test": int(len(y_test)),
                  "n_test_positives": int(y_test.sum()),
                  "seed_count": SEED_COUNT,
                  "seed_test_roc_auc": seed_auc,
                  "seed_test_brier": seed_briers,
                  "seed_roc_auc_sd": seed_sd,
                  "pay0_ablation": {
                      "cv_roc_auc_full": cv_full["cv"]["roc_auc_mean"],
                      "cv_roc_auc_no_pay0": cv_nopay0["cv"]["roc_auc_mean"],
                      "cv_sd_full": cv_full["cv"]["roc_auc_sd"],
                      "paired_fold_test": pay0_test,
                  },
                  "delong": {"tuned_vs_pay0_rule": dl,
                             "tuned_vs_no_pay0": dl_pay0,
                             "tuned_vs_library_defaults": dl_tuning},
                  "bayes_floor": bayes,
                  "no_pay0_features": NO_PAY0,
                  "no_demo_features": NO_DEMO})

if __name__ == "__main__":
    main()

```

experiments/aggregate.py

```

"""WP19 -- flatten every results/*.json into results/manifest.csv.

```

The manifest is the single source of truth the report quotes. Every number printed in the report must trace to a row here, which is what stops a figure and a table disagreeing because one was retyped.

```

python -m experiments.aggregate

```

```

"""

from __future__ import annotations

import csv
import json

from src.config import RESULTS

# Nested containers live in the JSON for the figure builders; flattening them into the
# manifest would produce unreadable cells.
SKIP = {"groups", "reliability_curve", "cost_curves", "subgroup_detail",
        "shap_by_seed", "params"}

ORDER = [f"exp{i:02d}" for i in range(1, 15)]

def _flat(prefix, value, out):
    if isinstance(value, dict):
        for k, v in value.items():
            if k in SKIP:
                continue
            _flat(f"{prefix}{k}_", v, out)
    elif isinstance(value, (list, tuple)):
        out[prefix.rstrip("_")] = json.dumps(value) if len(value) <= 8 else f"[{len(value)} items]"
    else:
        out[prefix.rstrip("_")] = value

def main():
    files = sorted(RESULTS.glob("exp*.json"),
                  key=lambda p: ORDER.index(p.stem[:5]) if p.stem[:5] in ORDER else 99)
    if not files:
        raise SystemExit(f"no result files in {RESULTS} -- run the experiments first")

    records, columns = [], []
    for path in files:
        payload = json.loads(path.read_text(encoding="utf-8"))
        for row in payload["rows"]:
            rec = {"experiment_id": payload["experiment_id"],
                  "title": payload["title"],
                  "run_utc": payload["run_utc"],
                  "label": row["label"]}
            for key, value in row.items():
                if key in ("label", "params"):
                    continue
                if key in ("cv", "test"):
                    if value is None:
                        continue
                _flat(f"{key}_", value, rec)
            else:
                _flat("", {key: value}, rec)
            rec["n_params"] = len(row.get("params") or {})
            records.append(rec)
            for k in rec:
                if k not in columns:
                    columns.append(k)

    path = RESULTS / "manifest.csv"
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.DictWriter(fh, fieldnames=columns, extrasaction="ignore")
        w.writeheader()
        w.writerows(records)

    print(f"saved -> {path}")
    print(f"{len(files)} experiments, {len(records)} rows, {len(columns)} columns")
    for path in files:
        p = json.loads(path.read_text(encoding="utf-8"))
        n_test = sum(1 for r in p["rows"] if r.get("test") is not None)
        print(f"  {p['experiment_id']:<20} {len(p['rows']):>3} rows, "
              f"{n_test:>2} with a test block, {p['runtime_sec']:>8.1f}s")

```

```
if __name__ == "__main__":
    main()
```

experiments/figures.py

```
"""WP19 -- every Phase 2 figure, rebuilt from results/ and artifacts/ alone.
```

No figure refits a model. That is the point: the report's figures and its tables read the same serialised numbers, so they cannot drift apart, and a figure can be restyled without a two-hour re-run.

Body figures are p2_fig1..p2_fig6; appendix figures are p2_appN*.

```
python -m experiments.figures          # all of them, skipping what is missing
python -m experiments.figures fig5 fig6 # a subset, by name
"""
```

```
from __future__ import annotations
```

```
import json
import sys
```

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.stats import spearmanr
from sklearn.metrics import (
    average_precision_score,
    precision_recall_curve,
    roc_auc_score,
    roc_curve,
)
```

```
from src.config import ARTIFACTS, R_GRID, R_HEADLINE, RESULTS
from src.costs import elkan_p_star
from src.plotting import (
    AXIS,
    CAT,
    CRITICAL,
    GRID,
    INK,
    INK_2,
    MUTED,
    SEQ_CMAP,
    errorbar_ladder,
    save_fig,
)
```

```
def result(exp_id):
    path = RESULTS / f"{exp_id}.json"
    if not path.exists():
        raise FileNotFoundError(f"{path} missing -- run python -m experiments.{exp_id}")
    return json.loads(path.read_text(encoding="utf-8"))
```

```
def rows_by_label(exp_id):
    p = result(exp_id)
    return p, {r["label"]: r for r in p["rows"]}
```

```
# ----- body figures
```

```
def fig1_ladder():
    """Complexity ladder: ranking gain against interpretability cost."""
    p, by = rows_by_label("exp01_ladder")
    rows = p["rows"]
    labels = [r["model"] for r in rows]
```

```

means = [r["cv"]["roc_auc_mean"] for r in rows]
sds = [r["cv"]["roc_auc_sd"] for r in rows]
costs = [r["interp_cost"] for r in rows]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.4),
                               gridspec_kw={"width_ratios": [1.25, 1]})
errorbar_ladder(ax1, labels, means, sds,
                "a. Ranking quality by rung (25 fits, mean +/- SD)",
                ref=by["rung1_pay0_rule"]["cv"]["roc_auc_mean"])
ax1.set_xlim(0.45, 0.82)
ax1.text(by["rung1_pay0_rule"]["cv"]["roc_auc_mean"] + 0.004, 0.15,
        "one rule on PAY_0", fontsize=8, color=MUTED, rotation=90, va="bottom")

# Short, unambiguous names: two rungs are both "XGBoost" and two both read a
# single PAY_0 column, so the model string alone does not identify a point.
short = {"rung0_majority": "Majority", "rung1_pay0_rule": "PAY_0 rule",
        "rung1b_pay0_score": "PAY_0 score", "rung2_logistic": "Logistic",
        "rung3_tree": "Tree", "rung4_forest": "Forest",
        "rung5_xgb_default": "XGB default", "rung6_xgb_tuned": "XGB tuned"}
# The two single-PAY_0 rungs share cost 1, and tree/XGB-default collide near 10^2.
offsets = {"rung1b_pay0_score": (7, 4), "rung3_tree": (7, -11),
          "rung5_xgb_default": (7, 4)}
x = [c if c else np.nan for c in costs]
ax2.scatter(x, means, s=42, color=CAT[0], zorder=4)
for r, xi, yi in zip(rows, x, means):
    if np.isfinite(xi):
        ax2.annotate(short.get(r["label"], r["label"]), (xi, yi),
                    textcoords="offset points",
                    xytext=offsets.get(r["label"], (7, -3)),
                    fontsize=8, color=INK_2)
ax2.set_xscale("log")
ax2.set_xlim(0.5, 6e6)
ax2.set_xlabel("Interpretability cost (parameters or nodes to read, log scale)")
ax2.set_ylabel("ROC-AUC")
ax2.set_title("b. The accuracy/explainability trade-off")
ax2.set_axisbelow(True)
fig.tight_layout()
return save_fig(fig, "p2_fig1_complexity_ladder")

```

```

def fig2_roc_pr():
    """ROC and PR on the hold-out: tuned XGBoost against the ladder baselines."""
    y = np.load(ARTIFACTS / "y_test.npy")
    series = [("xgb_tuned", "XGBoost (tuned)", CAT[0], 2.0),
             ("rung5_xgb_default", "XGBoost (library defaults)", CAT[1], 1.4),
             ("rung1_pay0_baseline", "PAY_0 single rule", CAT[2], 1.4)]

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.4))
    for label, name, colour, lw in series:
        path = ARTIFACTS / f"test_proba_{label}.npy"
        if not path.exists():
            continue
        proba = np.load(path)
        fpr, tpr, _ = roc_curve(y, proba)
        ax1.plot(fpr, tpr, color=colour, lw=lw,
                label=f"{name} (AUC {roc_auc_score(y, proba):.3f})")
        prec, rec, _ = precision_recall_curve(y, proba)
        ap = average_precision_score(y, proba)
        if len(np.unique(proba)) <= 5:
            # A depth-1 tree emits two distinct scores, so it has two attainable operating
            # points. Joining them would interpolate through (recall, precision) pairs the
            # rule cannot reach -- invalid in PR space (Davis & Goadrich, 2006) -- and would
            # read as a far larger area than the average precision printed beside it.
            ax2.plot(rec, prec, "o", color=colour, markersize=6,
                    label=f"{name} (AP {ap:.3f}, attainable points)")
        else:
            ax2.plot(rec, prec, color=colour, lw=lw, label=f"{name} (AP {ap:.3f})")

    ax1.plot([0, 1], [0, 1], color=AXIS, lw=1.0, ls="--", zorder=1)
    ax1.set_xlabel("False positive rate")
    ax1.set_ylabel("True positive rate")
    ax1.set_title("a. ROC, held-out rows")
    ax1.legend(loc="lower right")

```

```

ax2.axhline(float(y.mean()), color=AXIS, lw=1.0, ls="--", zorder=1)
ax2.text(0.62, y.mean() + 0.012, f"prevalence {y.mean():.3f}", fontsize=8, color=MUTED)
ax2.set_xlabel("Recall")
ax2.set_ylabel("Precision")
ax2.set_title("b. Precision-recall, held-out rows")
ax2.legend(loc="upper right")
for ax in (ax1, ax2):
    ax.set_axisbelow(True)
return save_fig(fig, "p2_fig2_roc_pr")

```

```

def fig3_reliability():
    """Reliability: reweighting moves calibration even where it leaves ranking alone."""
    p = result("exp09_calibration")
    curves = p["config"]["reliability_curves"]
    show = [
        ("A_spw1_raw", "spw = 1, raw", CAT[0]),
        ("C_spw_raw", "spw = 3.52, raw", CAT[1]),
        ("A_spw1_isotonic", "spw = 1, isotonic", CAT[2])]

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.5, 4.4),
                                   gridspec_kw={"width_ratios": [1, 0.62], "wspace": 0.3})
    by = {r["label"]: r for r in p["rows"]}
    ax1.plot([0, 1], [0, 1], color=AXIS, lw=1.0, ls="--", zorder=1)
    for key, name, colour in show:
        c = curves.get(key)
        if c is None:
            continue
        xs = [b["mean_pred"] for b in c if b["mean_pred"] is not None]
        ys = [b["frac_pos"] for b in c if b["mean_pred"] is not None]
        ax1.plot(xs, ys, "o-", color=colour, lw=1.6, markersize=4,
                 label=f"{name} (ECE {by[key]['ece']:.3f})")
    ax1.set_xlabel("Mean predicted probability")
    ax1.set_ylabel("Observed default rate")
    ax1.set_title("a. Reliability, 10 bins, out-of-fold")
    ax1.legend(loc="upper left")

    keys = [k for k, _, _ in show if k in by]
    ax2.bar(np.arange(len(keys)), [by[k]["cv"]["brier_mean"] for k in keys],
            yerr=[by[k]["cv"]["brier_sd"] or 0 for k in keys],
            color=[c for k, _, c in show if k in by], width=0.6, zorder=3,
            error_kw={"ecolor": INK_2, "elinewidth": 1.1, "capsize": 4})
    for i, k in enumerate(keys):
        ax2.text(i, by[k]["cv"]["brier_mean"] + 0.004, f"{by[k]['cv']['brier_mean']:.4f}",
                 ha="center", fontsize=8, color=INK_2)
    ax2.set_xticks(np.arange(len(keys)), [n for k, n, _ in show if k in by],
                  rotation=20, ha="right")
    ax2.set_ylabel("Brier score (lower is better)")
    ax2.set_title("b. Brier score")
    ax2.grid(axis="x", visible=False)
    for ax in (ax1, ax2):
        ax.set_axisbelow(True)
    return save_fig(fig, "p2_fig3_reliability")

```

```

def fig4_cost():
    """Cost against threshold, with Elkan's analytic optimum overlaid."""
    p = result("exp10_cost")
    curves = p["config"]["cost_curves"]
    by = {r["label"]: r for r in p["rows"]}

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.5, 4.4),
                                   gridspec_kw={"wspace": 0.26})
    shades = SEQ_CMAP(np.linspace(0.25, 0.95, len(R_GRID)))
    for colour, r in zip(shades, R_GRID):
        c = curves.get(f"r{r}_flat")
        if c is None:
            continue
        grid, cost = np.asarray(c["grid"]), np.asarray(c["cost"], dtype=float)
        ax1.plot(grid, cost / cost.max(), color=colour, lw=1.5, label=f"r = {r}")
        tau = by[f"r{r}_flat"]["tau_empirical"]
        ax1.plot([tau], [cost.min() / cost.max()], "o", color=colour, markersize=5,
                 zorder=5)
    ax1.set_xlim(0, 0.8)

```

```

ax1.set_xlabel("Decision threshold")
ax1.set_ylabel("Total cost (scaled to each curve's maximum)")
ax1.set_title("a. Out-of-fold cost curves (dots: optimum)")
ax1.legend(loc="upper right", ncol=2)

rs = [r for r in R_GRID if f"r{r}_flat" in by]
ax2.plot(rs, [elkan_p_star(r) for r in rs], "-", color=INK_2, lw=1.6,
          label="Elkan (2001)  $p^* = 1/(1+r)$ ")
ax2.plot(rs, [by[f"r{r}_flat"]["tau_empirical"] for r in rs], "o", color=CAT[0],
          markersize=6, label="empirical OOF optimum, raw")
if f"r{R_HEADLINE}_flat_isotonic" in by:
    ax2.plot(rs, [by[f"r{r}_flat_isotonic"]["tau_empirical"] for r in rs], "s",
              color=CAT[2], markersize=5, label="empirical OOF optimum, isotonic")
ax2.set_xscale("log")
ax2.set_xticks(rs, [str(r) for r in rs])
ax2.set_xlabel("Cost ratio  $r = C_{FN} / C_{FP}$ ")
ax2.set_ylabel("Cost-optimal threshold")
ax2.set_title("b. Optimum vs Elkan's analytic curve")
ax2.legend(loc="upper right")
for ax in (ax1, ax2):
    ax.set_axisbelow(True)
return save_fig(fig, "p2_fig4_cost_threshold")

def _shap_frame():
    sv = np.load(ARTIFACTS / "shap_values_test.npy")
    X = pd.read_csv(ARTIFACTS / "shap_explain_X.csv")
    return sv, X

def fig5_beeswarm():
    """SHAP beeswarm on held-out decisions."""
    import shap

    sv, X = _shap_frame()
    base = np.load(ARTIFACTS / "shap_base_values_test.npy")
    expl = shap.Explanation(values=sv, base_values=base, data=X.to_numpy(),
                           feature_names=list(X.columns))
    fig = plt.figure(figsize=(7.6, 5.6))
    # Keeps SHAP's diverging default rather than the house sequential map: a beeswarm
    # encodes feature value on colour, and a single-hue ramp makes the low end vanish.
    shap.plots.beeswarm(expl, max_display=14, show=False, color_bar_label="Feature value")
    ax = plt.gca()
    ax.set_xlabel("SHAP value (log-odds contribution)")
    ax.set_title("Per-client attribution on 2,000 held-out decisions", loc="left",
                  fontsize=11, fontweight="bold", color=INK)
    ax.grid(axis="y", visible=False)
    fig.tight_layout()
    return save_fig(fig, "p2_fig5_shap_beeswarm")

def fig6_dependence():
    """PAY_0 dependence coloured by PAY_2 -- the model's internals against the EDA gradient."""
    sv, X = _shap_frame()
    p = result("exp11_shap")
    profile = p["config"]["pay0_profile"]
    j = list(X.columns).index("PAY_0")

    rng = np.random.default_rng(42)
    jitter = rng.uniform(-0.18, 0.18, len(X))

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12.4, 4.4),
                                   gridspec_kw={"width_ratios": [1.15, 1]})
    sc = ax1.scatter(X["PAY_0"] + jitter, sv[:, j], c=X["PAY_2"], cmap=SEQ_CMAP,
                     s=11, alpha=0.75, linewidths=0, zorder=3)
    ax1.axhline(0.0, color=AXIS, lw=1.0, zorder=2)
    cb = fig.colorbar(sc, ax=ax1, pad=0.02)
    cb.set_label("PAY_2 (previous month's status)", fontsize=9, color=INK_2)
    cb.outline.set_visible(False)
    ax1.set_xlabel("PAY_0 (most recent repayment status, jittered)")
    ax1.set_ylabel("SHAP value for PAY_0 (log-odds)")
    ax1.set_title("a. The model's response to PAY_0")

```

```

ax1.set_xticks(sorted(int(c) for c in profile))

codes = sorted(int(c) for c in profile)
rate = [100 * profile[str(c)]["observed_default_rate"] for c in codes]
mshap = [profile[str(c)]["mean_shap_logodds"] for c in codes]
# Codes 4, 5 and 8 hold a handful of clients each, so their rates are one or two
# people wide. Faded, so the eye does not read them as evidence.
sparse = [profile[str(c)]["n"] < 20 for c in codes]
ax2.bar(np.arange(len(codes)), rate, width=0.6, zorder=3,
        color=[CAT[0]] * len(codes),
        alpha=None, label="observed default rate (%)")
for bar, thin in zip(ax2.patches, sparse):
    if thin:
        bar.set_alpha(0.28)
ax2.set_xticks(np.arange(len(codes)),
               [f"{c}\nn={profile[str(c)]['n']:,}" for c in codes], fontsize=8)
ax2.set_ylabel("Observed default rate (%)")
ax2.set_xlabel("PAY_0 code")
ax2b = ax2.twinx()
ax2b.plot(np.arange(len(codes)), mshap, "o-", color=CRITICAL, lw=1.6, markersize=5,
          zorder=4, label="mean SHAP (log-odds)")
ax2b.set_ylabel("Mean SHAP for PAY_0 (log-odds)", color=CRITICAL)
ax2b.grid(False)
# Headroom so the legend cannot sit on the code-2 peak.
ax2.set_ylim(0, 128)
lo, hi = min(mshap), max(mshap)
ax2b.set_ylim(lo - 0.1 * (hi - lo), hi + 0.42 * (hi - lo))
dense = [c for c in codes if profile[str(c)]["n"] >= 20]
rho_dense = spearmanr([profile[str(c)]["mean_shap_logodds"] for c in dense],
                      [profile[str(c)]["observed_default_rate"] for c in dense]).statistic
ax2.set_title(f"b. Convergent validation: Spearman rho {rho_dense:.3f} "
              f"over the {len(dense)} codes with n >= 20")
ax2.grid(axis="x", visible=False)
ax2.set_axisbelow(True)
h1, l1 = ax2.get_legend_handles_labels()
h2, l2 = ax2b.get_legend_handles_labels()
ax2.legend(h1 + h2, l1 + l2, loc="upper left")
return save_fig(fig, "p2_fig6_shap_pay0_dependence")

```

----- appendix figures

```

def app1_optuna():
    """Optimisation history and hyperparameter importances."""
    import optuna

    db = ARTIFACTS / "optuna_study.db"
    study = optuna.load_study(study_name="xgb_roc_auc", storage=f"sqlite:///db")
    done = [t for t in study.trials if t.value is not None]
    values = [t.value for t in done]
    running_best = np.maximum.accumulate(values)

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.2))
    ax1.scatter(range(len(values)), values, s=16, color=MUTED, zorder=3, label="trial")
    ax1.plot(running_best, color=CAT[0], lw=1.8, zorder=4, label="best so far")
    pruned = [i for i, t in enumerate(study.trials)
               if t.state == optuna.trial.TrialState.PRUNED]
    ax1.set_xlabel(f"Completed trial ({len(pruned)} further trials pruned)")
    ax1.set_ylabel("Mean 5-fold ROC-AUC")
    ax1.set_ylim(min(values) - 0.002, max(values) + 0.002)
    ax1.set_title("a. TPE optimisation history")
    ax1.legend(loc="lower right")

    try:
        imp = optuna.importance.get_param_importances(study)
    except Exception:
        imp = {}
    if imp:
        names = list(imp)[::-1]
        ax2.barh(np.arange(len(names)), [imp[n] for n in names], color=CAT[0],
                 height=0.6, zorder=3)

```

```

        ax2.set_yticks(np.arange(len(names)), names)
        ax2.set_xlabel("fANOVA importance (share of objective variance)")
    ax2.set_title("b. Hyperparameter importance")
    ax2.grid(axis="y", visible=False)
    for ax in (ax1, ax2):
        ax.set_axisbelow(True)
    return save_fig(fig, "p2_app1_optuna")

```

```

def app2_learning_curve():
    """Train and validation ROC-AUC against training fraction."""
    p = result("exp08_learning_curve")
    rows = sorted(p["rows"], key=lambda r: r["fraction"])
    n = [r["n_train_rows_mean"] for r in rows]
    va = np.array([r["cv"]["roc_auc_mean"] for r in rows])
    va_sd = np.array([r["cv"]["roc_auc_sd"] for r in rows])
    tr = np.array([r["train_roc_auc_mean"] for r in rows])
    tr_sd = np.array([r["train_roc_auc_sd"] for r in rows])

    fig, ax = plt.subplots(figsize=(7.2, 4.4))
    ax.plot(n, tr, "o-", color=CAT[1], lw=1.6, markersize=5, label="training fold")
    ax.fill_between(n, tr - tr_sd, tr + tr_sd, color=CAT[1], alpha=0.16, lw=0)
    ax.plot(n, va, "o-", color=CAT[0], lw=1.8, markersize=5, label="validation fold")
    ax.fill_between(n, va - va_sd, va + va_sd, color=CAT[0], alpha=0.18, lw=0)
    ax.set_xlabel("Training rows used")
    ax.set_ylabel("ROC-AUC")
    ax.set_title("Learning curve, 5 fractions x 5 seeds (mean +/- SD)")
    ax.legend(loc="center right")
    ax.set_axisbelow(True)
    return save_fig(fig, "p2_app2_learning_curve")

```

```

def app3_recency():
    """How much repayment history is actually worth collecting."""
    p = result("exp07_recency")
    rows = sorted(p["rows"], key=lambda r: r["k_months"])
    full = rows[-1]["cv"]["roc_auc_mean"]
    sd = rows[-1]["cv"]["roc_auc_sd"]

    fig, ax = plt.subplots(figsize=(7.2, 4.2))
    k = [r["k_months"] for r in rows]
    m = [r["cv"]["roc_auc_mean"] for r in rows]
    ax.errorbar(k, m, yerr=[r["cv"]["roc_auc_sd"] for r in rows], fmt="o-",
                color=CAT[0], ecolor=INK_2, elinewidth=1.1, capsize=4, lw=1.8,
                markersize=5, zorder=4)
    ax.axhspan(full - sd, full + sd, color=CAT[0], alpha=0.12, lw=0, zorder=1)
    ax.axhline(full, color=AXIS, lw=1.1, zorder=2)
    ax.text(1.05, full + 0.0004, "all six months +/- 1 SD", fontsize=8, color=MUTED)
    ax.set_xticks(k)
    ax.set_xlabel("Most recent months of history retained")
    ax.set_ylabel("ROC-AUC")
    ax.set_title("Recency ablation (25 repeated-CV fits)")
    ax.set_axisbelow(True)
    return save_fig(fig, "p2_app3_recency")

```

```

def app4_groups():
    """Leave-one-group-out against only-one-group."""
    p = result("exp06_groups")
    by = {r["label"]: r for r in p["rows"]}
    allg = by["all_groups"]["cv"]["roc_auc_mean"]
    drop = sorted([r for r in p["rows"] if r["label"].startswith("drop_")],
                  key=lambda r: r["cv"]["roc_auc_mean"])
    only = sorted([r for r in p["rows"] if r["label"].startswith("only_")],
                  key=lambda r: r["cv"]["roc_auc_mean"])

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.2), sharex=True)
    errorbar_ladder(ax1, [r["label"].replace("drop_", "") for r in drop],
                    [r["cv"]["roc_auc_mean"] for r in drop],
                    [r["cv"]["roc_auc_sd"] for r in drop],
                    "a. Leave one group out", ref=allg)
    errorbar_ladder(ax2, [r["label"].replace("only_", "") for r in only],

```

```

                [r["cv"]["roc_auc_mean"] for r in only],
                [r["cv"]["roc_auc_sd"] for r in only],
                "b. One group only", ref=allg)
ax1.text(allg + 0.0015, 0.1, "all groups", fontsize=8, color=MUTED, rotation=90)
return save_fig(fig, "p2_app4_feature_groups")

```

```

def app5_waterfalls():
    """The most confident true positive beside the worst false negative."""
    import shap

    sv, X = _shap_frame()
    base = np.load(ARTIFACTS / "shap_base_values_test.npy")
    cases = result("exp11_shap")["config"]["waterfall_cases"]

    fig = plt.figure(figsize=(12, 5.2))
    for i, (key, title) in enumerate(((("true_positive", "a. Most confident true positive"),
                                       ("false_negative", "b. Worst false negative"))):
        idx = cases[key]["row"]
        plt.subplot(1, 2, i + 1)
        expl = shap.Explanation(
            values=sv[idx], base_values=float(np.atleast_1d(base)[0]),
            data=X.iloc[idx].to_numpy(), feature_names=list(X.columns))
        shap.plots.waterfall(expl, max_display=11, show=False)
        ax = plt.gca()
        ax.set_title(f"{title} (p = {cases[key]['proba']:.3f}, y = {cases[key]['y']})",
                    loc="left", fontsize=10, fontweight="bold", color=INK)
    fig.tight_layout()
    return save_fig(fig, "p2_app5_shap_waterfalls")

def app6_fairness():
    """Per-subgroup rates with Wilson intervals, and error rates at the frozen threshold."""
    p = result("exp13_fairness")
    detail = p["config"]["subgroup_detail"]
    attrs = ["SEX", "EDUCATION", "MARRIAGE", "AGE_BAND"]

    # Portrait, one attribute per row. A 2x4 landscape layout has to be scaled to about
    # half size to fit a page width, which puts the tick labels below 4pt in print.
    fig, axes = plt.subplots(4, 2, figsize=(9, 12.5))
    for col, attr in enumerate(attrs):
        groups = detail[f"full|{attr}"]
        names = [g["group"] for g in groups]
        x = np.arange(len(groups))
        rate = np.array([g["observed_default_rate_pct"] for g in groups])
        lo = np.array([g["wilson_lo_pct"] for g in groups])
        hi = np.array([g["wilson_hi_pct"] for g in groups])
        sel = np.array([100 * g["selection_rate"] for g in groups])

        ax = axes[col, 0]
        ax.bar(x - 0.19, rate, width=0.36, color=CAT[0], zorder=3, label="observed rate")
        ax.errorbar(x - 0.19, rate, yerr=[rate - lo, hi - rate], fmt="none",
                   ecolor=INK_2, elinewidth=1.0, capsize=3, zorder=4)
        ax.bar(x + 0.19, sel, width=0.36, color=CAT[1], zorder=3, label="model flag rate")
        ax.set_xticks(x, [f"{n}\nn={g['n']:,}" for n, g in zip(names, groups)],
                     rotation=25, ha="right", fontsize=7.5)
        ax.set_title(f"{attr} -- observed vs flagged", fontsize=10)
        ax.grid(axis="x", visible=False)
        ax.set_axisbelow(True)
        ax.set_ylabel("Percent")
        if col == 0:
            ax.legend(loc="upper left", fontsize=8)

        ax = axes[col, 1]
        for name, key, colour in ((("recall", "recall", CAT[0]), ("FPR", "fpr", CAT[3]),
                                   ("ROC-AUC", "roc_auc", CAT[2]))):
            vals = [g[key] for g in groups]
            ax.plot(x, vals, "o-", color=colour, lw=1.5, markersize=4, label=name)
        ax.set_xticks(x, names, rotation=25, ha="right", fontsize=7.5)
        ax.set_ylim(0, 1)
        ax.grid(axis="x", visible=False)
        ax.set_axisbelow(True)

```

```

        ax.set_title(f"{attr} -- at the frozen r={R_HEADLINE} threshold", fontsize=10)
        if col == 0:
            ax.legend(loc="upper left", fontsize=8)

    fig.suptitle("Subgroup performance, full model, out-of-fold training predictions",
                 x=0.005, ha="left", fontsize=11, fontweight="bold", color=INK)
    fig.tight_layout(rect=(0, 0, 1, 0.975))
    return save_fig(fig, "p2_app6_fairness")

def app7_importance_agreement():
    """Do the importance measures even agree on an ordering?"""
    p = result("exp12_stability")
    matrix = p["config"]["importance_agreement_matrix"]
    names = list(matrix)
    M = np.array([[matrix[a][b] for b in names] for a in names], dtype=float)

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.5, 4.8),
                                   gridspec_kw={"width_ratios": [1, 1.1]})
    im = ax1.imshow(M, cmap=SEQ_CMAP, vmin=0, vmax=1)
    ax1.set_xticks(np.arange(len(names)), names, rotation=35, ha="right")
    ax1.set_yticks(np.arange(len(names)), names)
    for i in range(len(names)):
        for j in range(len(names)):
            ax1.text(j, i, f"{M[i, j]:.2f}", ha="center", va="center", fontsize=8,
                     color="white" if M[i, j] > 0.6 else INK)
    ax1.set_title("a. Spearman rho between importance measures")
    ax1.grid(False)
    cb = fig.colorbar(im, ax=ax1, pad=0.02, shrink=0.85)
    cb.outline.set_visible(False)

    rows = sorted(p["rows"], key=lambda r: r["shap_rank"])[1:14]
    y = np.arange(len(rows))[:-1]
    ax2.barh(y, [r["rank_sd_across_seeds"] for r in rows], height=0.6,
             color=[CRITICAL if r["label"].startswith("BILL_AMT") else CAT[0]
                   for r in rows], zorder=3)
    ax2.set_yticks(y, [r["label"] for r in rows], fontsize=8)
    ax2.set_xlabel("SD of SHAP rank across 5 reseeded refits (positions)")
    ax2.set_title("b. Where the explanation is unstable (BILL_AMT in red)")
    ax2.grid(axis="y", visible=False)
    ax2.set_axisbelow(True)
    fig.tight_layout()
    return save_fig(fig, "p2_app7_importance_agreement")

def app8_engineered_encoding():
    """The two ablations the body only gets a sentence for."""
    eng = result("exp05_engineered")["rows"]
    enc = result("exp04_encoding")["rows"]

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 3.8))
    errorbar_ladder(ax1, [r["label"] for r in eng],
                    [r["cv"]["roc_auc_mean"] for r in eng],
                    [r["cv"]["roc_auc_sd"] for r in eng],
                    "a. Engineered features against the raw 23",
                    ref=next(r["cv"]["roc_auc_mean"] for r in eng
                             if r["label"] == "raw_23"))
    errorbar_ladder(ax2, [r["label"] for r in enc],
                    [r["cv"]["roc_auc_mean"] for r in enc],
                    [r["cv"]["roc_auc_sd"] for r in enc],
                    "b. Encoding of the ordinal PAY_n columns",
                    ref=next(r["cv"]["roc_auc_mean"] for r in enc
                             if r["label"] == "native_int"))
    fig.tight_layout()
    return save_fig(fig, "p2_app8_engineered_encoding")

def app9_seed_variance():
    """Seed spread and the PAY_0 ablation, the two robustness checks."""
    p = result("exp14_final")
    cfg = p["config"]
    aucs = cfg["seed_test_roc_auc"]

```

```

ab = cfg["pay0_ablation"]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 3.9),
                               gridspec_kw={"width_ratios": [1.1, 1]})
ax1.plot(np.arange(len(aucs)), aucls, "o", color=CAT[0], markersize=6, zorder=4)
mean = float(np.mean(aucs))
sd = cfg["seed_roc_auc_sd"]
ax1.axhspan(mean - sd, mean + sd, color=CAT[0], alpha=0.14, lw=0, zorder=1)
ax1.axhline(mean, color=AXIS, lw=1.1, zorder=2)
ax1.set_xticks(np.arange(len(aucs)))
ax1.set_xlabel("Refit (seed 42-51)")
ax1.set_ylabel("Test ROC-AUC")
ax1.set_title(f"a. Seed variance: {mean:.5f} +/- {sd:.5f}")

labels = ["all 23 features", "PAY_0 dropped"]
means = [ab["cv_roc_auc_full"], ab["cv_roc_auc_no_pay0"]]
sds = [ab["cv_sd_full"], ab["cv_sd_full"]]
ax2.bar(np.arange(2), means, yerr=sds, width=0.5, color=[CAT[0], CRITICAL], zorder=3,
        error_kw={"ecolor": INK_2, "elinewidth": 1.1, "capsize": 4})
ax2.set_ylim(min(means) - 0.03, max(means) + 0.01)
ax2.set_xticks(np.arange(2), labels)
ax2.set_ylabel("CV ROC-AUC (25 fits)")
ax2.set_title(f"b. PAY_0 ablation: {ab['paired_fold_test']['mean_delta']:+.4f} paired")
ax2.grid(axis="x", visible=False)
for ax in (ax1, ax2):
    ax.set_axisbelow(True)
fig.tight_layout()
return save_fig(fig, "p2_app9_robustness")

```

```

BUILDERS = {
    "fig1": fig1_ladder, "fig2": fig2_roc_pr, "fig3": fig3_reliability,
    "fig4": fig4_cost, "fig5": fig5_beeswarm, "fig6": fig6_dependence,
    "app1": app1_optuna, "app2": app2_learning_curve, "app3": app3_recency,
    "app4": app4_groups, "app5": app5_waterfalls, "app6": app6_fairness,
    "app7": app7_importance_agreement, "app8": app8_engineered_encoding,
    "app9": app9_seed_variance,
}

```

```

def main(names=None):
    todo = names or list(BUILDERS)
    built, skipped = [], []
    for name in todo:
        try:
            BUILDERS[name]()
            built.append(name)
        except FileNotFoundError as e:
            skipped.append(f"{name}: {e}")
        finally:
            plt.close("all")
    print(f"\nbuilt {len(built)}: {' '.join(built)}")
    for s in skipped:
        print(f"skipped {s}")

```

```

if __name__ == "__main__":
    main(sys.argv[1:] or None)

```

experiments/verify.py

```

"""Section 7 verification. Every claim the report rests on, checked mechanically.

```

```

Run last. Each check prints PASS or FAIL and the script exits non-zero if any failed, so
it is usable as a pre-submission gate rather than something to read and interpret.

```

```

python -m experiments.verify
"""

```

```

from __future__ import annotations

```

```

import json

```

```

import numpy as np

from src.config import ARTIFACTS, N_FOLDS, N_REPEATS, RESULTS, SEED, TEST_SIZE
from src.data import (RAW_FEATURES, TARGET, clean, cv_folds, load_raw, load_splits,
                      make_splits, repeated_cv_folds)

CHECKS = []

def check(name):
    def deco(fn):
        CHECKS.append((name, fn))
        return fn
    return deco

# ----- data integrity

@check("Phase 1 assertions still hold after cleaning")
def _phase1():
    df = clean(load_raw())
    assert len(df) == 29944, f"expected 29,944 rows, got {len(df):,}"
    assert int(df[TARGET].sum()) == 6622, f"expected 6,622 defaults, got {df[TARGET].sum():,}"
    assert set(df["EDUCATION"].unique()) <= {1, 2, 3, 4}, "EDUCATION domain drifted"
    assert set(df["MARRIAGE"].unique()) <= {1, 2, 3}, "MARRIAGE domain drifted"
    return f"29,944 rows / 6,622 defaults / prevalence {df[TARGET].mean():.4%}"

@check("No duplicate feature rows survive cleaning")
def _dedup():
    df = clean(load_raw())
    dups = int(df.duplicated(subset=list(RAW_FEATURES)).sum())
    assert dups == 0, f"{dups} duplicate feature rows remain -- they would straddle the split"
    return "0 duplicate feature-rows, so none can straddle train/test"

# ----- splits

@check("Split sizes, stratification and zero overlap")
def _splits():
    df = clean(load_raw())
    y = df[TARGET].to_numpy(dtype=int)
    s = load_splits()
    tr, te = s["train_idx"], s["test_idx"]

    assert len(tr) == 23955, f"train is {len(tr):,}, expected 23,955"
    assert len(te) == 5989, f"test is {len(te):,}, expected 5,989"
    assert len(np.intersect1d(tr, te)) == 0, "train and test indices overlap"
    assert len(tr) + len(te) == len(df), "split does not partition the frame"
    assert abs(len(te) / len(df) - TEST_SIZE) < 0.001, "test fraction drifted"

    gap = abs(y[tr].mean() - y[te].mean())
    assert gap < 0.001, f"stratification gap {gap:.5f} exceeds 0.1pp"
    assert int(y[te].sum()) == 1324, f"expected 1,324 test positives, got {y[te].sum():,}"
    return (f"23,955 / 5,989, overlap 0, prevalence gap {gap:.5f}, "
            f"{int(y[te].sum()):,} test positives")

@check("Fold structures partition the training rows")
def _folds():
    df = clean(load_raw())
    y = df[TARGET].to_numpy(dtype=int)
    s = load_splits()
    y_tr = y[s["train_idx"]]

    folds = cv_folds(s)
    assert len(folds) == N_FOLDS

```

```

seen = np.concatenate([va for _, va in folds])
assert len(seen) == len(y_tr) and len(np.unique(seen)) == len(y_tr), \
    "tuning folds do not partition the training rows exactly once"
for tr, va in folds:
    assert len(np.intersect1d(tr, va)) == 0, "a tuning fold overlaps its own training half"

rfolds = repeated_cv_folds(s)
assert len(rfolds) == N_FOLDS * N_REPEATS, f"expected 25 folds, got {len(rfolds)}"
gaps = [abs(y_tr[va].mean() - y_tr.mean()) for _, va in rfolds]
assert max(gaps) < 0.01, f"repeated-CV stratification gap {max(gaps):.4f} too large"
return f"5 tuning folds + 25 variance folds, max stratification gap {max(gaps):.5f}"

@check("Splits reproduce exactly on a re-run with the same SEED")
def _reproducible():
    before = load_splits()
    after = make_splits(force=True)
    for k in ("train_idx", "test_idx", "fold_id", "rep_fold_id"):
        assert np.array_equal(before[k], after[k]), f"{k} changed on re-run with SEED={SEED}"
    return f"all four index arrays identical after force-rebuild at SEED={SEED}"

# ----- result contract

@check("Every results/*.json validates against the section 3.2 schema")
def _schema():
    paths = sorted(RESULTS.glob("exp*.json"))
    assert paths, "no result files found -- run the experiments first"
    top = {"experiment_id", "title", "run_utc", "seed", "versions",
           "runtime_sec", "config", "rows", "notes"}
    cv_keys = {"roc_auc_mean", "roc_auc_sd", "pr_auc_mean", "pr_auc_sd",
               "brier_mean", "brier_sd", "n_fits"}
    for p in paths:
        d = json.loads(p.read_text(encoding="utf-8"))
        missing = top - set(d)
        assert not missing, f"{p.name} missing top-level keys {missing}"
        assert d["notes"].strip(), f"{p.name} has an empty notes field"
        assert d["rows"], f"{p.name} has no rows"
        labels = [r["label"] for r in d["rows"]]
        assert len(labels) == len(set(labels)), f"{p.name} has duplicate labels"
        for r in d["rows"]:
            if r.get("cv") is not None:
                absent = cv_keys - set(r["cv"])
                assert not absent, f"{p.name}:{r['label']} cv missing {absent}"
    return f"{len(paths)} result files, all conformant, all with non-empty notes"

@check("Frozen thresholds are scalars strictly inside (0, 1)")
def _thresholds():
    found = 0
    for p in sorted(RESULTS.glob("exp*.json")):
        d = json.loads(p.read_text(encoding="utf-8"))
        for r in d["rows"]:
            for key in ("oof_threshold", "tau_empirical"):
                if isinstance(r.get(key), (int, float)):
                    tau = float(r[key])
                    assert 0.0 < tau < 1.0, f"{p.name}:{r['label']} {key}={tau} out of range"
                    found += 1
            t = r.get("test")
            if isinstance(t, dict) and isinstance(t.get("threshold"), (int, float)):
                tau = float(t["threshold"])
                assert 0.0 < tau < 1.0, f"{p.name}:{r['label']} test threshold {tau} out of range"
                found += 1
    if not found:
        return "SKIP -- no thresholds yet (run exp03_imbalance and exp10_cost)"
    return f"{found} thresholds, all scalar and inside (0, 1)"

@check("Only the licensed experiments populate a test block")
def _test_discipline():
    licensed = {"exp14_final"}

```

```

offenders = []
for p in sorted(RESULTS.glob("exp*.json")):
    d = json.loads(p.read_text(encoding="utf-8"))
    if d["experiment_id"] in licensed:
        continue
    for r in d["rows"]:
        if r.get("test"):
            offenders.append(f"{d['experiment_id']}:{r['label']}")
assert not offenders, f"unlicensed test blocks: {offenders}"
return "every test block outside exp14_final is null"

@check("SHAP values sum to the model margin")
def _shap_additivity():
    path = RESULTS / "exp11_shap.json"
    if not path.exists():
        return "SKIP -- results/exp11_shap.json absent (run exp11_shap)"
    cfg = json.loads(path.read_text(encoding="utf-8"))["config"]
    err = float(cfg["additivity_max_error"])
    assert err < 1e-4, f"max additivity error {err:.3e} exceeds 1e-4"

    # Recompute independently from the cached values, so this is not merely a
    # restatement of the assertion exp11 already made.
    sv = np.load(ARTIFACTS / "shap_values_test.npy")
    base = np.load(ARTIFACTS / "shap_base_values_test.npy")
    n_feat = sv.shape[1]
    assert n_feat == len(RAW_FEATURES), f"SHAP matrix has {n_feat} columns"
    spread = float(np.ptp(base))
    assert spread < 1e-6, f"base value is not constant (spread {spread:.2e})"
    return (f"exp11 recorded max error {err:.2e} in log-odds space over "
            f"{len(sv):,} held-out rows x {n_feat} features; base value constant")

# ----- leakage audit

@check("Leakage audit: the hold-out is imported only where licensed")
def _leakage():
    from pathlib import Path
    licensed = {"exp11_shap.py", "exp12_stability.py", "exp14_final.py",
               "figures.py", "verify.py"}
    exp_dir = Path(__file__).resolve().parent
    offenders = {}
    for p in sorted(exp_dir.glob("*.py")):
        if p.name in licensed:
            continue
        text = p.read_text(encoding="utf-8")
        hits = [tok for tok in ("test_context", "X_test", "y_test", "test_idx")
                 if tok in text]
        if hits:
            offenders[p.name] = hits
    assert not offenders, f"hold-out referenced in selection code: {offenders}"
    return ("no tuning or threshold-selection script references the hold-out; "
            f"licensed: {' '.join(sorted(licensed - {'verify.py'}))}")

# ----- driver

def main():
    width = 62
    passed = failed = skipped = 0
    print("=" * width)
    print("Section 7 verification")
    print("=" * width)
    for name, fn in CHECKS:
        try:
            detail = fn()
        except AssertionError as e:
            print(f"FAIL {name}\n      {e}")
            failed += 1
        except Exception as e:
            # missing artifact, bad JSON -- report, do not crash

```

```
        print(f"ERROR {name}\n      {type(e).__name__}: {e}")
        failed += 1
    else:
        if isinstance(detail, str) and detail.startswith("SKIP"):
            print(f"SKIP  {name}\n      {detail[7:]}")
            skipped += 1
        else:
            print(f"PASS  {name}\n      {detail}")
            passed += 1
    print("=" * width)
    print(f"{passed} passed, {failed} failed, {skipped} skipped")
    if failed:
        raise SystemExit(1)

if __name__ == "__main__":
    main()
```